

The BEAM Book

Implementation Deep Dives

Erik Stenman

Table of Contents

About These Notes	1
1. Alternative Runtimes and Embedded Platforms	2
1.1. AtomVM: Erlang on Microcontrollers	2
1.2. WebAssembly: AtomVM, Popcorn and Other Routes	3
1.3. Earlier Approaches: Ling, Erjang and Firefly	4
1.4. ERTS as the Whole Machine: Nerves, GRiSP and Tyn	5
2. Parse Transforms, Leex and Yecc	10
2.1. Writing a Parse Transform	10
2.2. Other Compiler Tools	16
2.3. Syntax Tools and Merl	22
3. Compiler Analysis and Encoding Details	23
3.1. SSA pass ordering	23
3.2. Type lattice and serialized type layout	23
3.3. Alias-analysis bookkeeping	24
3.4. Record-update compiler and backend passes	24
3.5. Assembly pass catalogue	25
4. Inspecting Process Structures and Heaps	26
4.1. Optional: Inspecting a Heap with ETP (OTP 24)	26
4.2. Reading the PCB dump	28
4.3. GC fields in the PCB	30
4.4. Process dictionary buckets	31
4.5. Dictionary insertion and update mechanics	31
4.6. Complete process backtrace	32
5. Term Tags and Heap Object Encodings	34
5.1. The compiler type lattice and serialized entries	34
5.2. Immediate tag layout	34
5.3. PID bit fields	35
5.4. Small integers and secondary immediate tags	35
5.5. Atom table structure	35
5.6. Atom comparison implementation note	36
5.7. Header tags and their history	37
5.8. Bitstring header fields	38
5.9. Bignum encoding and arithmetic source guide	39
5.10. Reference encoding	39

5.11. Native record instance and definition layout	40
6. Building a Bytecode Interpreter and Threaded Dispatcher	42
6.1. Bytecode emulation	42
6.2. Threaded Code	44
6.3. The Real BEAM	46
7. AtomVM Execution and Compatibility	50
8. BEAM Chunk Layouts and the Complete Reader	52
8.1. The BEAM File Format	52
8.2. Running the Complete Reader	72
9. Implementation detail: the BEAM loader	75
9.1. Source Files and Generated Tables	75
9.2. Parsing and Decoding	77
9.3. Transformation Engine and Rules	78
9.4. Implementing Specific Instructions in the Interpreter	82
9.5. Handing Over to the JIT	85
9.6. Code Publication Internals	87
9.7. Exact Selection Rules and Assembly	88
9.8. Literal-key Generator	89
10. JIT Implementation: Emitters and Native-Code History	91
10.1. Earlier execution engines	91
10.2. Inspecting interpreter handlers	91
10.3. Stack safety and the history of native Erlang	92
10.4. Runtime stack-switch helpers	94
10.5. Interpreter and JIT register conventions	94
10.6. Emitters, global fragments and moves	95
10.7. Type-chunk encoding	102
10.8. Record-update loader rules	103
10.9. Copy-path assembly	103
10.10. Reuse-path assembly	104
10.11. In-place safety-check assembly	104
10.12. High-water source comment	105
10.13. Destructive-update compiler rewriting	105
10.14. Fun layouts and header encoding	105
10.15. Creating and calling fun objects	107
10.16. Captured binary-match assembly	108
10.17. Binary extraction shortcuts	110

11. Scheduler Implementation: Counters, Timers and Migration	111
11.1. Reduction-counter fields	111
11.2. Atomic process-state flags	111
11.3. Timing wheels and timer entries	112
11.4. Interpreter and JIT scheduler entry points	114
11.5. Task-stealing lock and search mechanics	114
11.6. Migration planning and queue limits	115
12. Memory Implementation: Allocator Reference and Term Layouts	118
12.1. Allocator catalogue	118
12.2. System allocation and segment-cache parameters	119
12.3. Carrier-size defaults and growth	120
12.4. Allocation strategies and the full allocator catalogue	120
12.5. Allocator flag reference and original tuning examples	127
12.6. Fun-object C layout	132
12.7. Repeated tuple-update invariants	132
12.8. Message-fragment allocation and layout	133
12.9. Message container and signal-queue pointers	133
12.10. Atom-table allocation classes	134
13. ETS Implementation: Structures, Locks and Matching	135
13.1. Table headers and status flags	135
13.2. Magic references and the meta name table	136
13.3. ETS BIF table lookup	136
13.4. Ownership lists and yielding deletion	136
13.5. DbTerm storage layout	137
13.6. Compressed-table encoding	137
13.7. Hash-table buckets, indexing and splitting	138
13.8. Adaptive-lock resizing thresholds	138
13.9. Hash growth counters and scheduler groups	139
13.10. AVL traversal stacks	139
13.11. Contention-adapting node statistics and reclamation	140
13.12. The pattern-matching machine and compiled program	140
13.13. Match pseudo processes	141
13.14. Continuation fields	141
13.15. Persistent-term key-table representation	141
14. I/O Implementation: Requests and Polling	142
14.1. I/O protocol request catalogue	142

14.2. Port-task queue mechanics	144
14.3. Scheduler-pollset migration threshold	144
15. Distribution Implementation: Remote-Spawn Controls	145
15.1. Remote-spawn control messages	145
16. Native Implementation: Runtime Extensions and Main-Thread Integration	146
16.1. Borrowing the macOS main thread	146
16.2. Implementing and building a yielding factorial BIF	147
17. Shell Setup and Connection Exercises	153
17.1. Connecting three nodes	153
17.2. Shell configuration exercises	155
17.3. BREAK build and signal options	159
18. Application Debugging Background	160
18.1. Application debugging background	160
18.2. Heap-setting background	163
18.3. Exceptions and supervision background	164
19. BeamAsm Profiling Metadata Options	167
20. Additional Runtime Settings	168
20.1. Memory tuning introduction	168
20.2. Small-footprint settings and representation details	168
20.3. Super carriers and locked mappings	170
20.4. Scheduler wakeup threshold	170
20.5. Scheduler binding type catalogue	170
20.6. Native-code queue-scan workaround	170
20.7. Time correction compatibility settings	171
Appendix A: Third-party Notices	172
A.1. Erlang/OTP Excerpts	172
A.2. Source Copyright Notices	172
A.3. Apache License, Version 2.0	173
References	178

About These Notes

This companion preserves specialised implementation material from the second edition. The main book follows the consequences for application code; these notes are for investigating a particular runtime mechanism. Internal structures and generated code retain their release qualifications.

© 2013–2026 Erik Stenman and contributors. Material reused from the first edition retains its Creative Commons Attribution 4.0 licence. New second-edition material is © 2026 Erik Stenman, all rights reserved. Erlang/OTP excerpts are © Ericsson AB and contributors under Apache 2.0. Source notes, editorial changes and the complete licence appear in [Appendix A](#). Contributor attribution remains in the main book and its first-edition preface.

Alternative Runtimes and Embedded Platforms

These optional implementation details accompany the developer-facing chapters. The examples retain their original OTP version qualifications.

1.1. AtomVM: Erlang on Microcontrollers

AtomVM[\[S1\]](#) brings Erlang processes and message passing to small embedded systems, including ESP32, STM32, and Raspberry Pi Pico boards. Its runtime and selected libraries fit devices whose memory budget makes a conventional ERTS installation impractical. Unix and WebAssembly ports also let you try the application before moving it to the target board.

Consider a sensor application. You can give one process the job of reading a peripheral and send its measurements to another process that handles network communication. We still have processes sending messages, as we did on ERTS. AtomVM supplies its own scheduler, garbage collector, and heap layout, along with processes and monitors; ERTS's native extensions do not become part of that runtime.

To build the application, you start with the standard Erlang or Elixir compiler on your development machine. AtomVM's tools package the resulting `.beam` modules and required libraries into `.avm` archives for deployment. On the board, the sensor process reaches its hardware through AtomVM's platform APIs, which include GPIO, I2C, SPI, and UART support. Check the APIs and drivers for your chosen board in the project overview[\[S2\]](#) and getting-started guide[\[S3\]](#). The available functions and deployment steps depend on the platform and AtomVM release you use.

[Figure 1](#) compares the layers that support those processes. The edge between the library layer and the runtime layer is each runtime's bytecode and API boundary; matching file formats do not make the implementations interchangeable.

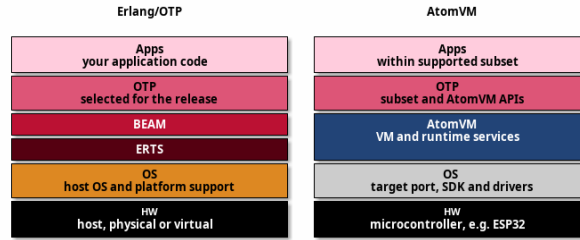


Figure 1. ERTS and AtomVM machine stacks: a typical OS host and an embedded target

The left stack contains standard ERTS and the OTP libraries selected for an application release. The right stack contains AtomVM’s runtime and its supported library subset, with hardware access supplied by its platform port. That lower layer varies by target; the diagram does not require every board to run Linux or the same RTOS. AtomVM’s Unix and WebAssembly ports instead use their host environments, as described in the platform build guide[\[S4\]](#).

Before taking the sensor application to a board, check both the compiler version and the operations its modules use. AtomVM 0.6.6 documents compiler versions OTP 21 through 27; the 0.7.0-alpha.1 documentation lists OTP 26 through 29. Those lists tell you which compilers were tested with each release; both also warn that some BEAM instructions remain unsupported. Even if AtomVM accepts your OTP 29 module, a function it calls may still be missing. Check its dependencies against the selected release’s 0.6.6 notes[\[S5\]](#), 0.7.0-alpha.1 notes[\[S6\]](#), and documented differences[\[S7\]](#) and test the operations you need; compiler support does not imply the full OTP 29 runtime or library is available.

We will follow the same application down into the runtime in [AtomVM: Reimplementing the BEAM for Small Systems](#), where we look at how AtomVM executes the code with its interpreter and newer native-code options.

1.2. WebAssembly: AtomVM, Popcorn and Other Routes

A browser cannot load a `.beam` file on its own. One route is to compile AtomVM to WebAssembly, then let that runtime execute your BEAM code. The application modules still contain BEAM bytecode; the runtime that executes them is the Wasm program. AtomVM documents this route for both browsers and Node.js in its WebAssembly getting-started guide[\[S8\]](#). The available libraries and host services follow this AtomVM port, so check those before moving a program from ERTS.

Popcorn, from Software Mansion, adds the tools for using this route from an Elixir web application. Its stable releases package the code for AtomVM and provide JavaScript interoperability; the 0.4 line moves to a WebAssembly build of OTP's own BEAM instead. Its architecture guide[S9] shows the Elixir bytecode bundle running on AtomVM/Wasm in an iframe, with communication to the JavaScript application in the main window. It also describes patches to the bundled Erlang and Elixir libraries for functionality that AtomVM does not supply. Start with the project examples[S10] and check its supported libraries before bringing your own dependencies.

Other Wasm projects make different choices. The experimental `erwasm`[S11] translates Erlang compiler assembly output into WebAssembly text and supplies a small runtime of its own. Its documented limitations include no garbage collector or proper scheduler. The separate `erwasm-http`[S12] project explores WASI HTTP handlers using WasmFX tooling. These are compiler and runtime experiments, with a much narrower scope than the runtime used throughout this book.

Lunatic[S13] applies Erlang's ideas about isolated processes, supervision and messages to programs compiled to WebAssembly. It executes Wasm modules rather than BEAM bytecode. Our process model can help you understand its design, but its processes and memory layout belong to a different runtime.

1.3. Earlier Approaches: Ling, Erjang and Firefly

Ling, the runtime behind Erlang on Xen[S14], illustrates an earlier route: packaging Erlang modules with a replacement runtime that boots as a Xen guest. Its architectural distinction is the removal of a general-purpose guest operating system beneath the Erlang runtime.

Erjang[S15] takes a different route by translating `.beam` modules into Java `.class` files and implementing runtime services on the JVM. These projects provide historical context for alternative implementations; their architectural ideas do not establish compatibility with current OTP 29 applications.

Firefly, originally called Lumen, replaced both the compiler and the runtime. DockYard developed it around a runtime written in Rust, with native code and WebAssembly as targets. The repository[S16] was archived on 10 June 2024. Its README marks the installation section as a placeholder and says the Rebar and Mix integration was never provided; its executable-build instructions name native x86-64 and AArch64 targets. Read it as an exploration of another compiler and runtime design, rather than a completed way to

run an OTP application in a browser. DockYard’s 2022 announcement[\[S17\]](#) records the project’s goals.

1.4. ERTS as the Whole Machine: Nerves, GRiSP and Tyn

Return to our sensor application. If it needs the OTP libraries and ERTS behavior we used on the development machine, we can keep that runtime and change the platform beneath it. Nerves supplies embedded Linux, GRiSP can run ERTS on RTEMS, and Tyn places it above a cloud hypervisor. These choices leave different work for the operating system and the application release.

Nerves[\[S18\]](#) packages the BEAM for embedded Linux devices, with Elixir as its front language. A Nerves firmware image contains a minimal Linux kernel built with Buildroot, just enough of a userland to boot, and your OTP release; the Erlang runtime is started as one of the first OS processes and supervises the system from then on. There is no package manager, no shell-based init system, and no on-device compiler. Complete firmware images start in the 20 to 30 MB range and are written to an inactive partition and activated atomically, so a failed update falls back to the previous image. Officially supported targets include most Raspberry Pi models, BeagleBone Black, generic x86_64, and the GRiSP 2 board.

Follow the sensor application down through [Figure 2](#). Its Elixir code and OTP libraries still run on ERTS. Below that runtime, Nerves keeps a small Linux system with the libraries and utilities the firmware needs. Buildroot constructs that system on the build host; it is not another runtime layer on the board. The Nerves getting-started guide[\[S19\]](#) shows how Mix assembles the application release and target system into firmware.

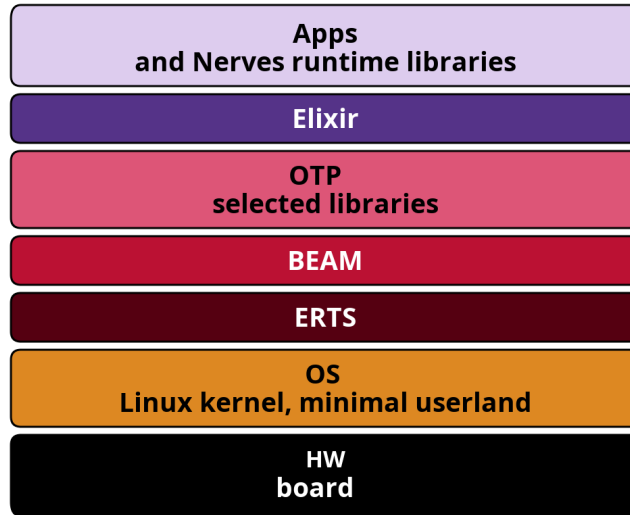


Figure 2. Nerves runtime stack on an embedded Linux target

GRiSP is developed by Peer Stritzinger GmbH[S20] in Germany. Its original approach, now called GRiSP Metal, runs ERTS on RTEMS, without Linux. RTEMS supplies operating-system services and support for the board. This is what "BEAM on bare metal" means here. The `grisp` runtime library exposes hardware interfaces to Erlang code.

Now compare the bottom of Figure 3 with the Nerves stack. ERTS remains above the operating system. Erlang applications use the OTP libraries directly; Elixir applications add the Elixir libraries above them. The GRiSP project[S21] supplies Rebar3 and Mix plugins for building and deploying from the development machine. Those tools are outside the runtime stack shown here.

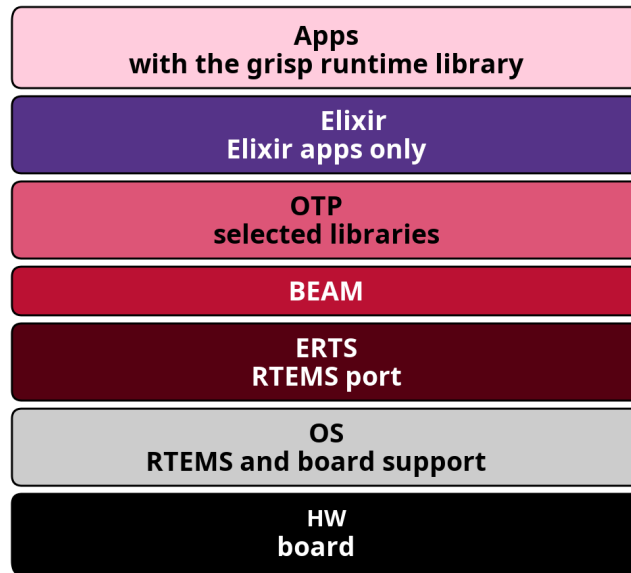


Figure 3. GRiSP RTEMS runtime stack (GRiSP Metal)

The RTEMS kernel's real-time facilities do not give arbitrary Erlang code a hard deadline guarantee. ERTS still schedules Erlang processes and collects their heaps. Choose the port, drivers and OTP release for your target, then check the timing requirements of the sensor application. A GRiSP board can also be a Nerves target, so the board name alone does not tell you which software stack is running.

The Linux alternatives keep a different bottom half of the stack. GRiSP Alloy[S22] uses Buildroot-based real-time Linux; GRiSP Forge uses Yocto-based real-time Linux. Both retain ERTS above Linux and use Linux drivers and system services.

Tyn[S23] points the same idea at the cloud instead of at hardware. Tyn is a unikernel: a single-purpose kernel, written in Rust, whose only job is to host one BEAM node. There is no Linux underneath; the kernel boots on a hypervisor (KVM/QEMU, and AWS Nitro instances) and starts an unmodified ERTS directly. The project demonstrates a stock Phoenix web application running on the OTP 27 BEAM this way. Tyn is a young project with sharp edges the README lists itself, and it is included here as a signpost rather than a recommendation: it shows that "the BEAM as the whole machine" is now being explored above the hypervisor as well as on embedded boards.

For the sensor board, check the peripheral drivers and deployment tools alongside the runtime. Nerves keeps a Linux layer; the RTEMS GRISP stack removes it while exposing board interfaces through Erlang APIs. For the receiving server, Tyn replaces that general-purpose OS with a kernel built to host the node. All three retain ERTS's scheduler, memory allocators and garbage collector, although their platform services and available tools differ.

If the board cannot fit ERTS, the AtomVM choice in [Section 1.1](#) changes the runtime as well. Follow [AtomVM: Reimplementing the BEAM for Small Systems](#) when you need to check what that means for the processes in the same sensor application.

Sources and links

- [S1] AtomVM Project. AtomVM. Project website. <https://atomvm.org/>
- [S2] AtomVM Project. AtomVM Documentation. AtomVM 0.7.0-alpha.1. <https://doc.atomvm.org/v0.7.0-alpha.1/index.html>
- [S3] AtomVM Project. Getting Started Guide. AtomVM 0.7.0-alpha.1. <https://doc.atomvm.org/v0.7.0-alpha.1/getting-started-guide.html>
- [S4] AtomVM Project. Build Instructions. AtomVM 0.7.0-alpha.1. <https://doc.atomvm.org/v0.7.0-alpha.1/build-instructions.html>
- [S5] AtomVM Project. Release Notes. AtomVM 0.6.6. <https://doc.atomvm.org/v0.6.6/release-notes.html>
- [S6] AtomVM Project. Release Notes. AtomVM 0.7.0-alpha.1. <https://doc.atomvm.org/v0.7.0-alpha.1/release-notes.html>
- [S7] AtomVM Project. Differences between AtomVM and BEAM. AtomVM 0.7.0-alpha.1. <https://doc.atomvm.org/v0.7.0-alpha.1/differences-with-beam.html>
- [S8] AtomVM Project. Getting Started Guide. AtomVM 0.6.6; Getting Started with AtomVM WebAssembly. <https://doc.atomvm.org/v0.6.6/getting-started-guide.html#getting-started-with-atomvm-webassembly>
- [S9] Software Mansion. Popcorn's Architecture. Popcorn documentation. <https://popcorn.hexdocs.pm/architecture.html>
- [S10] Software Mansion. Popcorn: running Elixir in the browser. <https://github.com/software-mansion/popcorn>
- [S11] erwasm Project. erwasm. <https://github.com/erwasm/erwasm>
- [S12] erwasm Project. erwasm-http. <https://github.com/erwasm/erwasm-http>
- [S13] Lunatic Project. Lunatic: Erlang-inspired runtime for WebAssembly. <https://github.com/lunatic-solutions/lunatic>
- [S14] Cloudozer. Ling: Erlang on Xen. <https://github.com/cloudozer/ling>
- [S15] Erjang Project. Erjang: a JVM-based Erlang VM. <https://github.com/trifork/erjang>
- [S16] Firefly Project. Firefly: alternative BEAM implementation. <https://github.com/GetFirefly/firefly>
- [S17] DockYard. DockYard R&D: Firefly Optimizes Your Elixir Compilation. September 1, 2022. <https://dockyard.com/blog/2022/09/01/dockyard-r-d-firefly-optimizes-your-elixir-compilation>
- [S18] Nerves Project. Nerves. Project website. <https://nerves-project.org/>
- [S19] Nerves Project. Getting Started. Nerves documentation. <https://nerves.hexdocs.pm/getting-started.html>

[S20] GRiSP Project. GRiSP. Embedded Erlang and Elixir ecosystem. <https://www.grisp.org/>

[S21] GRiSP Project. GRiSP Erlang Runtime Library. <https://github.com/grisp/grisp>

[S22] GRiSP Project. Software Stacks for Erlang & Elixir. Software. <https://www.grisp.org/software>

[S23] Tyn Project. Tyn kernel. <https://github.com/tyn-os/kernel>

Parse Transforms, Leex and Yecc

These optional implementation details accompany the developer-facing chapters. The examples retain their original OTP version qualifications.

2.1. Writing a Parse Transform

The easiest way to tweak the Erlang language is through Parse Transformations (or parse transforms). Parse Transformations come with all sorts of warnings, like this note in the OTP documentation:

Programmers are strongly advised not to engage in parse transformations and no support is offered for problems encountered.

— Erlang/OTP documentation

When you use a parse transform you are basically writing an extra pass in the compiler and that can if you are not careful lead to very unexpected results. But to use a parse transform you have to declare the usage in the module using it, and it will be local to that module, so as far as compiler tweaks goes this one is relatively safe.

The biggest problem with parse transforms as I see it is that you are inventing your own syntax, and it will make it more difficult for anyone else reading your code. At least until your parse transform has become as popular and widely used as e.g. QLC.

OK, so you know now that you shouldn't use it, but if you have to anyway, let's give you an example of how to do it.

Let's say for example that you for some reason would like to write JSON code directly in your Erlang code, then you are in luck since the tokens of JSON and of Erlang are basically the same. Also, since the Erlang compiler does most of its sanity checks in the linter pass which follows the parse transform pass, you can allow an AST which does not represent valid Erlang.

To write a parse transform you need to write an Erlang module (let's call it *p*) which exports the function `parse_transform/2`. This function is called by the compiler during the

parse transform pass if the module being compiled (let's call it *m*) contains the compiler option `{parse_transform, p}`. The arguments to the function are the AST of the module *m* and the compiler options given to the call to the compiler.



Important

Compiler options passed to a parse transform

You will not get any compiler options given in the file, which is a bit of a nuisance since you can't give options to the parse transform from the code.

The compiler does not collect the options given in `-compile` attributes until the `compile_directives` pass, which runs after the parse transforms.

The documentation of the abstract format is somewhat dense and it is quite hard to get a grip on the abstract format by reading the documentation. I encourage you to use the *syntax_tools* and especially `erl_syntax_lib` for any serious work on the AST.

Here we will develop a simple parse transform just to get an understanding of the AST. Therefore we will work directly on the AST and use the old reliable `io:format` approach instead of *syntax_tools*.

First we create an example of what we would like to be able to compile `json_test.erl`:

```
-module(json_test).
-compile({parse_transform, json_parser}).
-export([test/1]).

test(V) ->
  <<{{
    "name" : "Jack (\\"Bee\\" ) Nimble",
    "format": {
      "type"      : "rect",
      "widths"    : [1920,1600],
      "height"    : (-1080),
      "interlace" : false,
      "frame rate": V
    }
  }}>>.
```

Then we create a minimal parse transform module `json_parser.erl`:

```
-module(json_parser).
-export([parse_transform/2]).

parse_transform(AST, _Options) ->
  io:format("~p~n", [AST]),
  AST.
```

This identity parse transform returns an unchanged AST but it also prints it out so that you can see what an AST looks like.

```
1> c(json_parser).
{ok,json_parser}
2> c(json_test).
[{attribute,{1,1},file,{"json_test.erl",1}},
 {attribute,{1,2},module,json_test},
 {attribute,{3,2},export,[{test,1}]},
 {function,
  {5,1},
  test,1,
  [{clause,
    {5,1},
    [{var,{5,6},'V'}],
    [],
    [{bin,
      {6,5},
      [{bin_element,
        {6,7},
        {tuple,
          {6,7},
          [{tuple,
            {6,8},
            [{remote,
              {7,15},
              {string,{7,7},"name"},
              {string,{7,17},"Jack (\\"Bee\\" ) Nimble"}},
              {remote,
                {8,15},
                {string,{8,7},"format"},
                {tuple,
                  {8,17},
                  [{remote,{9,31},{string,{9,19},"type"},{string,{9,33},"rect"}},
                   {remote,
                     {10,32},
                     {string,{10,19},"widths"},
                     {cons,
                       {10,34},
```

```

        {integer,{10,35},1920},
        {cons,{10,40},{integer,{10,40},1600},{nil,{10,44}}}},
    {remote,
     {11,31},
     {string,{11,19},"height"},
     {op,{11,34},'-',{integer,{11,35},1080}}},
    {remote,
     {12,31},
     {string,{12,19},"interlace"},
     {atom,{12,33},false}},
    {remote,
     {13,31},
     {string,{13,19},"frame rate"},
     {var,{13,33},'V'}}]]]]]],
    default,default]]]]]],
{eof,{16,1}}]
json_test.erl:7:7: illegal expression
%   7|         "name"  : "Jack (\\"Bee\\" ) Nimble",
%   |         ^

json_test.erl:8:7: illegal expression
%   8|         "format": {
%   |         ^

json_test.erl:5:6: Warning: variable 'V' is unused
%   5| test(V) ->
%   |         ^

error

```

The parser has produced an AST, but the linter rejects these forms as illegal Erlang expressions. Now that we can see the AST, we can write functions to traverse it and rewrite the JSON code into Erlang code.[\[S1\]](#)

```

-module(json_parser).
-export([parse_transform/2]).

parse_transform(AST, _Options) ->
    json(AST, []).

-define(FUNCTION(Clauses), {function, Label, Name, Arity, Clauses}).

%% We are only interested in code inside functions.
json([?FUNCTION(Clauses) | Elements], Res) ->
    json(Elements, [?FUNCTION(json_clauses(Clauses)) | Res]);
json([Other|Elements], Res) -> json(Elements, [Other | Res]);
json([], Res) -> lists:reverse(Res).

```

```

%% We are interested in the code in the body of a function.
json_clauses([clause, CLine, A1, A2, Code] | Clauses) ->
    [{clause, CLine, A1, A2, json_code(Code)} | json_clauses(Clauses)];
json_clauses([]) -> [].

-define(JSON(Json), {bin, _, [{bin_element
                                ' _
                                , {tuple, _, [Json]}
                                ' _
                                , _}]}).

%% Recognize wrappers among the direct expressions of a function body.
json_code([]) -> [];
json_code([?JSON(Json)|MoreCode]) -> [parse_json(Json) | json_code(MoreCode)
];
json_code([Other|MoreCode]) -> [Other | json_code(MoreCode)].

%% Json Object -> [{}] | [{Label, Term}]
parse_json({tuple, Line, []}) ->
    {cons, Line, {tuple, Line, []}, {nil, Line}};
parse_json({tuple, Line, Fields}) -> parse_json_fields(Fields, Line);
%% Json Array -> List
parse_json({cons, Line, Head, Tail}) -> {cons, Line, parse_json(Head),
                                           parse_json(Tail)};

parse_json({nil, Line}) -> {nil, Line};
%% Json String -> UTF-8 binary
parse_json({string, Line, String}) -> str_to_bin(String, Line);
%% Json Integer -> Integer
parse_json({integer, Line, Integer}) -> {integer, Line, Integer};
%% Json Float -> Float
parse_json({float, Line, Float}) -> {float, Line, Float};
%% Json Constant -> true | false | null
parse_json({atom, Line, true}) -> {atom, Line, true};
parse_json({atom, Line, false}) -> {atom, Line, false};
parse_json({atom, Line, null}) -> {atom, Line, null};

%% Variables, should contain Erlang encoded Json
parse_json({var, Line, Var}) -> {var, Line, Var};
%% Json Negative Integer or Float
parse_json({op, Line, '-', {Type, _, N}}) when Type == integer
    ; Type == float ->
    {Type, Line, -N}.
%% parse_json(Code)
-> io:format("Code: ~p~n", [Code]), Code.

-define(FIELD(Label, Code),
    {remote, L, {string, _, Label}, Code}).

parse_json_fields([], L) -> {nil, L};

```

```

%% Label : Json-Term --> [{<<Label>>, Term} | Rest]
parse_json_fields([?FIELD(Label, Code) | Rest], _) ->
    cons(tuple(str_to_bin(Label, L), parse_json(Code), L)
        , parse_json_fields(Rest, L)
        , L).

tuple(E1, E2, Line)    -> {tuple, Line, [E1, E2]}.
cons(Head, Tail, Line) -> {cons, Line, Head, Tail}.

str_to_bin(String, Line) ->
    {bin
    , Line
    , [{bin_element
        , Line
        , {string, Line, String}
        , default
        , [utf8]
        }
    ]
    }.

```

This deliberately limited transform scans every direct expression in a function body, preserving ordinary expressions and replacing JSON wrappers. String values and object labels become UTF-8 binaries: "λ" becomes the two bytes 206,187. The [utf8] segment type encodes each character in the string instead of treating its codepoint as one byte. It does not recursively search every Erlang expression for wrappers; nested objects inside a recognized JSON value are handled by `parse_json/1`.

And now we can compile `json_test` without errors:

```

1> c(json_parser).
{ok,json_parser}
2> c(json_test).
{ok,json_test}
3> json_test:test(42).
[{<<"name">>,<<"Jack (\\"Bee\\" ) Nimble">>},
{<<"format">>,
 [{<<"type">>,<<"rect">>},
  {<<"widths">>,[1920,1600]},
  {<<"height">>,-1080},
  {<<"interlace">>,false},
  {<<"frame rate">>,42}]]]

```

The AST generated by `parse_transform/2` must correspond to valid Erlang code unless you apply several parse transforms, which is possible. The validity of the code is checked by the following compiler pass.

2.2. Other Compiler Tools

There are a number of tools available to help you work with code generation and code manipulation. These tools are written in Erlang and not really part of the runtime system but they are very nice to know about if you are implementing another language on top of the BEAM.

In this section we will cover three of the most useful code tools: the lexer—Leex, the parser generator—Yecc, and a general set of functions to manipulate abstract forms—Syntax Tools.

2.2.1. Leex

Leex is the Erlang lexer generator. The lexer generator takes a description of a DFA from a definitions file `xrl` and produces an Erlang program that matches tokens described by the DFA.

The details of how to write a DFA definition for a tokenizer is beyond the scope of this book. For a thorough explanation I recommend the "Dragon book" ([\[DragonBook\]](#)). Other good resources are the man and info entry for `flex`, the lexer program that inspired leex, and the leex documentation itself. If you have info and flex installed you can read the full manual by typing:

```
> info flex
```

The online Erlang documentation also has the leex manual.[\[S2\]](#)

We can use the lexer generator to create an Erlang program which recognizes JSON tokens. By looking at the JSON definition[\[S3\]](#) we can see that there are only a handful of tokens that we need to handle. The example below handles those token forms, but makes two simplifications when turning them into Erlang values. Every number becomes a float, so large integers can lose precision and values outside the float range fail conversion. Each `\uXXXX` escape becomes one integer; surrogate pairs are not combined into a Unicode code point. This is a lexer-generator exercise, with those limits carried into the parser that follows.

Definitions.

```

Digit          = [0-9]
Digit1to9      = [1-9]
Integer        = (0|{Digit1to9}{Digit}*)
Exponent       = [eE][+-]?{Digit}+
HexDigit       = [0-9a-fA-F]
UnescapedChar  = [^\\\"\\\x00-\\x1f]
EscapedChar    = (\\\\\\|\\\\\\\"|\\\\\\b|\\\\\\f|\\\\\\n|\\\\\\r|\\\\\\t|\\\\\\/)
Unicode        = (\\\\\\u{HexDigit}{HexDigit}{HexDigit}{HexDigit})
Quote          = [\""]
Delim          = [\\[\\]:,{}]
Space          = [\\n\\s\\t\\r]

```

Rules.

```

{Quote}{Quote} : {token, {string, TokenLine, ""}}.
{Quote}({EscapedChar}|({UnescapedChar})|({Unicode}))+{Quote} :
  {token, {string, TokenLine, drop_quotes(TokenChars)}}.

```

```

null : {token, {null, TokenLine}}.
true  : {token, {true, TokenLine}}.
false : {token, {false, TokenLine}}.

```

```
{Delim} : {token, {list_to_atom(TokenChars), TokenLine}}.
```

```
{Space} : skip_token.
```

```

-?{Integer}(\\.{Digit}+)?({Exponent})? :
  {token, {number, TokenLine, number(TokenChars)}}.

```

Erlang code.

```
-export([t/0]).
```

```
% Erlang floats require a decimal point, including before an exponent.
```

```

number(Chars) ->
  {Mantissa, Exponent} =
    lists:splitwith(fun(C) -> C /= $e andalso C /= $E end, Chars),
    case lists:member(., Mantissa) of
      true -> list_to_float(Chars);
      false -> list_to_float(Mantissa ++ ".0" ++ Exponent)
    end.

```

```

drop_quotes([$" | QuotedString]) -> literal(lists:droplast(QuotedString)).
literal([$\\, $" | Rest]) ->
  [$" | literal(Rest)];
literal([$\\, $\\ | Rest]) ->
  [$\\ | literal(Rest)];
literal([$\\, $/ | Rest]) ->

```

```

[$/|literal(Rest)];
literal([$\\,$b | Rest]) ->
  [$\\literal(Rest)];
literal([$\\,$f | Rest]) ->
  [$\\literal(Rest)];
literal([$\\,$n | Rest]) ->
  [$\\literal(Rest)];
literal([$\\,$r | Rest]) ->
  [$\\literal(Rest)];
literal([$\\,$t | Rest]) ->
  [$\\literal(Rest)];
literal([$\\,$u,D0,D1,D2,D3|Rest]) ->
  Char = list_to_integer([D0,D1,D2,D3],16),
  [Char|literal(Rest)];
literal([C|Rest]) ->
  [C|literal(Rest)];
literal([]) ->[].

t() ->
  {ok,
   [{{'',1},
     {string,2,"no"},
     {':',2},
     {number,2,1.0},
     {'}',3}
   ],
   4}.

```

By using the Leex compiler we can compile this DFA to Erlang code, and by giving the option `dfa_graph` we also produce a dot-file which can be viewed with e.g. Graphviz.

```

1> leex:file(json_tokens, [dfa_graph]).
{ok, "./json_tokens.erl"}
2>

```

You can view the DFA graph (Figure 4) using for example `dotty`:

```

> dotty json_tokens.dot

```

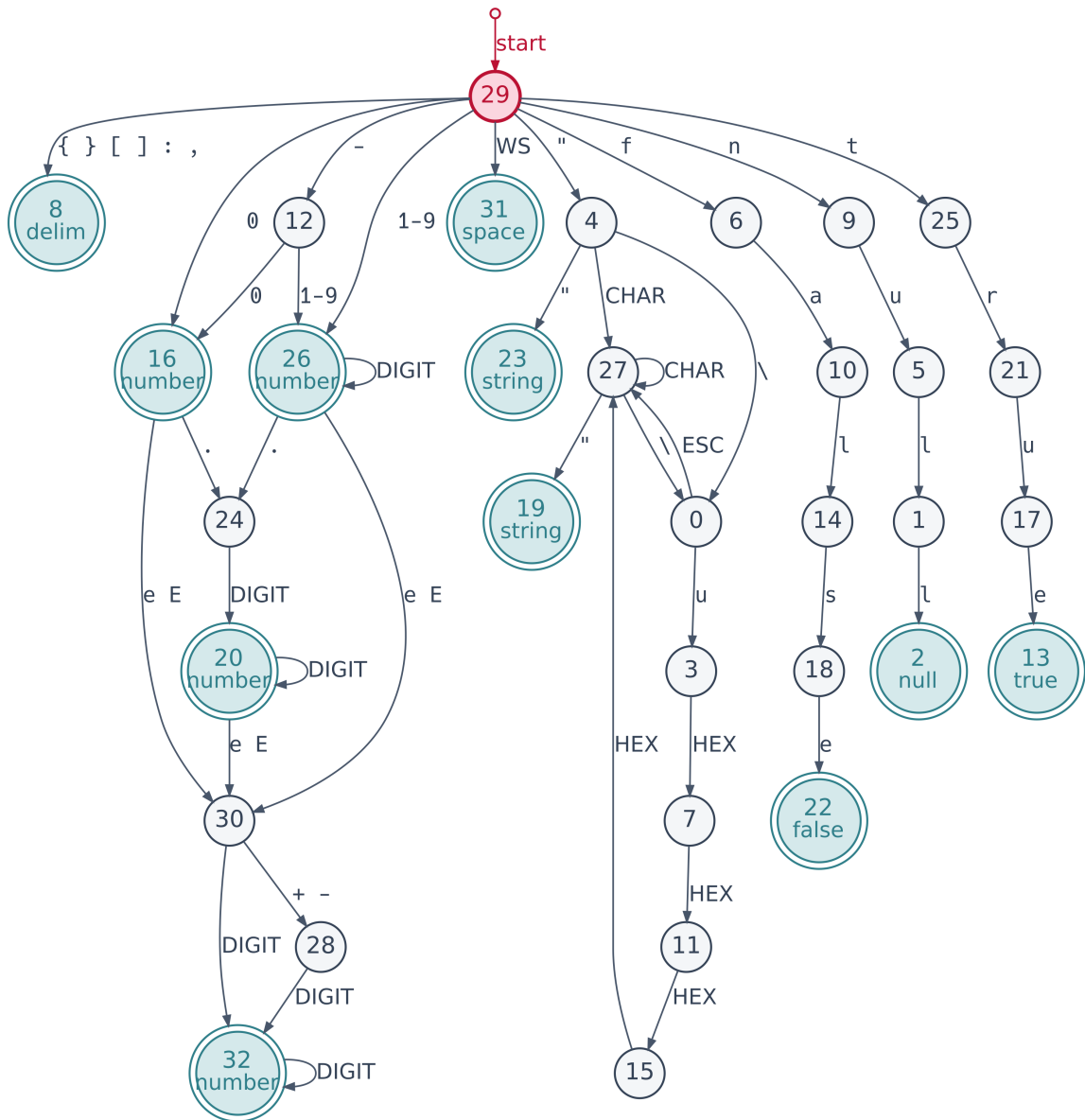


Figure 4. DFA graph for JSON tokens

Double circles mark accepting states. DIGIT means 0–9, HEX means a hexadecimal digit, and WS means space, tab, newline or carriage return. CHAR is an unescaped string character; ESC is one of `"`, `\`, `/`, `b`, `f`, `n`, `r` or `t` following a backslash. Follow `\` and then `u` to see the four HEX transitions required by a Unicode escape.

We can try our tokenizer on an example JSON file (`test.json`).

```
{
  "no" : 1,
  "name" : "Jack \"Bee\" Nimble",
  "escapes" : "\\b\\n\\r\\t\\f\\/\\/\\\"",
  "format": {
    "type"      : "rect",
    "widths"    : [1920,1600],
    "height"    : -1080,
    "interlace" : false,
    "unicode"   : "\\u002f",
    "frame rate": 4.5
  }
}
```

First we need to compile our tokenizer, then we read the file and decode its UTF-8 bytes to a character list. Finally we can use the `string/1` function that `leex` generates to tokenize the test file.

```
2> c(json_tokens).
{ok,json_tokens}
3> f(File), f(L), {ok, File} = file:read_file("test.json"), L = unicode
:characters_to_list(File), ok.
ok
4> f(Tokens), {ok, Tokens,_} = json_tokens:string(L), hd(Tokens).
{'{','1'}
5>
```

The shell function `f/1` tells the shell to forget a variable binding. This is useful if you want to try a command that binds a variable multiple times, for example as you are writing the lexer and want to try it out after each rewrite. We will look at the shell commands in detail in the later chapter.

Armed with a tokenizer for JSON we can now write a JSON parser using the parser generator `Yecc`.

2.2.2. Yecc

`Yecc` is a parser generator for Erlang. The name comes from `Yacc` (Yet Another Compiler Compiler) the canonical parser generator for C. The modern open source equivalent is `Bison`, which is often used with `Flex`.

Now that we have a lexer for JSON terms we can write a parser using yecc.

```

Nonterminals value values object array pair pairs.

Terminals number string true false null '[' ']' '{' '}' ',' ':' '.

Rootsymbol value.

value -> object  : '$1'.
value -> array   : '$1'.
value -> number  : get_val('$1').
value -> string  : get_val('$1').
value -> 'true'   : get_val('$1').
value -> 'null'   : get_val('$1').
value -> 'false'  : get_val('$1').

object -> '{' '}' : #{}.
object -> '{' pairs '}' : '$2'.

pairs -> pair : '$1'.
pairs -> pair ',' pairs : maps:merge('$1', '$3').

pair -> string ':' value : #{ get_val('$1') => '$3' }.

array -> '[' ']' : {}.
array -> '[' values ']' : list_to_tuple('$2').

values -> value : [ '$1' ].
values -> value ',' values : [ '$1' | '$3' ].

Erlang code.

get_val({_,_,Val}) -> Val;
get_val({Val, _}) -> Val.

```

Then we can use yecc to generate an Erlang program that implements the parser, and call the `parse/1` function provided with the tokens generated by the tokenizer as an argument.

```

5> yecc:file(yecc_json_parser), c(yecc_json_parser).
{ok,yecc_json_parser}
6> f(Json), {ok, Json} = yecc_json_parser:parse(Tokens).
{ok,#{"escapes" => "\b\n\r\t\f//\\",
      "format" => #{"frame rate" => 4.5,
                    "height" => -1080.0,
                    "interlace" => false,
                    "type" => "rect",
                    "unicode" => "/",
                    "widths" => {1920.0,1.6e3}},
      "name" => "Jack \"Bee\" Nimble",
      "no" => 1.0}}

```

The tools Leex and Yecc are nice when you want to compile your own complete language to the Erlang virtual machine. By combining them with Syntax tools and specifically Merl you can manipulate the Erlang Abstract Syntax tree, either to generate Erlang code or to change the behaviour of Erlang code.

2.3. Syntax Tools and Merl

Syntax Tools is a set of libraries for manipulating the internal representation of Erlang's Abstract Syntax Trees (ASTs).

The Syntax Tools application also includes the metaprogramming utility Merl. With Merl you can very easily manipulate the syntax tree and write parse transforms in Erlang code.

You can find the documentation for Syntax Tools on the Erlang.org site.[\[S4\]](#)

Sources and links

[\[S1\]](#) The translation here is done in accordance with EEP 18, Richard A. O'Keefe. EEP 18: JSON bifs. Erlang Enhancement Proposal 18, 2008. <http://www.erlang.org/eeps/eep-0018.html> (Erlang Enhancement Proposal 18: "JSON bifs")

[\[S2\]](#) Erlang/OTP Project. leex. Erlang/OTP 29 documentation. <https://www.erlang.org/docs/29/apps/parsetools/leex.html>

[\[S3\]](#) Ecma International. The JSON Data Interchange Syntax. ECMA-404, second edition, December 2017. <https://ecma-international.org/wp-content/uploads/ECMA-404.pdf>

[\[S4\]](#) Erlang/OTP Project. Erlang Syntax and Metaprogramming tools. Erlang/OTP 29 documentation. https://www.erlang.org/docs/29/apps/syntax_tools/chapter.html

Compiler Analysis and Encoding Details

These optional implementation details accompany the developer-facing chapters. The examples retain their original OTP version qualifications.

3.1. SSA pass ordering

Most of the back end lives here. The passes run in this order, each optionally followed by `beam_ssa_lint` (option `ssalint`): `beam_ssa_bool` for boolean expressions in guards, `beam_ssa_share` for sharing of identical code, `beam_ssa_recv` for receive optimizations, `beam_ssa_bsm` for binary matching, `beam_ssa_opt`, the main optimization pass and itself a sequence of about thirty sub-passes, `beam_ssa_throw` for exceptions, `beam_ssa_pre_codegen` for register allocation and stack frames, and `beam_ssa_codegen`, which produces the BEAM assembly code. `beam_validator` then checks the generated code for type safety, in its strong mode straight after code generation and in its weak mode after the assembly passes have rewritten it.

3.2. Type lattice and serialized type layout

The types are not the Erlang types of [The Erlang Type System and Tags](#). The compiler has a lattice of its own, defined in `beam_types.hrl`: any at the top; below it atoms, numbers (integers with optional bounds, and floats), lists (cons and nil), tuples with a known size and known elements, native records (the record data type new in OTP 29, see [The Erlang Type System and Tags](#)), and a group of other types for funs, maps, pids, ports, references and bitstrings; none at the bottom. A union type keeps the result of an expression that can return more than one of these. Two operations keep the lattice consistent: `meet` narrows a type when the code proves something new, such as after a type test, and `join` widens it where two paths merge. Each is only allowed to move in its own direction, which is what keeps the analysis sound.

A conservative subset of what the analysis knows at a value's use is passed on to the runtime. When `beam_ssa_codegen` emits an instruction whose operand has a useful type, it writes a typed register instead of a bare one, shown as `{tr,{x,0},Type}` in an assembler listing. The `beam_asm` pass gives each distinct type an index in a table and encodes the operand as the register followed by that index. The table becomes the `Type` chunk of the `.beam` file: a version number, a count, and one entry per type. An entry is a 16-bit word holding thirteen type bits, one each for `atom`, `bitstring`, `cons`, `float`, `fun`, `integer`, `map`, `nil`, `pid`, `port`, `reference`, `tuple` and `record`, and three flag bits that say whether a lower bound, an upper bound and a size unit follow, followed, when the bits say so, by a 64-bit lower bound, a 64-bit upper bound and an 8-bit size unit for bitstrings. A type is therefore a conservative approximation, never a full description: the JIT learns that a register holds an integer between 0 and 255, or that it holds a tuple. Exact tuple sizes known internally are discarded during type serialization. An instruction such as `is_tagged_tuple` can carry a size as a separate operand. The format is versioned, version 4 at OTP 29, and a module compiled with types cannot load on OTP 24 or older; the compiler makes sure of it by emitting an opcode those releases do not know. See [Modules and The BEAM File Format](#) for the chunk layout and [The JIT Compiler](#) for what `BeamAsm` does with the information.

3.3. Alias-analysis bookkeeping

Two pieces of bookkeeping make this tractable. The pass first computes, for every instruction, the set of variables that die there, by walking each basic block backwards from its terminator. And it can ignore back edges in the control flow graph: the only loops in BEAM code are receive loops, and no term is constructed inside one, so a back edge cannot change the alias status of anything. The results are attached to the SSA code as annotations for the next pass to read.

3.4. Record-update compiler and backend passes

The record case is worth following end to end, since five passes take part. A record update in the source, `R#rec{f = V}`, is expanded before Core Erlang into a chain of `setelement/3` calls, guarded by a check that the value is a tuple of the right size with the right tag. In `beam_ssa_opt` the `ssa_opt_update_tuple` sub-pass folds each chain into one `update_tuple` pseudo instruction carrying all the updated elements. The type analysis then converts `update_tuple` into `update_record` whenever it knows the source is a tuple of an exact size, below 512 elements, large enough for every updated index, and gives the instruction the hint

reuse: a request to the runtime to return the source unchanged if the update turns out to change nothing. Alias analysis and the destructive update pass may then replace the hint with `inplace`. An `update_tuple` whose source type stayed unknown is broken apart again by `ssa_opt_deoptimize_update_tuple`, and `beam_ssa_pre_codegen` turns those back into the `setelement/3` calls of the original program.

What reaches the runtime is `update_record Hint Size Src Dst Updates, opcode 181`, with the updates as a list of index and value pairs. The interpreter specializes it into `i_update_record_copy` or `i_update_record_in_place`. `BeamAsm` does the same in `emit_update_record` and `emit_update_record_in_place`. With the `reuse` hint and a single update it first compares the field with the new value, since setting a field to the value it already has is common and a comparison is far cheaper than a copy. Otherwise it copies the tuple word by word, stopping at each updated position to write the new value, and with `inplace` it writes the new values straight into the existing tuple.

3.5. Assembly pass catalogue

A few low level passes then work on this level: `beam_a` normalizes the code for the passes that follow, `beam_block` partitions it into basic blocks and optimizes within them, `beam_jump` shortens and shares jumps and removes unreachable code, `beam_clean` removes unused labels and functions, `beam_trim` trims stack frames where values are no longer needed, `beam_flatten` turns the blocks back into straight line code, and `beam_z` does the final rewrites the loader expects. `beam_asm` then encodes the instructions, builds the atom, literal, line and type tables, and writes the chunks of the `.beam` file.

Inspecting Process Structures and Heaps

These optional implementation details accompany the developer-facing chapters. The examples retain their original OTP version qualifications.

4.1. Optional: Inspecting a Heap with ETP (OTP 24)

The following session comes from an OTP 24 debug build. We will use ETP in GDB to find a tuple on a process heap. You can follow the addresses in the captured output without building that older runtime yourself. If you want to continue with the process memory model, go directly to [The PCB](#).

The startup output below is abbreviated. On that build I started the debugger and then ran the emulator:

```
$ERL_TOP/bin/cerl -debug -rgdb
...
OTP release: 24
ERTS version: 12.3.2.17
Arch: x86_64-pc-linux-gnu
Word size: 64-bit
Debug compiled: yes
...
(gdb) run
...
Eshell V12.3.2.17 (abort with ^G)
1>
```

Save this module as `mini_proc.erl`. It keeps a tuple on the heap while waiting for a message:

```
-module(mini_proc).
-export([wait/0]).

wait() ->
  X = {timestamp, erlang:timestamp()},
  receive _ -> X end,
  X.
```

Now we can compile and start that process in the shell within the debugger:

```
1> c(mini_proc).
{ok,mini_proc}
9> P3 = spawn(mini_proc, wait, []).
<0.101.0>
```

By pressing Ctrl+c we get back to the debugger:

```
Thread 1 "beam.debug.smp" received signal SIGINT, Interrupt.
[Switching to Thread 0x7ffff79b3c40 (LWP 2170)]
(gdb)
```

In this session I had trouble with the ETP commands until I switched to a scheduler thread. Thread 7 worked on this run; thread numbers will differ on another node. Here is the entry for our waiting process:

```
(gdb) thread 7
[Switching to thread 7 (Thread 0x7ffffb40b96c0 (LWP 2191))]
(gdb) etp-processes
...
---
Pix: 808
Pid: <0.101.0>
State: prq-prio-normal | usr-prio-normal | act-prio-normal

Flags:
Current function: unknown
I: #Cp<0x7ffffb53711dc>
Heap size: 233
Old-heap size: 0
Mbuf size: 0
Msgq len: 0 (inner=0, outer=0)
Msgq Flags: on-heap
Parent: <0.93.0>
Pointer: (Process*)0x7fff9ed6e030
...
(gdb)
```

We see the address of the '<0.101.0>' process in the Pointer field. We can use the `etp-stackdump` and `etp-heapdump` commands to inspect the stack and heap of the process. The `etp-stackdump` command takes a process pointer as an argument.

```
(gdb) set $p = ((Process *) 0x7fff9ed6e030)
(gdb) etp-stackdump $p
% Stacktrace (3)
I: #Cp<0x7fffb53711dc>.
0: {timestamp,{1747,147859,212377}}.
1: #Cp<0x7fffb4c14a78>.
2: #Cp<0x7fffb4c14a78>.
(gdb) etp-heapdump $p
% heapdump (7):
  0x7fff9eaf8278: | H:   3-tuple | I:       1747 | I:   147859 | I:   212377
| H:   2-tuple |   timestamp | B:0x9eaf827a
(gdb)
```

The stack holds a reference to the tuple {timestamp, {1747,147859,212377}}. In the heap dump we can see both the outer 2-tuple and its 3-tuple timestamp.

4.2. Reading the PCB dump

The dump below continues the optional OTP 24 session. You can read it as a specimen: find id, then heap, htop, stop and hend. We will use these fields to connect the PCB to our memory picture. If you skipped the debugger session, start with [the field explanations](#) after the dump.

```
print *((Process *) 0x7fff9ed6e030)
$2 = {common =
  {id = 3470333576787,
    refc = {atmc = {counter = 1}, sint = 1},
    tracer = 59, trace_flags = 167772160,
    timer = {counter = 0},
    u = {alive = {started_interval = 263, reg = 0x0, links = 0x0,
      lt_monitors = 0x0, monitors = 0x0},
      release = {later = 263, func = 0x0, data = 0x0, next =
0x0}}},
  htop = 0x7fff9eaf82b0,
  stop = 0x7fff9eaf89a8,
  fcalls = 3998,
  freason = 0,
  fvalue = 59,
  heap = 0x7fff9eaf8278,
  hend = 0x7fff9eaf89c0,
  abandoned_heap = 0x0,
  heap_sz = 233,
  min_heap_size = 233,
  min_vheap_size = 46422,
  max_heap_size = 3,
```

```

    arity = 0,
    arg_reg = 0x7fff9ed6e0f8,
    max_arg_reg = 6,
    def_arg_reg = {588235, 322955, 59, 14178673876263027908,
14178673876263027908, 4000},
    i = 0x7fffb53711dc <mini_proc:wait/0+140>,
    catches = 0, rcount = 0, schedule_count = 0,
    reds = 6, flags = 0, group_leader = 2267742733347, ftrace = 59, next =
0x555556142888,
    uniq = 0,
    sig_qs = {first = 0x0, last = 0x7fff9ed6e170, save = 0x7fff9ed6e170,
cont = 0x0,
        cont_last = 0x7fff9ed6e188,
        nmsigs = {next = 0x0, last = 0x0},
        recv_mrk_blk = 0x0, len = 0, flags = 2},
    bif_timers = 0x0, dictionary = 0x0,
    seq_trace_clock = 0, seq_trace_lastcnt = 0, seq_trace_token = 59,
    u = {real_proc = 0x8f9cb, terminate = 0x8f9cb,
        initial = {module = 588235, function = 322955, arity = 0}},
    current = 0x0, parent = 3195455669715, static_flags = 0,
    high_water = 0x7fff9eaf8278,
    old_hend = 0x0,
    old_htop = 0x0,
    old_heap = 0x0,
    gen_gcs = 0,
    max_gen_gcs = 65535,
    off_heap = {first = 0x0, overhead = 0},
    mbuf = 0x0, live_hf_end = 0xfffffffffffffffff8, msg_frag = 0x0, mbuf_sz =
0,
    psd = {counter = 0},
    bin_vheap_sz = 46422, bin_old_vheap_sz = 46422, bin_old_vheap = 0,
    sys_task_qs = 0x0, dirty_sys_tasks = 0x0,
    state = {counter = 42}, dirty_state = {counter = 0},
    sig_inq = {first = 0x0, last = 0x7fff9ed6e2a8, len = 0,
        nmsigs = {next = 0x0, last = 0x0}},
    trace_msg_q = 0x0, lock = {flags = {counter = 0},
        queue = {0x0, 0x0, 0x0, 0x0, 0x0},
        locked = {{counter = 0}, {counter = 0}},
{counter = 0},
        {counter = 0}, {counter = 0}}},
    scheduler_data = 0x0, run_queue = {counter = 140736249294720},
    last_htop = 0x7fff9eaf8298, last_mbuf = 0x0, heap_hfrag = 0x0,
    last_old_htop = 0x0,
    debug_rets_in = 4000
}

```

The OTP 29 struct keeps the same core fields as this OTP 24 dump, with a few additions and renames. The `len` field of `sig_qs` is now `mq_len`, `sig_inq` no longer carries a `len` field, and

`return_trace_frames`, `wrt_bins`, `xstate`, `sig_inq_contention_counter`, `sig_inq_buffers` and `paused_bif_timers` have been added. Compare with `erl_process.h` in the release you are running before reading a dump of your own.

Two fields in the PCB dump deserve special attention because they implement the signal-based message passing architecture introduced in OTP 21. The `sig_qs` struct holds the *private* signal queues: a middle queue (`cont/cont_last`) where incoming signals land after being moved from the outer queue, and the inner message queue (`first/last`) where decoded messages wait for a receive. The `save` pointer tracks selective receive position. The `sig_inq` struct is the *outer* signal queue, the only part that other processes (or schedulers) may write to under the process lock. Signals arrive in `sig_inq` and are later fetched into `sig_qs` by the receiving process. The `nmsigs` sub-fields in both structs track non-message signals (links, monitors, exits) that require special handling during the fetch. We will look at this three-queue architecture in detail in the next section.

If you want to dig even deeper than we will go in this chapter you can look at the C source code. The PCB is implemented as a C struct called `process` in the file `erl_process.h`[\[S1\]](#). The struct has grown over the years, so consult the version matching your OTP release.

[illegible]

```
4> show:tag_to_type(3470333576787).
pid
5>
```

4.3. GC fields in the PCB

The GC is controlled by these fields in the PCB:

Selected fields from OTP 29 `erl_process.h`; unrelated fields omitted.

```
Eterm *high_water;
Eterm *old_hend;      /* Heap pointers for generational GC. */
Eterm *old_hstop;
Eterm *old_heap;
Uint max_heap_size;   /* Maximum size of heap (in words). */
Uint16 gen_gcs;        /* Number of (minor) generational GCs. */
Uint16 max_gen_gcs;   /* Max minor gen GCs before fullsweep. */
```

4.4. Process dictionary buckets

The process dictionary is a very straightforward hash table implementation. In fact the PD memory area is just the plain bucket array, and each slot is either `NIL` (the empty list) if the bucket is empty, or otherwise it's a tagged pointer to data on the heap—a simple tuple `{Key, Value}` if there is only one entry in the bucket, or a `NIL`-terminated list `[{Key1, Value1}, ..., {KeyN, ValueN}]` if the bucket has `N` entries. You can find the implementation in the file `erl_process_dict.c`[\[S2\]](#).

This way, there is no additional complexity for managing dynamically growing and shrinking bucket lists, and no separate memory management for these data structures apart from the bucket array—it is all handled by the normal creation and freeing of tuples and lists on the heap (which is already as efficient as you'd like). It also means that the garbage collector can easily traverse the hash table much like any Erlang data. (In [Garbage Collection](#) we will see the details of how garbage collection works.)

4.5. Dictionary insertion and update mechanics

Note that, as with any hash table, storing an element in the PD is not completely free—it will result in an extra tuple for the key/value entry and an extra cons cell if the bucket was not empty, which might also cause garbage collection to be triggered. It could also require resizing of the bucket array. Updating an existing key with an immediate value writes the new value into the existing `{Key, Value}` tuple in place. Updating it with a boxed value unlinks the old tuple from its bucket list and prepends a new tuple, so the rest of the list stays where it is.

4.6. Complete process backtrace

The main chapter shows an excerpt of this captured shell backtrace. Addresses and shell frames vary with the runtime and session.

```
1> erlang:process_display(self(), backtrace).
Program counter: 0x00007fdf9baa2e78 (erlang:self/0 + 32)

0x00007fddedf247480 Return addr 0x00007fdf9baa2118 (erlang:process_display/2 +
128)
y(0)      []
y(1)      backtrace
y(2)      <0.89.0>

0x00007fddedf2474a0 Return addr 0x00007fdf9bbb7644 (erl_eval:do_apply/7 + 308)
y(0)      none
y(1)      []

0x00007fddedf2474b8 Return addr 0x00007fdf9bf0e430 (shell:exprs/7 + 536)
y(0)      []
y(1)      []
y(2)      []
y(3)      []
y(4)      cmd
y(5)      []
y(6)      {value,#Fun<shell.5.46532814>}
y(7)      {eval,#Fun<shell.23.46532814>}
y(8)      #Ref<0.908573933.3456499713.143140>
y(9)      []

0x00007fddedf247510 Return addr 0x00007fdf9bf0dc60 (shell:eval_exprs/7 + 152)
y(0)      []
y(1)      []
y(2)      []
y(3)      []
y(4)      <0.88.0>
y(5)      Catch 0x00007fdf9bf0dd0e (shell:eval_exprs/7 + 326)

0x00007fddedf247548 Return addr 0x00007fdf9bf0d87c (shell:eval_loop/4 + 564)
y(0)      #Ref<0.908573933.3456499713.143140>
y(1)      #Ref<0.908573933.3456499713.143160>
y(2)      <0.88.0>

0x00007fddedf247568 Return addr 0x00007fdf9ba2d1f8 (<terminate process
normally>)
true
```

Sources and links

[S1] Erlang/OTP Project. Erlang/OTP source code: `erl_process.h`. OTP-29.0; `erts/emulator/beam/erl_process.h`.
<https://github.com/erlang/otp>

[S2] Erlang/OTP Project. Erlang/OTP source code: `erl_process_dict.c`. OTP-29.0;
`erts/emulator/beam/erl_process_dict.c`. <https://github.com/erlang/otp>

Term Tags and Heap Object Encodings

These optional implementation details accompany the developer-facing chapters. The examples retain their original OTP version qualifications.

5.1. The compiler type lattice and serialized entries

The compiler's type analysis (see [The Compiler](#)) works in a lattice of its own, defined in `beam_types.hrl`: any at the top, none at the bottom, and in between atoms, numbers split into floats and integers with optional bounds, lists split into cons and nil, tuples with a known size and known elements, native records, and a group of other types for funs, maps, pids, ports, references and bitstrings with a known size unit. It is deliberately coarser than what Dialyzer works with, because every type in it has to be cheap to test and cheap to encode.

The analysis is conservative. What it records for a register is a type the value is guaranteed to have, never a guess, and when it cannot prove anything the register is left untyped. The compiler writes the result into the `.beam` file as a table of types. Each entry starts with a 16-bit word: thirteen bits identify possible base types, and three indicate whether an integer lower bound, upper bound or bitstring size unit follows. An operand in the instruction stream then refers to a register together with an index into this table. That encoding carries the type table's information, a bitset and a few bounds per typed operand. Instructions also carry their own operands, so "the JIT knows the type" always means "the JIT knows the value is one of these few kinds, within these limits".

5.2. Immediate tag layout

The Immediate tag is further divided by the next two bits, as [Figure 5](#) shows:

<i>process identifier</i>	0	0	1	1	Pid
<i>port identifier</i>	0	1	1	1	Port
<i>see the next stage</i>	1	0	1	1	Immediate 2
<i>integer value, 28 or 60 bits</i>	1	1	1	1	Small integer

Figure 5. The immediate tags: two more bits above the primary tag 11

5.3. PID bit fields

A local PID immediate uses four tag bits and the remaining bits for its identifier: 28 bits on a 32-bit build, or 60 bits on a 64-bit build since OTP 26. OTP 29 divides that identifier into a 28-bit process number and any remaining serial bits. The process-table slot is derived from the identifier and then checked against the stored process identity; the process number is not itself a table offset.

5.4. Small integers and secondary immediate tags

By having all four tag bits as ones for small integers the emulator can make an efficient test when doing integer arithmetic to see if both arguments are immediates. (`is_both_small(x,y)` is defined as `(x & y & 1111) == 1111`).

The Immediate 2 tag is further divided by two more bits, as [Figure 6](#) shows:

<i>index into the atom table</i>	0	0	1	0	1	1	Atom
<i>catch label</i>	0	1	1	0	1	1	Catch
	1	0	1	0	1	1	<i>unused</i>
<i>all zero</i>	1	1	1	0	1	1	Nil, the empty list

Figure 6. The immediate 2 tags: two more bits above 10 11

5.5. Atom table structure

In the atom table atoms are stored as C structs like this (from `erts/emulator/beam/atom.h` at OTP-29.0):

```
typedef struct atom {
    IndexSlot slot;      /* MUST BE LOCATED AT TOP OF STRUCT!!! */
    Sint16 len;          /* length of atom name (UTF-8 encoded) */
    Sint16 latin1_chars; /* 0-255 if atom can be encoded in latin1;
                        otherwise, -1 */
    int ord0;            /* ordinal value of first 3 bytes + 7 bits */
    union{
        byte* name;      /* name of atom, used by templates */
        Eterm bin;       /* name of atom, used when atom is in table*/
    } u;
} Atom;
```

The `len` field is the length of the name in bytes, UTF-8 encoded, and `latin1_chars` is the number of characters if the name can be written in Latin-1, or -1 if it can not. Thanks to the `len` and the `ord0` fields the order of two atoms can be compared efficiently as long as they differ within roughly the first four characters (the 31 bits that `ord0` holds).

The union at the end holds the name. While an atom is being looked up or inserted, the runtime works with a template whose `name` points at a plain byte array. Once the atom is in the table its name is an Erlang binary, referenced by `bin`. The binary is allocated as a global literal, the same kind of constant that module literals are kept in (see [The Memory Subsystem: Stacks, Heaps and Allocators](#)), so it is never garbage collected and never copied.

5.6. Atom comparison implementation note

Atom names and comparison cost

If you for some reason generate atoms with a pattern like name followed by a number and then store them in an ordered list or ordered tree the atom comparison will be more expensive if they all have the same first letters (e.g. `foo_1`, `foo_2`, etc.).

Not that you should ever generate atom names, since the atom table is limited. I'm just saying, there is an evil micro optimization to be found here.

You would of course never do this, but if you find code that generates atom with a number followed by a postfix name, now you know what the author of that code might have been thinking.

5.7. Header tags and their history

All other terms on the heap start with a *header word*. The header word uses a four bit subtag and the primary tag 00. For tuples and bignums, the remaining bits encode the *arity*, the number of additional words in the object. Other headers, including maps, pack more fields above the subtag; their layouts are described below. [Figure 7](#) lists the subtags.

arity	0	0	0	0	0	0	0	ARITYVAL, a tuple	
	0	0	0	1	0	0	0	unused	
arity, s is the sign	0	0	1	s	0	0	0	BIGNUM	
...	0	1	0	0	0	0	0	REF	
...	0	1	0	1	0	0	0	FUN	
...	0	1	1	0	0	0	0	FLONUM, a float	
...	0	1	1	1	0	0	0	RECORD, a native record	
...	1	0	0	0	0	0	0	HEAP_BITS	BITSTRINGS
...	1	0	0	1	0	0	0	SUB_BITS	
...	1	0	1	0	0	0	0	BIN_REF	
...	1	0	1	1	0	0	0	MAP	
...	1	1	0	0	0	0	0	EXTERNAL_PID	EXTERNAL THINGS
...	1	1	0	1	0	0	0	EXTERNAL_PORT	
...	1	1	1	0	0	0	0	EXTERNAL_REF	
...	1	1	1	1	0	0	0	unused	

Figure 7. Header words: a four-bit subtag above the primary tag 00



Version Info

This is the table at Erlang/OTP 29. Erlang/OTP 27.0 replaced the three binary subtags of earlier releases (REFC_BINARY, HEAP_BINARY, SUB_BINARY) and the aggregate object used during matching with the three bitstring objects listed here, and Erlang/OTP 29.0 added RECORD for native records in a slot that had become free.

How the tag scheme has changed, and why

The staged scheme has held its shape since R7A in 2000, when Pettersson laid out the reasoning: alignment leaves the low bits of a pointer free, so they carry the primary tag, and lists earn a primary tag of their own because a separate cons tag lets a cons cell live on the heap with no header word [Pettersson2000]. Two spare bits are not enough for every type, so a term that needs more room spends a second stage of bits, and a boxed term spends a header word. The four bits of header subtag are the scarce resource.

Two immediates have changed. A local pid or port carried a 28-bit identifier for two decades, small enough that a busy node reused identifiers within its lifetime; OTP 26 widened it to 60 bits on a 64-bit runtime, which in practice ends reuse for the life of the node. The empty list moved the other way. Through OTP 23 NIL set every bit above its tag to one, so on a 64-bit build it was 16#FFFFFFFFFFFFFFFB. OTP 24, the release that added the JIT, redefined it as the bare tag value 16#3B with zeros above. A small constant loads and compares in a single instruction, which is what lets the generated code test for the end of a list with one byte comparison (The JIT Compiler).

The boxed side is where subtags are added and retired. Before OTP 27 three of them named three binary representations, a heap binary, a reference counted binary and a sub binary, plus a fourth object used only while matching. OTP 27 replaced all four with three bitstring objects, HEAP_BITS, SUB_BITS and BIN_REF, folding binaries and bitstrings into one representation and freeing subtag slots. OTP 29 spent one of those freed slots on RECORD, the header of a native record (EEP 79), a term that reads as its own type rather than as a tagged tuple.

5.8. Bitstring header fields

Table 1. Bitstring header and payload

Object	Header	Following words
ErlHeapBits	Arity and HEAP_BITS tag	Bit size, then payload bytes padded to whole words
BinRef	Arity and BIN_REF tag	Pointer to the reference-counted Binary, then off-heap list link
ErlSubBits	Arity and SUB_BITS tag	Data pointer with flags, start and end bit positions, originating term

All three headers use the ordinary six-bit header tag: four subtag bits followed by primary tag 00. The remaining bits hold the header arity; lengths and offsets are stored in the object's following words. There is no extra padding field inside the header. The structures and header macros are defined in `erts/emulator/beam/erl_bits.h`.

Following the header:

- For **heap bitstrings**, the size in bits and then the payload bytes.
- For **binary references**, the pointer to the off-heap Binary and the links that let the garbage collector find it.

Sub bitstrings (subtag 1001) are views into one of the other two. They store the start and end bit offsets and a pointer to the object they refer to. This avoids copying payload data when slicing or matching parts of a binary.

5.9. Bignum encoding and arithmetic source guide

The header is `(2 bsl 6) bor 16#08`, or `16#88`: two limbs, the positive bignum subtag 0010, and primary tag 00. Negating `N` keeps the magnitude limbs and changes the sign bit in the header to give `16#8C`. It does not complement the limb bits.[\[S1\]](#)

A bignum holds any integer outside the small-immediate range. Its limbs are stored least significant first, each with type `ErlsDigit` and radix 2^{D_EXP} . `D\_EXP` is 64 on a 64-bit build and 32 on a 32-bit build, so our example needs three limbs on a 32-bit build. `BIG_V(xp)` points to the first limb, and `BIG_DIGIT(xp, i)` selects limb `i`, counting from zero.[\[S2\]](#)

Optional source reading: arithmetic on limbs

If you want to follow the arithmetic in `big.c`, start with `I_add` and `I_sub`: they propagate carry or borrow from one limb to the next. Multiplication uses the schoolbook routine `I_mul` or Karatsuba multiplication, `I_mul_karatsuba`, depending on operand size. Division and remainder use `I_div` and `I_rem`, based on Knuth's Algorithm D. `big_norm` trims the result, rebuilds its header, and returns a small immediate when the value fits.[\[S3\]](#)

5.10. Reference encoding

A reference is a unique term commonly used to identify a request and match its reply. OTP 29 constructs ordinary local references from a 64-bit counter value and a scheduler or thread identifier, packed into three 32-bit data words. The first word uses 18 bits; the next

word combines counter bits with the thread identifier and reserved markers. The final word holds the upper counter bits.

This representation is not one globally increasing 82-bit counter. The initial counter value is derived from time, and references from different schedulers need not follow a single numeric sequence. Treat a reference as an opaque identity, rather than deriving ordering or a wrap interval from its printed form. External references also carry node identity and creation, distinguishing node incarnations.

The heap object (`Erts0RefThing` in `erl_term.h`) holds the header, a 32-bit marker word that lets the runtime tell an ordinary reference from a magic reference, and the three 32-bit data words. On a 64-bit system the marker shares a word with one of the data words, so a local ref takes up three 64-bit words on the heap. On a 32-bit system each field is a word of its own, so a local ref takes up five 32-bit words.

+0	00000000 00000000 00000001 00 010000	header: arity 4, REF subtag
+4	mmmmmmmm mmmmmmmmm mmmmmmmmm mmmmmmmmm	marker: ordinary or magic reference
+8	00000000 0000000r rrrrrrrrrr rrrrrrrrrr	data[0], 18 bits used
+12	rrrrrrrrrr rrrrrrrrrrr rrrrrrrrrrr rrrrrrrrrrr	data[1], counter bits, thread id, markers
+16	rrrrrrrrrr rrrrrrrrrrr rrrrrrrrrrr rrrrrrrrrrr	data[2], upper counter bits

tag bits in **bold red**, field bits in blue; offsets in bytes from the start of the object

Figure 8. A local ref on a 32-bit system, first field at the top

Figure 8 shows the data-word capacity. Decoding the counter requires masking the thread and marker fields in `Data1`, as implemented by `erts_get_ref_numbers_value` in `erl_bif_unique.h`.

5.11. Native record instance and definition layout

On the heap a native record is a boxed term with the `RECORD` header subtag, `0111`. The header's arity counts the words that follow: one word holding a boxed pointer to the record's definition, then one word per field value.

```

/* A native-record value (instance). */
typedef struct {
    Eterm thing_word;          /* Header tag RECORD */

    /* Boxed-tagged ErtsRecordDefinition */
    Eterm record_definition;

    Eterm values[];
} ErtsRecordInstance;

```

The definition is a separate heap object shared by every instance. It is tagged as a tuple, so the garbage collector can treat it as one, and holds the module, the name, whether the record is exported, a tuple mapping the fields' declared order to their sorted order, the field names in atom index order, and a hash that becomes part of the phash2 value of every instance. Each loaded code generation of a module has its own canonical definition, kept in a per record entry together with the default values, so a record built by old code and one built by new code can be told apart. See `erl_record.h` [\[S4\]](#).

Sources and links

[\[S1\]](#) OTP 29.0 bignum header definitions in `erl_term.h`, Erlang/OTP Project. Erlang/OTP source code: `erl_term.h`. OTP-29.0; `erts/emulator/beam/erl_term.h`. <https://github.com/erlang/otp>.

[\[S2\]](#) OTP 29.0 limb definitions, Erlang/OTP Project. Erlang/OTP source code: `big.h`. OTP-29.0; `erts/emulator/beam/big.h`. <https://github.com/erlang/otp>.

[\[S3\]](#) OTP 29.0 bignum arithmetic, Erlang/OTP Project. Erlang/OTP source code: `big.c`. OTP-29.0; `erts/emulator/beam/big.c`. <https://github.com/erlang/otp>.

[\[S4\]](#) Erlang/OTP Project. Erlang/OTP source code: `erl_record.h`. OTP-29.0; `erts/emulator/beam/erl_record.h`. <https://github.com/erlang/otp>

Building a Bytecode Interpreter and Threaded Dispatcher

These optional implementation details accompany the developer-facing chapters. The examples retain their original OTP version qualifications.

6.1. Bytecode emulation

If we take a look at our [naive stack machine](#) for arithmetic expressions we see that we use Erlang atoms and pattern matching to select which instruction to execute. This is a very heavy machinery to just decode machine instructions. In a real machine implementation we would code each instruction as a "machine word" integer or even a single byte.

We can rewrite our stack machine to be a *byte code* machine implemented in C. First we rewrite our [simple compiler](#) so that it produces byte codes. This is straightforward, just replacing each instruction encoded as an atom with a byte representing the instruction (see below). To be able to handle integers larger than 255 we encode integers with a size byte followed by the integer encoded in bytes.

A simple bytecode compiler

```
compile(Expression, FileName) ->
    [ParseTree] = element(2,
        erl_parse:parse_exprs(
            element(2,
                erl_scan:string(Expression))),
        file:write_file(FileName, generate_code(ParseTree) ++ [stop()])).

generate_code({op, _Line, '+', Arg1, Arg2}) ->
    generate_code(Arg1) ++ generate_code(Arg2) ++ [add()];
generate_code({op, _Line, '*', Arg1, Arg2}) ->
    generate_code(Arg1) ++ generate_code(Arg2) ++ [multiply()];
generate_code({integer, _Line, I}) -> [push(), integer(I)].

stop() -> 0.
```

```

add()      -> 1.
multiply() -> 2.
push()     -> 3.
integer(I) ->
    L = binary_to_list(binary:encode_unsigned(I)),
    [length(L) | L].

```

This educational machine accepts nonnegative integer literals, addition, and multiplication, with at most 1,000 simultaneous stack entries. All literals and intermediate results must fit in a C int; it assumes trusted bytecode and does not implement Erlang’s arbitrary-sized integers. Unsigned byte reads preserve values from 128 through 255 in the encoding.

Now let us write a simple virtual machine and bytecode interpreter in C. (The full code can be found in the online appendix.)

```

#define STOP 0
#define ADD 1
#define MUL 2
#define PUSH 3

#define pop() (stack[--sp])
#define push(X) (stack[sp++] = X)

int run(const unsigned char *code) {
    int stack[1000];
    int sp = 0, size = 0, val = 0;
    const unsigned char *ip = code;

    while (*ip != STOP) {
        switch (*ip++) {
            case ADD: { int y = pop(); int x = pop(); push(x + y); break; }
            case MUL: { int y = pop(); int x = pop(); push(x * y); break; }
            case PUSH:
                size = *ip++;
                val = 0;
                while (size--) { val = val * 256 + *ip++; }
                push(val);
                break;
        }
    }
    return pop();
}

```

As you see, a virtual machine written in C does not need to be very complicated. This machine is just a loop, checking the byte code at each instruction by looking at the value pointed to by the *instruction pointer* (ip).

For each byte code instruction it will switch on the instruction byte code and jump to the case which executes the instruction. This requires a decoding of the instruction and then a jump to the correct code. If we compile `vsm.c` for 32-bit x86 with `gcc -S` (the listing below is from an older GCC; a 64-bit build uses `%rbp` and 64-bit moves but has the same shape) we see the inner loop of the decoder:

```
L11:
    movl    -16(%ebp), %eax
    movzbl  (%eax), %eax
    movsbl  %al, %eax
    addl    $1, -16(%ebp)
    cmpl    $2, %eax
    je      L7
    cmpl    $3, %eax
    je      L8
    cmpl    $1, %eax
    jne     L5
```

It has to compare the byte code with each instruction code and then do a conditional jump. In a real machine with many instructions this can become quite expensive.

6.2. Threaded Code

A better solution would be to have a table with the address of the code—then we could just use an index into the table to load the address and jump without the need to do any comparisons. This technique is sometimes called *token threaded code*. Taking this a step further we can actually store the address of the code snippet implementing the instruction as the “instruction” itself in the code memory. We could then let the interpreter loop read the next address instead of a single byte, call the routine at that address, and when it returns just advance to the next address. This is called *subroutine threaded code*. Note however that the code takes up much more space—one whole word per instruction instead of a single byte—but the speed advantage usually makes it worth it on hardware with plenty of memory. (Token threaded code is often used where memory is tight and speed might be less important.)

This approach will make the execution simpler at runtime, but it makes the VM more complicated by requiring a loader. The loader replaces the byte code instructions with addresses to the functions implementing the instructions.

A loader might look like this:

```

typedef void (*instructionp_t)(void);

void add() { int x,y; x = pop(); y = pop(); push(x + y); }
void mul() { int x,y; x = pop(); y = pop(); push(x * y); }
void pushi(){ int x; x = (int)(intptr_t)*ip++; push(x); }
void stop() { running = 0; }

instructionp_t *read_file(char *name) {
    FILE *file;
    instructionp_t *code;
    instructionp_t *cp;
    long size;
    int ch;
    unsigned int val;

    file = fopen(name, "r");

    if(file == NULL) exit(1);

    fseek(file, 0L, SEEK_END);
    size = ftell(file);
    code = calloc(size, sizeof(instructionp_t));
    if(code == NULL) exit(1);
    cp = code;

    fseek(file, 0L, SEEK_SET);
    while ( ( ch = fgetc(file) ) != EOF )
    {
        switch (ch) {
            case ADD: *cp++ = &add; break;
            case MUL: *cp++ = &mul; break;
            case PUSH:
                *cp++ = &pushi;
                ch = fgetc(file);
                val = 0;
                while (ch--) { val = val * 256 + fgetc(file); }
                *cp++ = (instructionp_t)(intptr_t) val;
                break;
        }
    }
    *cp = &stop;

    fclose(file);
    return code;
}

```

As we can see, we do more work at load time here, including the decoding of integers larger than 255. (Yes, I know, the code is not safe for very large integers.)

The decode and dispatch loop of the VM becomes quite simple though:

```
int run() {
    sp = 0;
    running = 1;

    while (running) (*ip++)();

    return pop();
}
```

Finally, instead of letting each instruction snippet be a function that returns, we can write the code so that each snippet ends with the same small piece of code that the main loop would use to read the next instruction and jump to it. This way, there is no bouncing back and forth between the main loop and the snippets—in fact, the main loop disappears, and each instruction snippet becomes individually responsible for ensuring that it continues to the next instruction. This technique is called *direct threaded code*. In order to implement this in C, BEAM uses the GCC extension “labels as values”.

6.3. The Real BEAM

We will look closer at the BEAM emulator later but for now we will take a quick look at how the actual BEAM add instruction is implemented. The interpreter’s instructions are no longer written straight into `beam_emu.c`. Since OTP 21 they live in `.tab` files under `erts/emulator/beam/emu/`, which the `beam_makeops` script expands into C with the label and dispatch macros below. The `Goto` macro is the computed goto of direct threaded code.

```
#define OpCase(OpCode)    lb_##OpCode
#define Goto(Re1)         goto *((void *)BeamCodeAddr(Re1))
```

Addition is the `plus` instruction in `arith_instrs.tab`, composed from a `fetch` part that loads the operands and an `execute` part that does the work. The dispatch to the next instruction hides in the `$NEXT0()` macro. This is the OTP 29 fast path, with the comments and the fallback for compilers without overflow builtins left out:

```
i_plus := plus.fetch.execute;

plus.head() {
    Eterm PlusOp1, PlusOp2;
```

```

}

plus.fetch(Op1, Op2) {
    PlusOp1 = $Op1;
    PlusOp2 = $Op2;
}

plus.execute(Fail, Dst) {
    if (ERTS_LIKELY(is_both_small(PlusOp1, PlusOp2))) {
        Sint lhs_tagged, rhs_untagged, res;

        lhs_tagged = PlusOp1;
        rhs_untagged = PlusOp2 & ~_TAG_IMMED1_MASK;

        if (ERTS_LIKELY(!__builtin_add_overflow(lhs_tagged, rhs_untagged,
&res))) {
            ASSERT(is_small(res));
            $Dst = res;
            $NEXT0();
        }
    }
    $OUTLINED_ARITH_2($Fail, mixed_plus, BIF_splus_2, PlusOp1, PlusOp2, $
Dst);
}

```

Only one operand is untagged before the addition, so the tag bits of the other operand pass through unchanged and the result needs no tagging of its own. The compiler's overflow builtin replaces the explicit range check that older releases did with `IS_SSMALL`. If the operands are not both small integers, or the sum overflows, the `OUTLINED_ARITH_2` macro calls `erts_mixed_plus` to handle floats and bignums.

To make it a little easier to understand how the BEAM dispatcher is implemented, let us take a somewhat imaginary example. We will start with some real external BEAM code, but then I will invent some internal BEAM instructions and implement them in C.

If we start with a simple add function in Erlang:

```
add(A,B) -> id(A) + id(B).
```

Compiled to BEAM code this will look as follows (or would, before the `swap` instruction was added in OTP 23 — see [Stack layout example](#)):

```

{function, add, 2, 2}.
{label, 1}.
  {func_info, {atom, add}, {atom, add}, 2}.
{label, 2}.
  {allocate, 1, 2}.
  {move, {x, 1}, {y, 0}}.
  {call, 1, {f, 4}}.
  {move, {x, 0}, {x, 1}}.
  {move, {y, 0}, {x, 0}}.
  {move, {x, 1}, {y, 0}}.
  {call, 1, {f, 4}}.
  {gc_bif, '+', {f, 0}, 1, [{y, 0}, {x, 0}], {x, 0}}.
  {deallocate, 1}.
return.

```

(See `add.erl` and `add.S` under `code/beam_chapter/src` for the full code; that `add.S` was produced by OTP 29 and therefore uses `swap`.)

Now if we zoom in on the three instructions between the function calls in this code:

```

{move, {x, 0}, {x, 1}}.
{move, {y, 0}, {x, 0}}.
{move, {x, 1}, {y, 0}}.

```

This code first moves the return value of the function call (`x0`) to register `x1`. Then it moves the previously saved `B` from the stack slot `y0` into the first argument register (`x0`). Finally it moves the value in `x1` to the stack slot (`y0`) so that it will survive the next function call.

Imagine that we would implement three instructions in BEAM called `move_xx`, `move_yx`, and `move_xy`. The real BEAM has its own operand-type specialisations of `move` (`ops.tab` declares `move xy xy`, `move x r` and so on), so the idea is real, but these three names and their bodies are invented for this example:

```

#define OpCase(OpCode)    lb_##OpCode
#define Goto(Rel) goto *((void *)Rel)
#define Arg(N) (Eterm *) I[(N)+1]

OpCase(move_xx):
{
  x(Arg(1)) = x(Arg(0));
  I += 3;
  Goto(*I);
}

OpCase(move_yx): {
  x(Arg(1)) = y(Arg(0));
  I += 3;
}

```

```

    Goto(*I);
}

OpCase(move_xy): {
    y(Arg(1)) = x(Arg(0));
    I += 3;
    Goto(*I);
}

```

Note that the star in `goto *` does not mean dereference, the expression means jump to an address pointer, we should really write it as `goto*`.

Now imagine that the compiled C code for these instructions ends up at memory addresses 0x3000, 0x3100, and 0x3200. When the BEAM code is loaded, the three move instructions in the code will be replaced by the memory addresses of the implementation of the instructions. Imagine that the code (`{move,{x,0},{x,1}}`, `{move,{y,0},{x,0}}`, `{move,{x,1},{y,0}}`) is loaded at address 0x1000:

Instruction	Address	Word
move X0 X1	0x1000	0x3000 (move_xx)
	0x1004	0 (source X register)
	0x1008	1 (destination X register)
move Y0 X0	0x100c	0x3100 (move_yx)
	0x1010	0 (source Y register)
	0x1014	0 (destination X register)
move X1 Y0	0x1018	0x3200 (move_xy)
	0x101c	1 (source X register)
	0x1020	0 (destination Y register)

This illustrative layout uses four-byte words. The instruction bodies are shown separately above, so the address table preserves its columns at the book's printed page width.

The word at address 0x1000 points to the implementation of the `move_xx` instruction. If the register `I` contains the instruction pointer, pointing to 0x1000, then the dispatch will be to fetch `*I` (i.e. 0x3000) and jump to that address (`goto* *I`).

In [Generic BEAM Instructions](#) we will look more closely at some real BEAM instructions and how they are implemented.

AtomVM Execution and Compatibility

These optional implementation details accompany the developer-facing chapters. The examples retain their original OTP version qualifications.

AtomVM: Reimplementing the BEAM for Small Systems

Let us return to the sensor application in [Section 1.1](#). Its two processes exchange measurements, but the code that schedules those processes and collects their heaps belongs to AtomVM. The core runtime is written in C and comes with a selected library set. You can see where it replaces ERTS in [Figure 1](#).

You still compile the sensor code with a supported Erlang or Elixir compiler. For bytecode execution, AtomVM's tools combine the `.beam` modules and their dependencies into `.avm` archives. The Packbeam format[\[S1\]](#) can hold several modules and other application files. Follow a function call from your module into a library or BIF, and you reach the part that also needs checking: AtomVM must provide the instructions and behavior that call relies on.

AtomVM's portable interpreter remains an execution option, but the 0.7 alpha releases also introduce native execution. Their JIT documentation[\[S2\]](#) describes compilation at module load time and ahead-of-time compilation that embeds native code in an `avmN` chunk. The compiler is written in Erlang, and supported targets include several ARM and RISC-V variants alongside x86-64 and AArch64. This is AtomVM's implementation; the ERTS register mappings and emitters in [The JIT Compiler](#) describe a separate runtime. Compiler requirements and available modes depend on the release and target build.

AtomVM tests its implementations against Erlang/OTP, but it also makes choices that your code can observe. For example, 0.7.0-alpha.1 supports integers with a magnitude of up to 256 bits. If your program builds larger integers, you have found a difference you must handle. The compatibility guide[\[S3\]](#) separates such VM limits from missing library functions. Use it when checking the operations in your application, including peripheral calls.

What matters to our sensor processes is that sending and receiving measurements behaves as the supported language operations specify. AtomVM can arrange its scheduler and garbage collector differently from ERTS while keeping that behavior. This is why the ERTS internals in this book can help you understand the problem without prescribing AtomVM's implementation.

If you need ERTS itself, Nerves, GRiSP, and Tyn take the other approach shown in [Section 1.4](#). Look at the memory available on your target, the peripherals you need, and the OTP functions your application uses. A memory figure alone cannot tell you which runtime will fit the job.

Sources and links

[S1] AtomVM Project. Packbeam Format. AtomVM 0.7.0-alpha.1. <https://doc.atomvm.org/v0.7.0-alpha.1/packbeam-format.html>

[S2] AtomVM Project. JIT Compilation. AtomVM 0.7.0-alpha.1. <https://doc.atomvm.org/v0.7.0-alpha.1/jit.html>

[S3] AtomVM Project. Differences between AtomVM and BEAM. AtomVM 0.7.0-alpha.1. <https://doc.atomvm.org/v0.7.0-alpha.1/differences-with-beam.html>

BEAM Chunk Layouts and the Complete Reader

These optional implementation details accompany the developer-facing chapters. The examples retain their original OTP version qualifications.

8.1. The BEAM File Format

The definitive source of information about the beam file format is the source code of `beam_lib.erl` [\[S1\]](#), together with `beam_asm.erl` in the compiler, which writes the chunks. `beam_lib:info/1` lists the chunks of any module and `beam_lib:chunks/2` decodes the standard ones, which is a quicker way to explore the format than reading either. The description of the format that Björn Gustavsson once published on `erlang.se` is no longer online.

The beam file format is based on the interchange file format (EA IFF), with two small changes. We will get to those shortly. An IFF file starts with a header followed by a number of “chunks”. There are a number of standard chunk types in the IFF specification dealing mainly with images and music. But the IFF standard also lets you specify your own named chunks, and this is what BEAM does.

Beam files differ from standard IFF files, in that each chunk is aligned on 4-byte boundary (i.e. 32 bit word) instead of on a 2-byte boundary as in the IFF standard. To indicate that this is not a standard IFF file the IFF header is tagged with “FOR1” instead of “FORM”. The IFF specification suggests this tag for future extensions.

Beam uses form type “BEAM”. Before we read the header, a word about the notation: listings captioned **Schematic layout** describe bytes in the file. They borrow Erlang’s bit syntax, with `Field:32/big` for a 32-bit big-endian field and `Field:Size/unit:8` for `Size` bytes. `repeat N` means `N` consecutive entries; `Padding4:0..3/unit:8` means zero to three alignment bytes. Brackets around an optional field name a condition described beside it. These schematics cannot be compiled as Erlang.

Reader functions and shell commands use Erlang source listings. A reader excerpt supplies clauses to add to the growing program; ... marks omitted code. The complete runnable reader is at [Section 8.2](#). A BEAM file header has this layout:

Schematic layout: BEAMHeader

```
BEAMHeader = <<
  IfHeader:4/unit:8 = "FOR1",
  Size:32/big,           // big endian, how many more bytes are there
  FormType:4/unit:8 = "BEAM"
>>
```

Each chunk has its own header and declared payload size. Zero to three padding bytes follow the payload, aligning the next chunk to a four-byte boundary; the declared size excludes this padding.



Warning

The alignment is important for some platforms, where unaligned machine-word access would create a hardware exception (named SIGBUS in Linux). This can turn out to be a performance hit or the exception could crash the VM.

Schematic layout: BEAMChunk

```
BEAMChunk = <<
  ChunkName:4/unit:8,           // "Code", "Atom", "StrT", "LitT", ...
  ChunkSize:32/big,
  ChunkData:ChunkSize/unit:8,  // data format is defined by ChunkName
  Padding4:0..3/unit:8
>>
```

This file format prepends all areas with the size of the following area making it easy to parse the file directly while reading it from disk. To illustrate the structure and content of beam files, we will write a small program that extracts all the chunks from a beam file. To make this program as simple and readable as possible we will not parse the file while reading, instead we load the whole file in memory as a binary, and then parse each chunk. The first step is to get a list of all chunks:

```

-module(beamfile1).
-export([read/1]).

read(Filename) ->
  {ok, File} = file:read_file(Filename),
  <<"FOR1",
    Size:32/integer,
    "BEAM",
    Chunks/binary>> = File,
  {Size, read_chunks(Chunks, [])}.

read_chunks(<<Name:4/binary, Size:32, Tail/binary>>, Acc) ->
  Padding = (4 - Size rem 4) rem 4,
  <<Chunk:Size/binary, _:Padding/binary, Rest/binary>> = Tail,
  read_chunks(Rest, [{binary_to_list(Name), Size, Chunk}|Acc]);
read_chunks(<<>>, Acc) -> lists:reverse(Acc).

```

This historical sample shows the positive-count atom format and an older Type chunk. Current compiler output has different sizes and encodings:

```

> beamfile1:read("beamfile1.beam").
{1116, [{ "AtU8", 127,
  <<0,0,0,16,9,98,101,97,109,102,105,108,101,49,4,114,101,
  97,100,4,102,105,108,...>>, { "Code", 362,
  <<0,0,0,16,0,0,0,0,0,0,178,0,0,0,16,0,0,0,5,1,16,...>>,
  { "StrT", 8, <<"FOR1BEAM">>},
  { "ImpT", 88, <<0,0,0,7,0,0,0,3,0,0,0,4,0,0,0,1,0,0,0,8,...>>,
  { "ExpT", 40, <<0,0,0,3,0,0,0,15,0,0,0,1,0,0,0,15,0,0,0,...>>, { "Meta", 29,
  <<131,108,0,0,0,1,104,2,100,0,16,101,110,97,98,108,101, 100,...>>,
  { "LocT", 28, <<0,0,0,2,0,0,0,10,0,0,0,1,0,0,0,11,0,...>>, { "Attr", 40,
  <<131,108,0,0,0,1,104,2,100,0,3,118,115,110,108,0,...>>, { "CInf", 122,
  <<131,108,0,0,0,3,104,2,100,0,7,118,101,114,115,...>>, { "Dbgi", 49,
  <<131,104,3,100,0,13,100,101,98,117,103,95,105,110,...>>,
  { "Line", 31, <<0,0,0,0,0,0,0,0,0,0,0,12,0,...>>,
  { "Type", 80, <<0,0,0,1,0,0,0,4,31,255,0,0,...>>}]}}

```

Here we can see the chunk names that beam uses.

8.1.1. Atom table chunk

Either Atom or AtU8 is mandatory and contains the atoms referred to by the module. Current compilers always write AtU8, independently of the source file's encoding. The older Atom chunk stores Latin-1 names; AtU8 stores UTF-8 names.

Since Erlang/OTP 28, the default AtU8 format uses a negative signed atom count. Each name starts with its byte length encoded as an unsigned compact operand, followed by that many UTF-8 bytes. Compact operands are described later in this chapter.

Schematic layout: AtU8Chunk

```
AtU8Chunk = <<
  ChunkName:4/unit:8 = "AtU8",
  ChunkSize:32/big,
  NegativeAtomCount:32/big-signed,
  AtomEntries/binary,          % compact unsigned byte length, then UTF-8
  bytes
  Padding4:0..3/unit:8
>>
```

A positive count selects the older format, where each name has a one-byte length prefix. Atom also uses this older layout. The compiler option `no_long_atoms` writes the older AtU8 form for compatibility with OTP 27 and earlier. UTF-8 byte lengths can exceed character counts, so a reader must decode the extracted name with `binary_to_atom(Name, utf8)`.

The module name is the first atom, with encoded index 1. Atom operand index 0 represents the special `nil` operand, the empty list.

The reader below consumes exactly the declared number of atoms, including empty atom names, and separates chunk padding from payload bytes. It is an educational reader for trusted files; decoding names creates atoms in the running VM.

Let us add a decoder for the atom chunk to our Beam file reader:

```
-module(beamfile2).
-export([read/1]).

%% Educational reader for trusted BEAM files. Atom decoding creates atoms.
read(Filename) ->
  {ok, <<"FOR1", Size:32, "BEAM", Chunks/binary>>} = file:read_file
  (Filename),
  Size = byte_size(Chunks) + 4,
  {Size, parse_chunks(read_chunks(Chunks, []), [])}.

read_chunks(<<Name:4/binary, Size:32, Tail/binary>>, Acc) ->
  Padding = (4 - Size rem 4) rem 4,
  <<Chunk:Size/binary, _:Padding/binary, Rest/binary>> = Tail,
  read_chunks(Rest, [{binary_to_list(Name), Size, Chunk}|Acc]);
read_chunks(<<>>, Acc) -> lists:reverse(Acc).

parse_chunks([{Name, _Size, <<Count:32/signed, Atoms/binary>>}|Rest], Acc)
  when Name == "Atom"; Name == "AtU8" ->
  Encoding = case Name of "Atom" -> latin1; "AtU8" -> utf8 end,
```

```

LengthFormat = case Count < 0 of true -> compact; false -> byte end,
parse_chunks(Rest, [{atoms, parse_atoms(abs(Count), Atoms,
                                         Encoding, LengthFormat)}|Acc]);
parse_chunks([Chunk|Rest], Acc) ->
  parse_chunks(Rest, [Chunk|Acc]);
parse_chunks([], Acc) -> lists:reverse(Acc).

parse_atoms(0, <<>>, _Encoding, _Format) -> [];
parse_atoms(Count, Data, Encoding, Format) when Count > 0 ->
  {Length, Tail} = case Format of
    byte -> <<N:8, Bs/binary>> = Data, {N, Bs};
    compact -> {0, N, Bs} = compact(Data), {N, Bs}
  end,
  <<Name:Length/binary, Rest/binary>> = Tail,
  [binary_to_atom(Name, Encoding)|parse_atoms(Count-1, Rest, Encoding,
Format)].

%% Decode a nonnegative compact operand, retaining its three-bit tag.
compact(<<B, Rest/binary>>) when B band 8 == 0 ->
  {B band 7, B bsr 4, Rest};
compact(<<B, B1, Rest/binary>>) when B band 16 == 0 ->
  {B band 7, ((B band 16#e0) bsl 3) bor B1, Rest};
compact(<<B, Tail/binary>>) ->
  {Length, Bytes} = case B bsr 5 of
    7 -> {0, N, Bs} = compact(Tail), {N+9, Bs};
    N -> {N+2, Tail}
  end,
  <<Value:Length/unit:8, Rest/binary>> = Bytes,
  {B band 7, Value, Rest}.

```

8.1.2. Export table chunk

The chunk named ExpT (for EXPort Table) is mandatory and contains information about which functions are exported.

The format of the export chunk is:

Schematic layout: ExportChunk

```
ExportChunk = <<
  ChunkName:4/unit:8 = "ExpT",
  ChunkSize:32/big,
  ExportCount:32/big,
  [ << FunctionName:32/big,
    Arity:32/big,
    Label:32/big
    >> || repeat ExportCount ],
  Padding4:0..3/unit:8
>>
```

FunctionName is the index in the atom table.

We can extend our `parse_chunks/2` function by adding the following clause after the atom handling clause:

```
parse_chunks([{"ExpT", _Size,
  <<_NumberOfentries:32/integer, Exports/binary>>
  | Rest], Acc) ->
  parse_chunks(Rest, [{exports, parse_exports(Exports)} | Acc]);

...

parse_exports(<<Function:32/integer,
  Arity:32/integer,
  Label:32/integer,
  Rest/binary>>) ->
  [{Function, Arity, Label} | parse_exports(Rest)];
parse_exports(<<>>) -> [].
```

8.1.3. Import table chunk

The chunk named `ImpT` (for `IMPort Table`) is mandatory and contains information about which functions are imported.

The format of the chunk is:

Schematic layout: ImportChunk

```

ImportChunk = <<
  ChunkName:4/unit:8 = "ImpT",
  ChunkSize:32/big,
  ImportCount:32/big,
  [ << ModuleName:32/big,
    FunctionName:32/big,
    Arity:32/big
    >> || repeat ImportCount ],
  Padding4:0..3/unit:8
>>

```

Here `ModuleName` and `FunctionName` are indexes in the atom table.



Note

The code for parsing the import table is similar to that which parses the export table, but not exactly: both are triplets of 32-bit integers, just their meaning is different. See [Section 8.2](#) for the complete reader and its entry point.

8.1.4. Code Chunk

The chunk named `Code` contains the beam code for the module and is mandatory. The format of the chunk is:

Schematic layout: CodeChunk

```

CodeChunk = <<
  ChunkName:4/unit:8 = "Code",
  ChunkSize:32/big,
  SubSize:32/big,
  InstructionSet:32/big,          % Must match code version in the emulator
  OpcodeMax:32/big,
  LabelCount:32/big,
  FunctionCount:32/big,
  Code:(ChunkSize-4-SubSize)/binary, % payload minus SubSize word and info
  Padding4:0..3/unit:8
>>

```

The field `SubSize` stores the byte length of the information fields after the `SubSize` word, normally 16 bytes. This makes it possible to add new information fields in the code chunk without breaking older loaders.

The `InstructionSet` field indicates which version of the instruction set the file uses. The version number is increased if any instruction is changed in an incompatible way.

The `OpcodexMax` field indicates the highest number of any opcode used in the code. New instructions can be added to the system in a way such that older loaders still can load a newer file as long as the instructions used in the file are within the range the loader knows about.

The field `LabelCount` contains the number of labels so that a loader can preallocate a label table of the right size in one call. The field `FunctionCount` contains the number of functions so that the functions table could also be preallocated efficiently.

The `Code` field contains instructions, chained together, where each instruction has the following format:

Schematic layout: Instruction

```
Instruction = <<
  InstructionCode:8,
  [beam_asm:encode(Argument) || repeat Arity]
>>
```

Here `Arity` is fixed per opcode in `genop.tab`, from which `beam_makeops` generates the table in `beam_opcodes.hrl` when the compiler is built from source.

The encoding produced by `beam_asm:encode` is explained below in the [Compact Term Encoding](#) section.

We can parse out the code chunk by adding the following code to our program:

```
parse_chunks([{"Code", Size, <<SubSize:32/integer,Chunk/binary>>
               } | Rest], Acc) ->
  <<Info:SubSize/binary, Code/binary>> = Chunk,
  %% The chunk size excludes its header; subtract the SubSize word.
  OpcodeSize = Size - SubSize - 4,
  <<OpCodes:OpcodeSize/binary, _Align/binary>> = Code,
  parse_chunks(Rest, [{code, parse_code_info(Info), OpCodes}
                      | Acc]);

...

parse_code_info(<<Instructionset:32/integer,
                OpcodeMax:32/integer,
                NumberOfLabels:32/integer,
                NumberOfFunctions:32/integer,
                Rest/binary>>) ->
  [{instructionset, Instructionset},
   {opcode_max, OpcodeMax},
   {number_of_labels, NumberOfLabels},
```

```
{numberOfFunctions, NumberOfFunctions} |
case Rest of
  <<>> -> [];
  _ -> [{newinfo, Rest}]
end].
```

We will learn how to decode the beam instructions in a later chapter, aptly named “BEAM Instructions”.

8.1.5. String table chunk

The `StrT` chunk holds byte sequences referenced by instructions, for example bytes used during binary construction or matching. The compiler always writes it and `beam_lib` lists it as mandatory, although the loader accepts a file without it. It can be empty even when the source contains a string: a function returning “hello” places that list-valued constant in `LitT`. Inspect the code operands together with both chunks to distinguish these representations.

The format of the chunk is:

Schematic layout: StringChunk

```
StringChunk = <<
  ChunkName:4/unit:8 = "StrT",
  ChunkSize:32/big,
  Data:ChunkSize/binary,
  Padding4:0..3/unit:8
>>
```

The string chunk can be parsed easily by just converting its binary payload into a list of bytes:

```
parse_chunks([{"StrT", _Size, <<Strings/binary>>} | Rest], Acc) ->
  parse_chunks(Rest, [{strings, binary_to_list(Strings)} | Acc]);
```

8.1.6. Attributes Chunk

The chunk named `Attr` is optional, but some OTP tools expect the attributes to be present. The release handler expects the "vsn" attribute to be present. You can get the version attribute from a file with: `beam_lib:version(Filename)`, this function assumes that there is an attribute chunk with a "vsn" attribute present.

The format of the chunk is:

Schematic layout: AttributesChunk

```
AttributesChunk = <<
  ChunkName:4/unit:8 = "Attr",
  ChunkSize:32/big,
  Attributes:ChunkSize/binary,
  Padding4:0..3/unit:8
>>
```

We can parse the attribute chunk like this:

```
parse_chunks([{"Attr", Size, Chunk} | Rest], Acc) ->
  <<Bin:Size/binary, _Pad/binary>> = Chunk,
  Attribs = binary_to_term(Bin),
  parse_chunks(Rest, [{attributes, Attribs} | Acc]);
```

8.1.7. Compilation Information Chunk

The chunk named `CInf` is optional, but some OTP tools expect the information to be present. The format of the chunk is:

Schematic layout: CompilationInfoChunk

```
CompilationInfoChunk = <<
  ChunkName:4/unit:8 = "CInf",
  ChunkSize:32/big,
  Data:ChunkSize/binary,
  Padding4:0..3/unit:8
>>
```

We can parse the compilation information chunk like this:

```
parse_chunks([{"CInf", Size, Chunk} | Rest], Acc) ->
  <<Bin:Size/binary, _Pad/binary>> = Chunk,
  CInfo = binary_to_term(Bin),
  parse_chunks(Rest, [{compile_info, CInfo} | Acc]);
```

8.1.8. Local Function Table Chunk

The chunk named `LocT` is optional and intended for cross reference tools.

The format is the same as that of the export table:

Schematic layout: LocalFunTableChunk

```
LocalFunTableChunk = <<
  ChunkName:4/unit:8 = "LocT",
  ChunkSize:32/big,
  FunctionCount:32/big,
  [ << FunctionName:32/big,
    Arity:32/big,
    Label:32/big
    >> || repeat FunctionCount ],
  Padding4:0..3/unit:8
>>
```



Note

The code for parsing the local function table is basically the same as that for parsing the export and the import table, and we can actually use the same function to parse entries in all tables. See [Section 8.2](#) for the complete reader and its entry point.

8.1.9. Literal Table Chunk

The chunk named `LitT` is optional and contains all literal values from the module source which are not immediate values. It is only present if there is at least one such literal.



Version Info

Up to Erlang/OTP 27 the literal table was always compressed with `zlib`. Since Erlang/OTP 28.0 it is written uncompressed by default, and the word after the chunk header is zero to say so; the compiler option `compressed_literals` restores the old form, in which that word is the uncompressed size.

The format of the chunk is:

Schematic layout: LiteralTableChunk

```
LiteralTableChunk = <<
  ChunkName:4/unit:8 = "LitT",
  ChunkSize:32/big,
  UncompressedSize:32/big,      % 0: the table follows uncompressed (OTP 28+)
                                % N: the table is zlib compressed, N bytes
  inflated
  Table:(ChunkSize-4)/binary,
  Padding4:0..3/unit:8
>>
```

Zero is not a valid uncompressed size, since the chunk is only written when there is at least one literal, which is what makes the word safe to use as a flag. The table itself has the same layout in both cases:

Schematic layout: LiteralTable

```
LiteralTable = <<
  Count:32/big,
  [ <<Size:32/big, Literal:Size/binary>> || repeat Count ]
>>
```

Each literal is encoded with the External Term Format, so `binary_to_term/1` decodes it, and a compressed table is inflated with `zlib:uncompress/1`. We can parse both forms like this:

```
parse_chunks([{"LitT", _ChunkSize,
               <<0:32, Table/binary>>} | Rest], Acc) ->
  parse_chunks(Rest, [{literals, parse_literal_table(Table)} | Acc]);
parse_chunks([{"LitT", _ChunkSize,
               <<UncompressedSize:32, Compressed/binary>>} | Rest], Acc) ->
  Table = zlib:uncompress(Compressed),
  parse_chunks(Rest, [{literals, parse_literal_table(Table)} | Acc]);
...

parse_literal_table(<<_Count:32, Literals/binary>>) ->
  parse_literals(Literals).

parse_literals(<<Size:32, Literal:Size/binary, Tail/binary>>) ->
  [binary_to_term(Literal) | parse_literals(Tail)];
parse_literals(<<>>) -> [].
```

Because the literals are external term format, the table follows that format's evolution. Two recent tags are worth knowing. `LOCAL_EXT` (Erlang/OTP 26.0) marks a term that is only

meaningful on the node that produced it, so it never appears in a literal table. `RECORD_EXT` (Erlang/OTP 29.0) encodes a native record: tag 67, a 32-bit field count, a flags byte whose lowest bit says whether the record is exported, then the module name, the record name, the field names and the field values, each as an ordinary encoded term. `term_to_binary/1` of a two field record `#pt{x = 1, y = 2}` defined in module `rec` gives, byte by byte, 131,67, 0,0,0,2, 0, 119,3,"rec", 119,2,"pt", 119,1,"x", 119,1,"y", 97,1, 97,2, where 119 is `SMALL_ATOM_UTF8_EXT` and 97 is `SMALL_INTEGER_EXT`. See [The Erlang Type System and Tags](#) for the record type itself and the external term format documentation for the full tag table.

8.1.10. Debug Information Chunk



Version Info

Since Erlang/OTP 20.0 the abstract code lives in the `Dbgi` chunk, wrapped in a `debug_info_v1` term that names the backend that produced it. Before that it was stored bare in a chunk named `Abst`, which `beam_lib` still reads.

If you give the flag `debug_info` to the compiler it stores the abstract syntax tree of the module in the chunk named `Dbgi`. OTP tools like the debugger, `Xref` and `Dialyzer` read it. The format of the chunk is:

Schematic layout: `DebugInfoChunk`

```
DebugInfoChunk = <<
  ChunkName:4/unit:8 = "Dbgi",
  ChunkSize:32/big,
  DebugInfo:ChunkSize/binary,    % term_to_binary({debug_info_v1, Backend,
Data})
  Padding4:0..3/unit:8
>>
```

`Backend` is the module that knows how to turn `Data` into abstract code, `erl_abstract_code` for modules compiled from Erlang, and for that backend `Data` is `{AbstractCode, CompilerOptions}`. Other languages on the BEAM supply their own backend, which is the point of the indirection: `beam_lib:chunks(File, [abstract_code])` asks the backend rather than assuming Erlang. The chunk can be encrypted, see [Section 8.1.12](#) below. We can parse it like this:

```

parse_chunks([{"Dbgi", _ChunkSize, <<>>} | Rest], Acc) ->
    parse_chunks(Rest, Acc);
parse_chunks([{"Dbgi", _ChunkSize, DebugInfo} | Rest], Acc) ->
    {debug_info_v1, Backend, Data} = binary_to_term(DebugInfo),
    parse_chunks(Rest, [{debug_info, Backend, Data} | Acc]);

```



Version Info

Erlang/OTP 28.0 added a second, separate debug chunk, DbgB, written when a module is compiled with `beam_debug_info`. For each debug point in the code it records the stack frame size and which variables live in which registers, so that the debugger introduced with OTP 28 can inspect variables in code that runs natively. Its contents are compact term encoded instructions rather than external term format, and `beam_lib:strip/1` keeps the chunk since Erlang/OTP 29.0.

8.1.11. Additional BEAM Chunks

Several chunks have been added since the format was first documented. A module compiled by Erlang/OTP 29 can carry, beyond the chunks covered above, AtU8, FunT, Line, Type, Meta, DbgB and Docs.



Version Info

Dbgi arrived in Erlang/OTP 20.0, Type and Meta in 25.0, and DbgB in 28.0. OTP 27.0 added compiler support for EEP 59 documentation attributes, which use the existing EEP 48 Docs chunk format. OTP 28.0 extended the documentation metadata. The literal table changed in 28.0 and the external term format gained `RECORD_EXT` in 29.0; both are described above.

UTF-8 Atom Chunk (AtU8): Current compilers always write this chunk. Its negative-count format and the older positive-count format are described in [Section 8.1.1](#).

Function Table (FunT): One entry per fun (anonymous function) in the module, 24 bytes each: the atom index of the name the compiler gave the fun, its arity, the label of its code, its index among the funs of the module, the number of free variables it captures, and an `oldUniq` field that `beam_asm` fills in from the MD5 checksum of the code once that is known.

Schematic layout: FunTableChunk

```
FunTableChunk = <<
  ChunkName:4/unit:8 = "FunT",
  ChunkSize:32/big,
  FunCount:32/big,
  [ <<FunctionName:32/big, Arity:32/big, Label:32/big,
    Index:32/big, NumFree:32/big, OldUniq:32/big>> || repeat FunCount ],
  Padding4:0..3/unit:8
>>
```

Line Chunk (Line): The mapping from positions in the code to source lines. A version word (0), a flags word, the number of line instructions in the code, the number of distinct line items, the number of file names other than the module's own source, then the line items and the file names.

Schematic layout: LineChunk

```
LineChunk = <<
  ChunkName:4/unit:8 = "Line",
  ChunkSize:32/big,
  Version:32/big = 0,
  Flags:32/big,
                                % bit 0: executable_line instructions present
                                % bit 1: force line counters (used by cover)

  NumLineInstructions:32/big,
  NumLineItems:32/big,
  NumFileNames:32/big,
  LineItems/binary,
                                % compact term encoded: an atom tagged file
                                % index when it changes, an integer tagged

  line
  [ <<NameLength:16/big, FileName:NameLength/binary>> || repeat NumFileNames
  ],
  Padding4:0..3/unit:8
>>
```

The flags word is where coverage metadata lives. Compiling with `line_coverage` (Erlang/OTP 27.0) inserts an `executable_line` instruction at every executable line and sets bit 0. The runtime ignores those instructions unless it was started with `+JPcover`, or `code:set_coverage_mode/1` was called before the module was loaded; after that `code:get_coverage/2` reports which lines ran. The cover tool adds `force_line_counters`, bit 1, so that the modules it instruments count lines whatever the system wide mode. This is the mechanism behind native coverage in the JIT.

Type Chunk (Type): The table of types the compiler inferred, which typed register operands in the code refer to by index. Format:

Schematic layout: TypeChunk

```
TypeChunk = <<
  ChunkName:4/unit:8 = "Type",
  ChunkSize:32/big,
  % Version 4 at OTP 29 (2 in OTP 26, 3 in OTP 27)
  Version:32/big,
  Count:32/big,
  TypeEntries:binary,
  Padding4:0..3/unit:8
>>
```

Schematic layout: TypeEntry

```
TypeEntry = <<
  % One bit per base type: atom, bitstring, cons, float,
  % fun, integer, map, nil, pid, port, reference, tuple,
  % record. Bits 13 to 15 say which extras follow.
  TypeBits:16/big,
  [ LowerBound:64/big-signed ], % if bit 13
  [ UpperBound:64/big-signed ], % if bit 14
  % If bit 15: bitstring size unit minus one
  [ Unit:8 ]
>>
```

The entries are not fixed size and are not external term format, so neither `binary_to_term/1` nor a fixed stride decodes them; a module whose two types carry no extras has a four byte table. The encoding is `beam_types:encode_ext/1` in the compiler and `beam_types:decode_ext/1` reads it back. The complete reader in `code/beam_modules_chapter/src/beamfile.erl` splits version-4 entries by their flags and retains each entry as bytes. It returns other Type versions as raw chunks, preserving their version and payload for a version-aware decoder. Exact tuple sizes from the compiler's internal analysis are discarded by this encoding; instructions can still carry tuple sizes as separate operands. What the types mean and how the JIT uses them is described in [The Compiler](#) and [The JIT Compiler](#).

Metadata Chunk (Meta): A `term_to_binary` encoded property list. Its one standard entry is `enabled_features`, the list of selectable language features (EEP 60) the module was compiled with, which the loader checks against the running system; see [The Compiler](#). The compiler's `extra_chunks` option can add entries.

Schematic layout: MetaChunk

```
MetaChunk = <<
  ChunkName:4/unit:8 = "Meta",
  ChunkSize:32/big,
  Meta:ChunkSize/binary,      % term_to_binary([enabled_features, [...]],
  ...)
  Padding4:0..3/unit:8
>>
```

Documentation Chunk (Docs): The EEP 48 storage format predates OTP 27 and is documented in the OTP 26 documentation[S2]. OTP 27 added compiler support for the documentation attributes of EEP 59, `-moduledoc` and `-doc`, compiled into the EEP 48 documentation format, so that `h(Module)` in the shell and any tool that calls `code:get_doc/1` find the documentation inside the module itself. The chunk is `term_to_binary` of a `docs_v1` record:

Schematic layout: DocsChunk

```
DocsChunk = <<
  ChunkName:4/unit:8 = "Docs",
  ChunkSize:32/big,
  Docs:ChunkSize/binary,
  Padding4:0..3/unit:8
>>

Docs = term_to_binary(
  {docs_v1,
    Anno,                                % where -moduledoc appears
    erlang,                              % the language
    <<"text/markdown">>,                 % the format of the doc text
    ModuleDoc,                           % #{<<"en">> => Text}, none or hidden
    Metadata,                             % #{otp_doc_vsn => {1,0,0}, behaviours =>
    [...],                                % source_anno => ..., source_path => ...,
    ...}
    [ {function, Name, Arity}, Anno, [Signature], Doc, EntryMetadata} | ... ]
  })
```

Types and callbacks appear as entries of `kind type` and `callback`. A function without a `-doc` attribute still gets an entry, with `none` as its documentation, and `-doc false` gives `hidden`. The `no_docs` compiler option leaves the chunk out. The metadata records the source path of the file, which is worth knowing when comparing builds.

Parsing the term encoded chunks is one clause each:

```

parse_chunks([{"Meta", _Size, Chunk} | Rest], Acc) ->
    parse_chunks(Rest, [{meta, binary_to_term(Chunk)} | Acc]);
parse_chunks([{"Docs", _Size, Chunk} | Rest], Acc) ->
    parse_chunks(Rest, [{docs, binary_to_term(Chunk)} | Acc]);

```

8.1.12. Encryption

You can encrypt the debug information in a BEAM file and supply the key when a tool such as Debugger or Xref needs to read it. The executable code remains available to the loader.

To encrypt the debug information, give the compiler a key with `{debug_info_key, Key}`. The key is a string; the OTP documentation recommends at least 32 characters, mixing upper and lower case letters, digits and special characters. OTP 29 uses `des3_cbc`, triple DES in Cipher Block Chaining mode, with keys derived from the string using `erlang:md5/1`.

To read the information back, register a key-returning function with `beam_lib:crypto_key_fun/1`. Alternatively, store keys in an `.erlang.crypt` file and compile with `encrypt_debug_info`. The file holds a list of tuples of the form `{debug_info, Mode, Module, Key}`. Mode is `des3_cbc`; Module is an atom for a particular module or `[]` for all modules. Key is the non-empty key string.

Without a registered key function, `beam_lib` searches for `.erlang.crypt` in the current directory, then the user's home directory, then `filename:basedir(user_config, "erlang")`. It uses the key in the first tuple whose Mode and Module match.

8.1.13. Compression

To compress the whole BEAM file, compile it with:

```
erlc +compressed module.erl
```

From Erlang, use `compile:file(Module, [compressed])`. The compiler writes the file through the file module's gzip output mode, so its first bytes are gzip magic bytes. Decompress this wrapper before parsing the ordinary IFF chunks. The loader also has to decompress it when loading the module; execution then uses the loaded code as usual.

This is separate from `compressed_literals`, which selects `zlib` compression inside the `LitT` chunk. Compare the file sizes and loading time for your module if the compression trade-off matters to your deployment.

8.1.14. Compact Term Encoding

An instruction needs operands such as a register number, a label or an integer. Let's examine how `beam_asm:encode` packs those operands into bytes in the Code chunk. The tag tells the loader how to interpret the value; these bytes have their own format, separate from the heap layouts in [The Erlang Type System and Tags](#).

Here's the tag format, including the typed register subtype added in OTP 25:

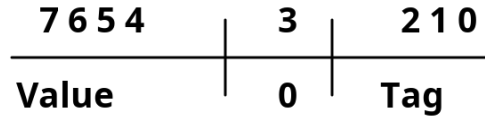
Table 2. Operand tag encoding: the low three bits give the tag, the high five bits the value or the extended tag

Bits 7 to 3	Bits 2 to 0	Meaning	Name in <code>beam_opcodes.hrl</code>
value	000	Unsigned operand	<code>tag_u</code>
value	001	Integer	<code>tag_i</code>
value	010	Atom	<code>tag_a</code>
value	011	X register	<code>tag_x</code>
value	100	Y register	<code>tag_y</code>
value	101	Label	<code>tag_f</code>
value	110	Character	<code>tag_h</code>
	111	Extended, see the rows below	<code>tag_z</code>
00000	111	Extended: inline float, historical; floats are now literals	
00010	111	Extended: list	
00100	111	Extended: floating point register	
00110	111	Extended: allocation list	
01000	111	Extended: literal	
01010	111	Register with type hint (new in OTP 25)	

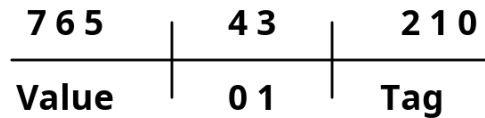
The extended subtypes used by OTP 29 are list (1), floating-point register (2), allocation list (3), literal reference (4), and typed register (5). Their first bytes are respectively 16#17, 16#27, 16#37, 16#47 and 16#57. Floating-point values themselves are stored in the literal table, separately from floating-point register operands.

It uses the first 3 bits of a first byte to store the tag which defines the type of the following value. If the bits were all 1 (special value 7 or `?tag_z` from `beam_opcodes.hrl`), then a few more bits are used.

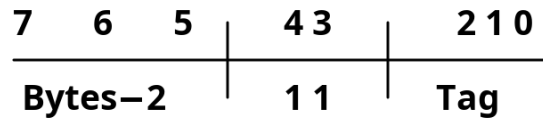
For values under 16, the value is placed entirely into bits 4-5-6-7 having bit 3 set to 0:



For values under 2048 (16#800) bit 3 is set to 1, marks that 1 continuation byte will be used and 3 most significant bits of the value will extend into this byte's bits 5-6-7:



Larger and negative values are first converted to bytes. Then if the value takes 2..8 bytes, bits 3-4 will be set to 1, and bits 5-6-7 will contain the (Bytes-2) size for the value, which follows:



If the following value is greater than 8 bytes, then all bits 3-4-5-6-7 will be set to 1, followed by a nested encoded unsigned literal (macro `?tag_u` in `beam_opcodes.hrl`) value of (Bytes-9):8, and then the data:

Table 3. An operand too large for the tag byte: the size escape, then the size, then the data

Bits 7 to 3	Bits 2 to 0	Then follows	And then
11111	tag	a nested encoded literal, the size minus 9	the data bytes

Tag Types

When reading compact term format, the resulting integer may be interpreted differently based on what is the value of Tag.

- Tag `u` holds an untagged unsigned operand, whose meaning depends on the instruction. An extended literal operand (subtype 4) is followed by an unsigned index into the literal table.
- Atom operands use the one-based atom-table index directly. Index 1 identifies the module atom; index 0 is the special NIL operand.
- For labels 0 means invalid value.
- If tag is character, the value is unsigned unicode codepoint.
- An extended list starts with its unsigned element count, followed by that many encoded operands. The instruction determines their meaning: `select_val` uses value/label pairs, while `put_tuple2` lists tuple elements and `init_yregs` lists registers. A list need not contain pairs.
- Tag Extended Register with type hint is followed by two more encoded terms: the register itself (an X or Y register tag) and an unsigned index into the Type chunk, see [Section 8.1.11](#).

Type hints let the JIT use types inferred by the compiler when selecting native instructions. See [The JIT Compiler](#) for examples.

Refer to `beam_asm:encode/2` in the compiler application for details about how this is encoded. Tag values are presented in this section, but also can be found in `compiler/src/beam_opcodes.hrl` which is generated by `beam_makeops`.

8.2. Running the Complete Reader

The complete implementation is `code/beam_modules_chapter/src/beamfile.erl`. It combines the incremental decoders above and preserves chunks it does not interpret. Let's use it to find a function we recognize. The file `code/beam_modules_chapter/src/beam_sample.erl` contains:

```
-module(beam_sample).
-export([answer/0]).

answer() -> {hello, 42}.
```

From the repository root, start a fresh OTP 29 shell. Compile both modules, then read the sample's atom table:

```
1> c("code/beam_modules_chapter/src/beamfile").
{ok,beamfile}
2> c("code/beam_modules_chapter/src/beam_sample").
{ok,beam_sample}
3> {_Size, Chunks} = beamfile:read("beam_sample.beam"),
    {atoms, Atoms} = lists:keyfind(atoms, 1, Chunks).
{atoms,[beam_sample,answer,module_info,erlang,
    get_module_info]}
4> {exports, Exports} = lists:keyfind(exports, 1, Chunks).
{exports,[{3,1,6},{3,0,4},{2,0,2}]}
5> lists:sort([{lists:nth(Name, Atoms), Arity, Label} ||
    {Name, Arity, Label} <- Exports]).
[{answer,0,2},{module_info,0,4},{module_info,1,6}]
```

The export `{2,0,2}` names atom `2`, `answer`, with arity `0` and entry label `2`. The two other exports are the compiler-generated `module_info` functions. We can now connect the integers in ExpT to names in AtU8.

Where did `{hello,42}` go? Look in the literal table and compare it with the function's result:

```
6> lists:keyfind(literals, 1, Chunks).
{literals,[{hello,42}]}
7> beam_sample:answer().
{hello,42}
```

The tuple is stored as one literal. Its atom `hello` travels inside that literal's external-term encoding, which is why it does not appear in our instruction operand atom table above. Other compiler versions or options can change the table order and labels; resolve the indexes through the file's own atom table each time.

The reader is intended for trusted teaching inputs: atom decoding creates atoms, and term-encoded chunks are decoded as Erlang terms. It expects an ordinary FOR1 file; use `beam_lib` to inspect a whole-file gzip-compressed BEAM without adding wrapper handling here. The fixture checks in `code/beam_modules_chapter/test/reader_checks.escript` exercise the modern and historical layouts discussed in this chapter.

Sources and links

[S1] Erlang/OTP Project. Erlang/OTP source code: beam_lib.erl. OTP-29.0; lib/stdlib/src/beam_lib.erl.
<https://github.com/erlang/otp>

[S2] Erlang/OTP Project. EEP-48: Documentation storage and format. Erlang/OTP 26 documentation.
https://www.erlang.org/docs/26/apps/kernel/eep48_chapter.html

Implementation detail: the BEAM loader

These OTP 29 implementation notes accompany the loader chapter. Start with the `sel` example there; return here when investigating a loader rule or debugging ERTS itself. The rules and generated code are version-specific.

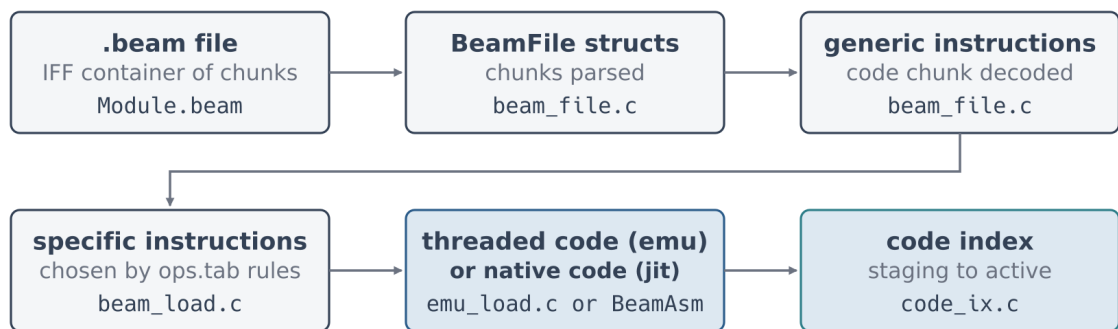


Figure 9. The loader pipeline, from file to running code

9.1. Source Files and Generated Tables

The source lives under `erts/emulator/beam`[\[S1\]](#) and is laid out by responsibility:

File	Role
<code>beam_file.c</code> , <code>beam_file.h</code>	Parses the IFF container and every chunk into a <code>BeamFile</code> struct, and decodes the code chunk one instruction at a time.
<code>beam_load.c</code>	The shared loading loop: <code>erts_prepare_loading</code> , <code>erts_finish_loading</code> , the transformation engine, and the choice of specific instruction.
<code>predicates.tab</code> , <code>generators.tab</code>	Guard functions and generator functions used by transformation rules in both flavors.
<code>emu/ops.tab</code> , <code>emu/predicates.tab</code> , <code>emu/generators.tab</code>	The interpreter's specific instructions and its own transformation rules.

File	Role
emu/instrs.tab, emu/*_instrs.tab, emu/macros.tab	The C implementation of every interpreter instruction, in the beam_makeops mini language.
emu/emu_load.c, emu/beam_emu.c	The interpreter's back end: operand encoding, packing, and the process_main loop that includes the generated instruction code.
jit/x86/ops.tab, jit/arm/ops.tab (and their predicates.tab, generators.tab)	The JIT's specific instructions and transformation rules, one set per architecture.
jit/asm_load.c, jit/beam_jit_main.cpp, jit/x86/	The JIT's back end: operand massaging, the C to C++ boundary, and the emitter functions.
utils/beam_makeops	The Perl script that turns all the .tab files into C, C++ and Erlang at build time.

None of the .tab files is read at runtime. When OTP itself is built, beam_makeops compiles them into tables and code, and the build runs it once per flavor with the flavor's own table list. For the emulator it writes beam_opcodes.h and beam_opcodes.c, which hold the opcode tables, the operand signatures, and the transformation rules compiled into C. For the interpreter it also writes beam_hot.h, beam_warm.h and beam_cold.h, the instruction bodies that beam_emu.c includes inside process_main. For the JIT it writes beamasm_emit.h, a switch statement that routes each specific instruction to its emitter, and beamasm_protos.h, the emitter prototypes. The output lands in the flavor's build directory, for example erts/emulator/x86_64-pc-linux-gnu/opt/jit/ or ../opt/emu/. The same script, run with -compiler on lib/compiler/src/genop.tab alone, produces beam_opcodes.erl and beam_opcodes.hrl for the compiler, which is how the two sides stay in agreement about opcode numbers.

The two flavors differ in how much work they push into the loader. At Erlang/OTP 29.0 on x86-64, genop.tab defines generic opcodes up to 191. The interpreter tables expand those into 422 generic instructions (the extra ones are internal, created by transformations) and 493 specific instructions. The JIT tables produce 369 generic and 245 specific instructions. The interpreter needs many specific variants because each one saves an operand decode or a dispatch at runtime; the JIT specializes inside the emitter instead, so one i_move s d covers every combination of source and destination.

9.2. Parsing and Decoding

9.2.1. Parsing the File

Before the loader can inspect `f/1`, it needs the atoms `sel`, `f`, `a` through `d`, and the code that refers to them. These live in separate chunks of our file, just as they did in the reader in [Modules and The BEAM File Format](#).

`beamfile_read` in `beam_file.c` [\[S2\]](#) does the work described in Erlang in [Modules and The BEAM File Format](#), only in C and with less patience. It checks the `FOR1` header and the BEAM form id, then looks up the chunks by their four letter ids: `AtU8`, `Code`, `StrT`, `ImpT`, `ExpT`, `FunT`, `LitT`, `Attr`, `CInf`, `Line`, `LocT`, `Type`, `Meta`, `DbgB` and `Recs`. An atom, code, export or import chunk that is missing or corrupt aborts the load with a matching error, and a file that still carries the pre-UTF-8 Atom chunk is rejected with a request to recompile. The optional chunks (lambdas, literals, lines, types, debug information, records) are parsed when present, and the local function table only when the JIT has been asked to dump assembly, since nothing else needs the names.

`erts_prepare_loading` then applies two checks on the code header. A file whose highest opcode exceeds `MAX_GENERIC_OPCODE` (191 at Erlang/OTP 29.0) was compiled for a newer runtime, and the error message says so. A file whose highest opcode is lower than that of `swap/2` was produced by Erlang/OTP 22 or earlier. Since OTP 23 the compiler records an artificial minimum in that field precisely so that the loader can tell old files apart; the minimum has been raised in later releases, to `update_record/5` at OTP 29, while the loader's check stays at `swap/2`. Such a file is refused with a request to recompile with OTP 25 or later. Older loaders used transformation rules that matched the problematic instruction sequences one by one; the header check replaced all of them.

9.2.2. Reading the Code Chunk

The code chunk is decoded lazily by `beamfile_get_code` and `beamcodereader_next`. Each call returns one generic instruction as a `BeamOp`: an opcode, an arity, and an array of operands where each operand is a `BeamOpArg` holding a type tag and a value. The tags are the single letters used throughout the transformation tables: `x` and `y` for registers, `i` for a small integer, `a` for an atom, `n` for `[]`, `q` for a literal that lives on the heap (a list, a tuple, a float, a bignum), `f` for a non-zero label, `p` for the zero label, and `u` for an untagged machine word. Register numbers, label numbers and atom indexes come out of the compact term encoding

described in [Modules and The BEAM File Format](#), and an atom index is replaced by the atom itself as it is read.

Two things happen during decoding that the file does not show. An instruction with a `{list,[...]}` operand, such as `select_val/3`, is flattened: the list becomes a `u` operand holding the element count, followed by the elements themselves, so `select_val {x,0} {f,1} {list,[a,{f,4},b,{f,5}]}` arrives at the rules as `select_val x0 f1 u=4 a f4 b f5`. The reader also recognizes the sequence label `| line | func_info` that starts every function and replaces it with the internal instructions `int_func_start/5` and `int_func_end/2`, so that the rest of the loader can find function boundaries without pattern matching on three instructions at a time.

9.3. Transformation Engine and Rules

9.3.1. The Loading Loop

The heart of the shared loader is `load_code` in `beam_load.c`. It keeps the decoded generic instructions in a linked list and runs the following loop until it meets `int_code_end`:

1. Read the next generic instruction and append it to the list.
2. If the instruction at the head of the list has transformation rules, run the transformation engine on the list.
3. If a rule fired, the list has changed; go back to step 2 with the new head. If the engine reports that it needs more instructions to decide (`TE_SHORT_WINDOW`), go back to step 1. If no rule matched, continue.
4. Choose a specific instruction for the head of the list.
5. Hand the instruction to the flavor's back end (`beam_load_emit_op`) and remove it from the list.

The choice in step 4 uses bit masks. Each generic opcode points at a run of specific instructions in the `opc` table, sorted by `beam_makeops` so that the most specific operand types come first: `move_nx` sits before `move_cx`, so an instruction whose source is `[]` gets the variant that does not store the operand at all. The loader builds one bit per operand type for the instruction in hand, three 32-bit words covering up to six operands, and takes the first specific instruction whose masks cover it. That is why a specific instruction may have at most six operands unless it is the only member of its family. When no variant matches and

no rule exists, the generic instruction is obsolete and the load fails with a request to recompile.

The transformation engine itself is an interpreter for a small bytecode. `beam_makeops` compiles every rule into a sequence of test, bind and construct operations stored in `beam_opcodes.c`, and the generated file keeps the original rule in a comment above each program, which makes it a useful place to look when a rule does not fire as expected.

9.3.2. The Transformation Language

The rules for the interpreter are in `emu/ops.tab`[\[S3\]](#) and the rules for the JIT in `jit/x86/ops.tab`[\[S4\]](#) and its AArch64 twin; the shared rule language is documented in `beam_makeops.md`[\[S5\]](#). Rules are tried in the order they appear in the file, so just as in Erlang pattern matching the first rule that matches wins, and the engine restarts from the top after every successful rewrite. A simple rule from the interpreter’s table looks like this:

```
move S x==0 | return => move_return S
```

This combines a move of any source to `x(0)` followed by a return into one `move_return` instruction. Reading it apart:

move

is the first instruction the pattern has to match. It may be a generic instruction from the compiler or an internal instruction produced by an earlier rule.

S

is a variable and binds the operand whatever its type. Variables start with an uppercase letter, may not be repeated on the left-hand side, and must be used on the right-hand side or in a predicate; an operand that is named only for documentation gets a leading underscore, as in `_Loc`.

x==0

is a constraint: the operand must be an X register, and its number must be 0. Type letters can be combined and can bind a variable at the same time, so `D=xy` matches either register kind and binds it to `D`, and `Fail=f` binds a non-zero label. An atom is written as it is spelled in the C source, for example `a==am_true`.

|

separates instructions on either side of the arrow.

return

is the second instruction in the pattern.

⇒

separates the pattern from the instructions to construct.

move_return S

is the instruction to construct, using the bound variable. The right-hand side may build several instructions separated by `|`, may create operands from type letters with default values (`x` alone means `X` register 1023, a scratch register the compiler never uses), and may be the single symbol `_` to delete the matched instructions.



Version Info

Erlang/OTP 25.0 tightened the syntax. A rule that removes instructions now writes `_` on the right-hand side instead of leaving it empty, and a long rule is broken only after `|` or `=>`; the trailing backslash that older tables used for line continuation is gone.

Two families of constraints deserve a mention because the tables use them constantly. `u$is_bif` and `u$is_not_bif` test whether an import table index refers to a BIF, which is how a `call_ext` that names a BIF is rewritten into a direct BIF call instruction while a call to an Erlang function stays a function call. `u$bif:Module:Name/Arity` matches a call to one particular BIF, so that a call to `erlang:element/2`, for example, can be replaced by a dedicated instruction that never enters the BIF.

9.3.3. Predicates and Generators

When a constraint needs more logic than a type letter can express, the rule calls a predicate written in C, and when the instruction to construct depends on a computation, it calls a generator. Both live in files named `predicates.tab` and `generators.tab`: shared ones in `erts/emulator/beam`, interpreter-only ones in `emu/`, and JIT-only ones in `jit/x86/` and `jit/arm/`. Take the rule that turns a `select_val` over dense integers into a jump table, as written for the interpreter:

```
is_integer Fail=f S | select_val S2=s Fail2=f Size=u Rest=* |
  equal(Fail, Fail2) | equal(S, S2) |
  use_jump_tab(Size, Rest, 2) =>
    jump_tab(S, Fail, Size, Rest)
```

The pattern matches an `is_integer` test followed by a `select_val` on the same source with the same failure label. Because variables may not repeat, the two occurrences get different names and the `equal/2` predicate ties them together. `Rest=*` binds the variable number of operands that follow the count, and `use_jump_tab/3` is a predicate that walks them: every value must be an integer, there must be at least the given number of values (two here; the JIT asks for six, because its dispatch code costs about 40 bytes), and the range from the smallest to the largest value may be at most twice the number of values, so that the table stays reasonably dense. If it returns non-zero, the generator `jump_tab/4` builds the replacement instruction. A predicate is defined with a `pred.` prefix and a generator with `gen.`, and both see the loader state through a variable `S`:

```
pred.use_jump_tab(Size, Rest, MinSize) {
  Sint min, max;
  Sint i;

  if (Size.val < 2 * MinSize || Size.val % 2 != 0) {
    return 0;
  }
  if (Rest[0].type != TAG_i || Rest[1].type != TAG_f) {
    /* Atoms. Can't use a jump table. */
    return 0;
  }
  min = max = Rest[0].val;
  for (i = 2; i < Size.val; i += 2) {
    if (Rest[i].type != TAG_i || Rest[i+1].type != TAG_f) {
      return 0;
    }
    if (Rest[i].val < min) {
      min = Rest[i].val;
    } else if (max < Rest[i].val) {
      max = Rest[i].val;
    }
  }
  return max - min <= Size.val;
}
```

The generator side allocates a new `BeamOp` with `$NewBeamOp`, names it with `$BeamOpNameArity`, fills in the operands and returns it. The `jump_tab` generator computes the range, allocates one slot per value in it, fills every slot with the failure label and then overwrites the slots that have a clause. A `select_val` with a single value gets no table at all; the generator returns an `is_ne_exact` test followed by a `jump` instead.

9.4. Implementing Specific Instructions in the Interpreter



Version Info

Erlang/OTP 21.0 rewrote `beam_makeops`. Until then a specific instruction was declared with a `%macro` line naming a C macro in `beam_emu.c` plus flags such as `-nonext` and `-pack`, and dispatch went through `PreFetch` and `NextPF` macros. Since OTP 21 the instruction bodies are written in `instrs.tab` and its siblings, `ops.tab` holds only rules and operand signatures, and the script generates the dispatch and the operand fetches itself. The `%macro` directive no longer exists.

Once the loader has chosen a specific instruction, the interpreter's back end in `emu_load.c` [S6] writes it to the code buffer. The first word of every instruction is the address of its C code, taken from the table that `process_main` fills in on its first run, and the operands follow, each translated according to the type letter in the specific instruction's signature: an `x` becomes a byte offset into the X register array, an `e` becomes a pointer to an export entry, an `f` becomes a label to patch once the target address is known, and a `c` becomes the tagged term itself. The result is threaded code, and `beam_emu.c` jumps from one instruction to the next with a computed `goto`.

9.4.1. Declaring and Implementing an Instruction

A specific instruction is declared in `emu/ops.tab` by its name followed by one type letter per operand. Several letters in one position are shorthand for one declaration per letter, so the line

```
move_return xcn
```

declares `move_return_x`, `move_return_c` and `move_return_n`. There is no `y` variant because a function that still has Y registers has to deallocate first, and a separate rule pairs `move | deallocate | return` into `move_deallocate_return`. The implementation lives in `emu/instrs.tab` [S7] and reads like C with untyped parameters:

```

move_return(Src) {
    /// -no_next
    x(0) = $Src;
    DTRACE_RETURN_FROM_PC(c_p, I);
    $RETURN();
    $DISPATCH_RETURN();
}

```

A definition whose name and arity match a family of specific instructions is that family's implementation. `$Src` expands to the code that fetches the operand for each variant, `$RETURN()` and `$DISPATCH_RETURN()` are macros from `emu/macros.tab`, and the comment line starting with `///` is a directive to the code generator. `-no_next` says that the body transfers control itself, so no dispatch to the following instruction should be generated. Running `beam_makeops` on the OTP 29.0 tables gives this for the X register variant in `beam_hot.h`:

```

OpCase(move_return_x):
{
    x(0) = xb(BeamExtraData(I[0]));
    DTRACE_RETURN_FROM_PC(c_p, I);
    SET_I(cp_val(*E));
    *E = NIL;
    if (FCALLS > 0 || FCALLS > neg_o_reds) {
        FCALLS--;
        Goto(*I);
    } else {
        c_p->current = NULL;
        c_p->arity = 1;
        goto context_switch3;
    }; ASSERT(!"Fell through 'move_return' (-no_next)");
}

```

`OpCase` expands to a label in `process_main`, `BeamExtraData(I[0])` picks the register offset out of the upper half of the instruction word, and `xb` turns a byte offset into an X register reference. An instruction that falls through to its successor gets the default dispatch instead, which fetches the address of the next instruction before executing the body, a prefetch that tends to help on superscalar CPUs. The generated code for `move_xy`, which packs both register offsets into the instruction word, shows the pattern:

```
OpCase(move_xy):
{
    Eterm tmp_packed1 = BeamExtraData(I[0]);
    BeamInstr next_pf = BeamCodeAddr(I[1]);
    yb((tmp_packed1>>BEAM_TIGHT_SHIFT)) = xb(tmp_packed1&BEAM_TIGHT_MASK);
    I += 1;
    ASSERT(VALID_INSTR(next_pf));
    GotoPF(next_pf);
}
```

The second directive, `-no_prefetch`, turns that early fetch off for instructions that may branch elsewhere, such as type tests; the `$FAIL()` macro in `macros.tab` carries it, so an `is_atom` body only has to call `$FAIL($Fail)`. The directives `%hot`, `%warm` and `%cold` in `ops.tab` decide whether an instruction's code goes into `beam_hot.h`, `beam_warm.h` (binary syntax) or `beam_cold.h` (tracing and rarely used instructions), which keeps the frequently executed bodies close together in memory.

9.4.2. Operand Packing

On a 64-bit machine built with the small code model, the address of an instruction's C code fits in 32 bits, and the loader uses the upper 32 bits of the instruction word for operands. Register offsets, small untagged integers (type `t`) and 32-bit integers (type `I`) are packable; tagged terms, export pointers and machine words (type `W`) are not. `beam_makeops` works out a packing scheme per specific instruction and stores it as a tiny program in the `opc` table, a string such as "22#" for `move_xy`, which the packing engine in `emu_load.c` runs after the operands have been translated. Two 16-bit fields fit beside the code pointer, so `BEAM_TIGHT_SHIFT` is 16 and `BEAM_TIGHT_MASK` is `0xFFFF`, and three register operands can share the upper half when they are stored as 10-bit register numbers instead of byte offsets. A `?` after an operand in the declaration, as in `is_eq_exact f? x xy`, tells the script that the operand is used only on some paths, so it can be placed in the instruction word while the operands used every time share a word of their own.

Packing halves the memory for a common instruction like `move_xy` from two words to one, which matters for cache behavior far more than for total memory, at the price of a shift and a mask when the operand is fetched.

9.4.3. Combined Instructions

The interpreter wants fast operand types like `x` and `y` for frequent instructions, and a family with several such variants would need several copies of its body. `beam_makeops` offers combined instructions to share the bulk of the code. The binary search variant of `select_val` is declared as

```
i_select_val_bins := select_val_bins.fetch.select;
```

with a head fragment declaring the shared variable, a `fetch` fragment that is duplicated once per source operand type, and a `select` fragment holding the search loop, of which there is one copy. The generated code has one `OpCase` label per variant, each fetching its operand and jumping to the shared body.

9.5. Handing Over to the JIT

In the `jit` flavor the loop in `load_code` is the same, the rules come from `jit/x86/ops.tab` or `jit/arm/ops.tab`, and `beam_load_emit_op` lives in `jit/asm_load.c` [S8]. It does not write a code buffer. It walks the operand signature to verify and normalize the operands, so that after the walk a small integer, an atom or `[]` is a tagged immediate of type `I`, a literal keeps its index, and a label is either a label number or 0 for "raise an exception on failure". It saves `int_func_start` for the function table and `line` for the line table, then calls the emitter:

```
/* Generate assembly code for the specific instruction. */
if (beamsm_emit(stp->ba, stp->specific_op, tmp_op) == 0) {
    BeamLoadError1(stp, "failed to emit asm for %d", stp->specific_op);
}
```

`stp->ba` is the module assembler created at the start of the load, one `BeamModuleAssembler` per module. `beamsm_emit` in `beam_jit_main.cpp` [S9] is the C to C++ boundary: it reinterprets the `BeamOpArg` array as a span of `ArgVal` (`ArgVal` is derived from `BeamOpArg`, and two static assertions check that the conversion is safe) and calls the assembler's `emit` method, which is a switch over the specific opcode generated by `beam_makeops` into `beamsm_emit.h`:

```

case op_i_move_sd: {
    emit_i_move(args[0], args[1]);
    break;
}

case op_i_select_val_lins_sfI: {
    emit_i_select_val_lins(args[0], args[1], args[2],
                           Span(args.begin() + 3, args.size() - 3));
    break;
}

```

Each case calls one emitter function, and an instruction declared with a variable-operand suffix receives those operands as a span. The emitters, one per family of specific instructions, are C++ functions in the `jit/x86/` and `jit/arm/` directories. Their files have an `instr_` prefix and a `.cpp` suffix. They drive the `asmjit` assembler to append native code to the module. The [JIT Compiler](#) walks through the move emitter, the register layout it relies on, and the code it produces; here it is enough to see that the loader's job ends with the call, and that the emitter is where the JIT's specialization happens.

That division of labor shows in the rule tables. The interpreter's `emu/ops.tab` has 42 rules and declarations that begin with a move instruction, and they feed `move_window` and `move_src_window` instructions that copy up to five registers in one dispatch. The JIT table has eleven, and the ones that remain exist for the sake of the native code rather than for dispatch:

```

move Src=c Dst => i_move Src Dst

move S1=d D1=d | move S2=d D2=d | consecutive_words(S1, D1, S2, D2) =>
    move_two_words S1 D1 S2 D2

move Src Dst => i_move Src Dst

i_move s d
move_two_words s d s d

```

Two moves between adjacent memory words become `move_two_words`, which the x86-64 emitter turns into one 16-byte vector load and store; everything else is `i_move`, and `mov_arg` inside `emit_i_move` picks the addressing mode. The `select_val` rules, on the other hand, are the same in both tables apart from the jump table threshold, because choosing a search strategy is loader work in both flavors.

When `int_code_end` arrives, `beam_load_finish_emit` in `asm_load.c` reserves space in the module for the line table and the debug table, embeds the string table, the attributes, the

compile info and the MD5 as read-only data, and calls `beamasm_codegen`. That step lays the code out in its final location. Because many operating systems enforce that a page is writable or executable but never both, the module is mapped twice, once executable and once writable, and every later modification, from literal patching to setting a breakpoint, goes through the writable view. The literals are then moved to their own literal area and patched into the code, exactly as the interpreter patches them into its code buffer, and the module is ready for the finish phase.

9.6. Code Publication Internals

The prepared module is a magic binary, and `erlang:finish_loading/1` turns it into current code. Running code reaches a module through a set of code access structures: the export table, the module table, the catch table and the address range table used to map a code address back to a function. Running code reads these structures without reader locks. An external call consults its export entry to find the target. The loader prepares updates in a separate generation while schedulers run. The runtime keeps three generations, selected by a global atomic variable, `the_active_code_index`, and `erts_finish_loading` proceeds as follows:

1. Seize the code load permission, suspending the process if another loader holds it.
2. Copy the active access structures into the staging generation, identified by `the_staging_code_index`.
3. Insert the new module into the staging structures, moving any previous version of the module to old code (the caller must have purged older code first).
4. Schedule a thread progress event and suspend the loader process until every scheduler has passed a full memory barrier.
5. Commit by writing the staging index to `the_active_code_index`.
6. Release the permission and resume the loader.

Readers never take a barrier, which means different cores may see the new code at slightly different times. That is safe because every form of process communication contains a barrier, so a process that receives a message caused by the new code is guaranteed to see the new code as well. A module with an `-on_load` function is inserted as a separate instance and `finish_loading` returns `on_load`, so that the caller can run the function before the code becomes current or purge it if the function fails. With three generations, the staging copy

only has to apply the changes since the last two loads instead of being rebuilt, and no scheduler can still be reading a generation by the time it is reused.

9.7. Exact Selection Rules and Assembly

The rules in both tables try the strategies in this order, and the first predicate that accepts the value list decides:

jump table

`use_jump_tab` accepts a list of integers that is dense enough, as described above. The generator produces `i_jump_on_val` (or `i_jump_on_val_zero` in the interpreter when the smallest value is 0), which subtracts the base, checks the range and indexes a table of labels. `jt/1` qualifies in both flavors; `g/1` qualifies in the interpreter, which asks for two values, and fails the JIT's minimum of six.

split by type

`mixed_types` fires when the list holds more than one operand type, and `split_values` rewrites the instruction into an `is_integer` test and two `select_val` instructions, one for the small integers and one for the rest, so that each half can use a better strategy.

literal chain

`floats_or_bignums` accepts a list of heap literals, and `select_literals` unrolls it into one `is_ne_exact` test per value followed by a `jump` to the failure label, since neither a table nor a sorted comparison works on values that are compared structurally.

sorted search

`fixed_size_values` accepts atoms, small integers, tuple arities and floats, and the `select_val` generator sorts the values with `beam_load_sort_select_vals` and picks by count. With two values the interpreter emits `i_select_val2`; with up to ten values both flavors emit `i_select_val_lins`, a linear scan, and the interpreter appends a sentinel so the scan needs no bounds check; above ten they emit `i_select_val_bins`, a binary search. The JIT's binary search emitter also switches to linear scans for any subtree of ten values or fewer, since measurements on random tables showed the comparisons to be cheaper than the branches at that size.

`select_tuple_arity` follows the same path with `i_select_tuple_arity`, after extracting the arity word from the tuple. Loading `sel` under `+JDDump true` shows both outcomes. `f/1` becomes a linear scan over the four atoms, each compared as its tagged word:

```
# i_select_val_lins_sfI
mov rsi, qword ptr [rbx]
cmp rsi, 220875
jz label_4
cmp rsi, 255755
jz label_6
cmp rsi, 300363
jz label_5
cmp rsi, 630347
jz label_3
jmp label_7
```

The four constants are the tagged atom indexes of a, b, c and d in the atom table of the node that loaded the module, so they differ from one node to another.

jt/1 becomes a jump table. The code checks the small integer tag, untags the value, subtracts the base of 10, bounds checks against the seven entries, and jumps through a table of labels stored next to the code:

```
# i_jump_on_val_sfWI
mov rdi, qword ptr [rbx]
mov eax, edi
and al, 15
cmp al, 15
jnz label_25
sar rdi, 4
sub rdi, 10
cmp rdi, 7
jnb label_25
lea rax, qword ptr [L37]
# L37: the table of seven label addresses, elided
jmp qword ptr [rax+rdi*8]
```

9.8. Literal-key Generator

A map lookup with a literal key, `{foo := V} = M`, would hash the atom `foo` on every execution if the instruction hashed at runtime. Both flavors let the loader do it once instead. A `get_map_elements` with a single key goes through the shared `get_map_element` generator, which looks the key up among the module's literals and, when it is one, produces `i_get_map_element_hash` with an extra `W` operand holding the value of `hashmap_make_hash` computed at load time. The instruction passes that hash straight to the map lookup, and a key that is a register falls back to the plain `i_get_map_element`, which hashes at runtime.

Sources and links

- [S1] Erlang/OTP Project. Erlang/OTP. OTP-29.0; erts/emulator/beam. <https://github.com/erlang/otp>
- [S2] Erlang/OTP Project. Erlang/OTP source code: beam_file.c. OTP-29.0; erts/emulator/beam/beam_file.c. <https://github.com/erlang/otp>
- [S3] Erlang/OTP Project. Erlang/OTP source code: ops.tab. OTP-29.0; erts/emulator/beam/emu/ops.tab. <https://github.com/erlang/otp>
- [S4] Erlang/OTP Project. Erlang/OTP source code: ops.tab. OTP-29.0; erts/emulator/beam/jit/x86/ops.tab. <https://github.com/erlang/otp>
- [S5] Erlang/OTP Project. beam_makeops. OTP-29.0; erts/emulator/internal_doc/beam_makeops.md. <https://github.com/erlang/otp>
- [S6] Erlang/OTP Project. Erlang/OTP source code: emu_load.c. OTP-29.0; erts/emulator/beam/emu/emu_load.c. <https://github.com/erlang/otp>
- [S7] Erlang/OTP Project. Erlang/OTP source code: instrs.tab. OTP-29.0; erts/emulator/beam/emu/instrs.tab. <https://github.com/erlang/otp>
- [S8] Erlang/OTP Project. Erlang/OTP source code: asm_load.c. OTP-29.0; erts/emulator/beam/jit/asm_load.c. <https://github.com/erlang/otp>
- [S9] Erlang/OTP Project. Erlang/OTP source code: beam_jit_main.cpp. OTP-29.0; erts/emulator/beam/jit/beam_jit_main.cpp. <https://github.com/erlang/otp>

JIT Implementation: Emitters and Native-Code History

Optional implementation detail accompanying the developer-facing chapter. The source extracts and version qualifications below describe the second-edition baseline.

10.1. Earlier execution engines

We will be talking about four execution engines in Erlang’s history in this chapter. `emu` is the interpreter, the engine every BEAM ran before OTP 24 and the one still built for platforms without a JIT (see [Operation and Maintenance](#)). `BeamAsm` is the JIT compiler that replaced it as the default. `JERICO` and `HiPE` came before it, as earlier answers to the same question. `JERICO` was a 1996 Master’s thesis project that compiled JAM bytecode to native SPARC code, and `HiPE` grew out of it into the native code compiler that shipped with OTP from R8 until OTP 24. Elixir compiles to the same BEAM instructions, so everything in this chapter applies to Elixir code as well.

10.2. Inspecting interpreter handlers

The handlers are labels inside `process_main`, and the build keeps its debug information, so `gdb` finds them by name: `info line process_main:lb_move_cx` gives the address, and `disassemble /s` from there shows the generated C beside the machine code. The whole `move_cx` handler is seven instructions. In this build `rbx` is `I`, the pointer into the threaded code, and `r12` is the base of the X registers, so the register roles differ from the JIT listings above:

```

; OpCase(move_cx): move a constant into an X register
endbr64
mov    rbp, QWORD PTR [rbx+0x18] ; I[3]: the next handler
mov    rdx, QWORD PTR [rbx+0x8]  ; I[1]: the constant (687 here)
add    rbx, 0x18                 ; I += 3
mov    rax, QWORD PTR [rbx-0x8]  ; I[2]: X register offset (24)
mov    QWORD PTR [r12+rax*1], rdx ; xb(I[2]) = I[1]
jmp    rbp                      ; GotoPF(next_pf)

```

The two loads from `I[1]` and `I[2]` fetch the constant and the register offset from the threaded code, the store is the move, and `jmp rbp` is the Goto. The JIT column did the same move as one `mov` with both the offset and the constant baked in. The addition and the tail call read the same way; `lb_i_plus_xxjd` and `lb_i_call_only_f` are their labels.

10.3. Stack safety and the history of native Erlang

Two consequences follow. The test that guards a heap allocation must also count the return address that the next `call` will push, or that call would overwrite a term just built, and the JIT keeps a small red zone of free words at the top of the stack so that it can call a shared fragment or a trace handler even when the current function has no frame. And the stack pointer can never be trusted across a call into C: the shared `garbage_collect` fragment reloads both the heap and the stack pointer from the process after the collector returns, because the collector may have moved the block.

10.3.1. JERICO: From JAM to SPARC

JERICO was written in C by Christer Jonsson and me, Erik Johansson as I was then, and finished in October 1996 [[JohanssonJonsson1996](#)]. It compiled functions individually, mixing native and emulated code within the same application, and fell back to the emulated version of a function when its register allocation failed. On nine sequential benchmarks, measured on a Sun SPARCsystem 600, the native code ran on average more than four times faster than the JAM emulator and 55 percent faster than Erlang compiled via C, in about 60 percent of the code size; on small programs it frequently reached a factor of ten over JAM [[Stenman2002](#)].

JERICO already had the shape of a just-in-time compiler. It was a runtime compiler integrated with the JAM emulator: each loaded function carried a header saying whether a native version existed and where it was, and calls crossed between emulated and native

code through four fixed entry and return points. JERICO also introduced backpatching for hot-code loading: when a function was replaced, all call sites pointing to it were patched in place to redirect to the new code.

JERICO had problems scaling to large programs (tens of thousands of lines of code), and adding new optimizations to the C codebase was slow.

10.3.2. HiPE: High Performance Erlang

HiPE (High Performance Erlang) rewrote JERICO from scratch in Erlang rather than C, which made it easier to experiment with new optimizations. Where JERICO used a single intermediate format, HiPE introduced a layered architecture with multiple intermediate representations: BEAM or JAM bytecode was first translated to a functional intermediate code called Icode, then lowered to RTL (register transfer language), and finally to native SPARC or x86 machine code. The project was developed at Uppsala University and described in the dissertation *Efficient Implementation of Concurrent Programming Languages* [Stenman2002]. It also evaluates linear-scan register allocation, whose algorithm is described by Poletto and Sarkar [PolettoSarkar1999].

Like JERICO, HiPE could compile individual functions at runtime. The compiler ran in a separate Erlang process, so compilation happened concurrently with application execution. Profiling tools measured call counts to identify hot spots, and those functions could then be compiled to native code on demand. HiPE could also produce pre-compiled native code stored in `.beam` files, sometimes called fat binaries.

The interface between native and emulated code was the second stack of [One Process, Two Stacks](#), with mode switching at every call and return that crossed between the two. The garbage collector had to scan both stacks, exception handling had to bridge the two modes, and every new BEAM runtime feature had to be made compatible with native code execution. The HiPE compiler report explains this interface and the native compilation pipeline [HiPE2003].

Measured against BEAM in OTP R8, HiPE 1.0 ran sequential benchmarks between two and six times faster on both x86 and SPARC, while process-bound programs such as a message ring gained almost nothing [Stenman2002]. Large industrial applications saw little benefit for the same reason. My own measurements on an UltraSPARC in 1999 put the difference in numbers: on the `length` micro benchmark HiPE ran 16 times faster than JAM where BEAM ran 4.5 times faster, but on SCCT, a 50,000 line part of the AXD 301 switch, the factors were 1.6 and 1.4 [Johansson1999]. The industrial code spent its time in message passing, built-in

functions and garbage collection rather than in sequential computation, and that is where later work went.

10.4. Runtime stack-switch helpers

Before calling into the runtime, the generated code stores the stack and heap pointers into the process control block, points `rsp` at the scheduler thread's C stack, and makes the call there; on return it reloads `rsp` from the process. The helpers that emit this sequence are `emit_enter_runtime` and `emit_leave_runtime`, and the C stack address is computed from the register base pointer, since the scheduler's register block lives on that stack.

10.5. Interpreter and JIT register conventions

The following table summarizes the differences, aspect by aspect.

Table 4. Interpreter dispatch vs. BeamAsm native execution

Aspect	Interpreter (direct threaded code)	BeamAsm (native code)
Instruction dispatch	Each instruction ends with an indirect branch (<code>goto *I</code>) to the address of the next handler.	No dispatch. Instructions compile to a contiguous stream of native code with fall-through or direct jumps to known targets.
Type checking	The loader selects handlers, including specialized and combined instructions. Each selected handler performs the checks in its C implementation; redundant work depends on that selected sequence.	The JIT reads the Type chunk from the <code>.beam</code> file and eliminates checks made redundant by the supplied type and range constraints. Knowing that a value is an integer still leaves the small-integer and bignum cases to distinguish.
Argument access	BEAM x-registers are elements in a C array (<code>ErtsSchedulerRegisters</code>). Each instruction loads its operands from memory via the array base pointer, unpacks argument encoding, and stores the result back.	The <code>ErtsSchedulerRegisters</code> struct pointer is pinned in a callee-save CPU register (<code>rbx</code> on x86-64). Operands are still loaded from this struct, but the base address is always in a register, and the JIT can specialize the load for each argument type, avoiding the generic argument-unpacking path.

Aspect	Interpreter (direct threaded code)	BeamAsm (native code)
Process state access	The process pointer (<code>C_p</code>), heap top and reduction counter are represented by C variables and struct fields. The C compiler decides which values it can keep in CPU registers.	Dedicated CPU registers hold frequently used state (<code>r13</code> for the process pointer, <code>r15</code> for the heap pointer, <code>r14</code> for the reduction counter on x86-64). Reading a field through the process pointer still requires a memory access.
Function calls	A call instruction stores the return address in a stack slot, sets the instruction pointer <code>I</code> to the callee's first instruction, and re-enters the emulator dispatch loop.	A call emits a native <code>call</code> instruction (a branch-and-link on AArch64), and the return address lands on the Erlang stack as One Process, Two Stacks describes.
Code size and I-cache	The implementation of each instruction handler exists once in the emulator binary. Loaded code contains only instruction addresses and arguments, so the code footprint is small.	Each loaded function gets its own block of native instructions. Total code size is roughly 10% larger than the interpreter representation. Common sequences (garbage collection, context switching, error handling) are shared as global code fragments to limit the increase.

10.6. Emitters, global fragments and moves

10.6.1. BeamAsm Architecture

Every BEAM instruction has an emitter, a C++ function that writes the native code for that instruction. The emitter for `move` has two jobs: work out where the source and the destination live, an offset from the register base for an X register or a stack slot for a Y register, and write the load and store that carry the word between them. Most emitters are that simple. A few look at the instruction that follows and fold work the two would otherwise repeat.

The code generator uses the `asmjit` library on both x86-64 and AArch64. Both backends emit into a buffer and let `asmjit` resolve the relocations once, when the finished code is copied into executable memory; after that the code holds absolute addresses and stays where it was committed.

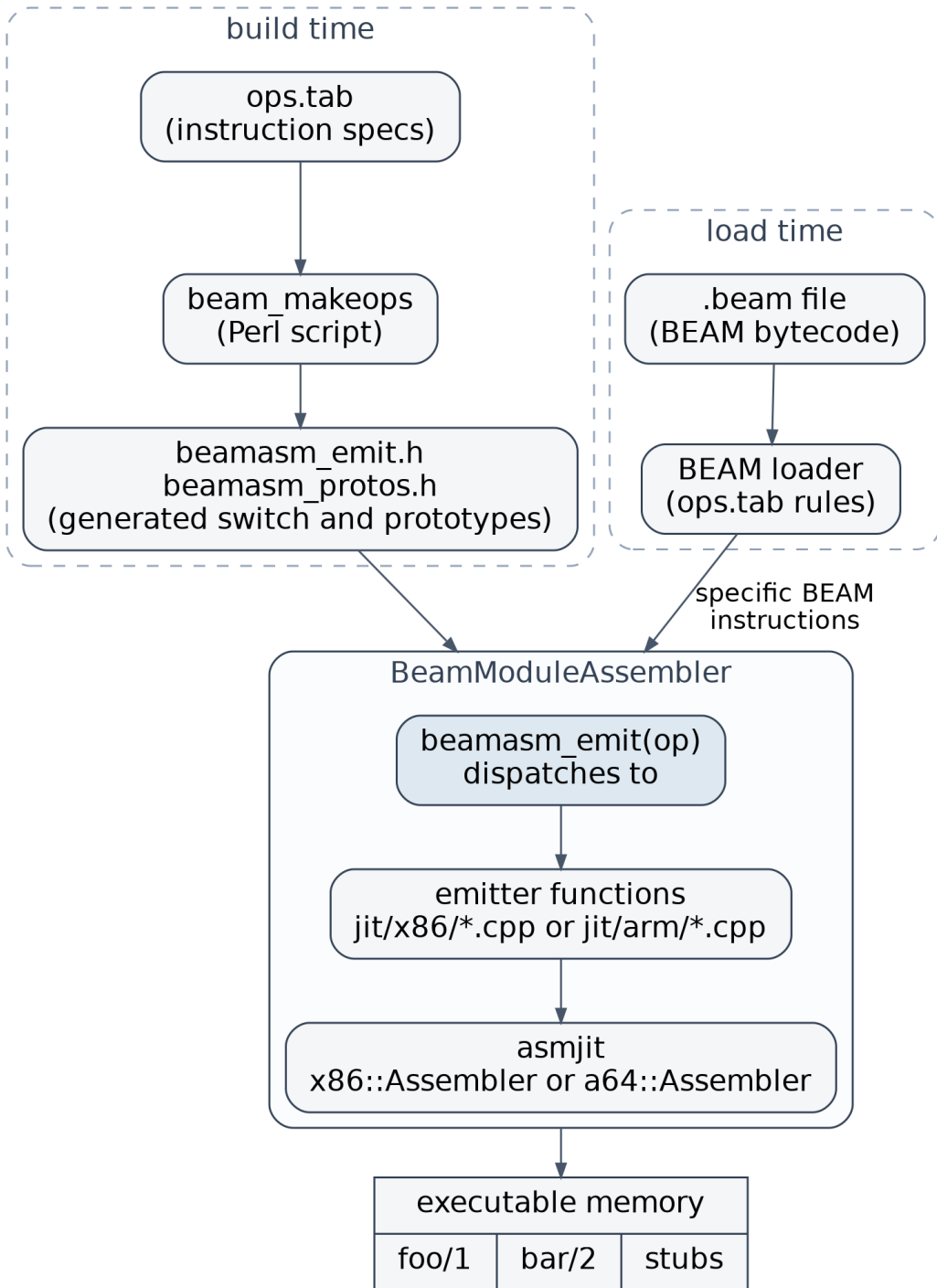


Figure 10. BeamAsm code generation architecture

The left side of [Figure 10](#) shows what happens at build time, when OTP itself is compiled. The `beam_makeops` Perl script reads `ops.tab` and generates C++ header files that define a switch statement dispatching each BEAM opcode to its emitter function. The right side shows what happens at module load time: the BEAM loader parses the `.beam` file and applies transformation rules, then the `BeamModuleAssembler` walks the resulting instruction sequence, calling the appropriate emitter for each one. The emitters use `asmjit` to produce native machine instructions, which are written into a block of executable memory. The emitters live under `erts/emulator/beam/jit/`, the x86-64 backend in `x86/` and the AArch64 backend in `arm/`. The generated headers `beamsm_emit.h` and `beamsm_protos.h` (see [Generic BEAM Instructions](#)) connect each specific instruction to its emitter; the body of each emitter is hand-written in the backend source.

Translation Units and Code Stubs

When `BeamAsm` loads a module, it generates native code for every function in that module. The generated code for a function is a contiguous block of native instructions, called a translation unit.

Not all operations are inlined into translation units. Certain common sequences, like garbage collection entry points, stack growing, and exception handling, are generated once as code stubs (also called global code fragments). Translation units call into these stubs when needed, keeping the per-function code compact and reducing pressure on the instruction cache.

The full list of global fragments is generated at build time from a table in the `BeamAsm` source (see `beam_asm_global.hpp.pl` in the OTP tree). The following are among the most important.

`process_main`

The outer scheduler loop. It picks the next process to run, restores its registers from the process control block, and jumps to the saved instruction pointer. When a process yields or exits, control returns here.

`garbage_collect`

The shared garbage collection entry point. When an allocation check in a translation unit fails, the code jumps here with the number of words needed and the number of live registers. This fragment calls `erts_garbage_collect_nobump` and, on return, reloads the heap pointer and stack pointer from the process control block.

export_trampoline

Dispatches remote calls through export entries. It inspects the trampoline field of the `Export` struct and branches to the correct handler: a BIF call, the error handler for undefined functions, or a generic breakpoint handler for tracing.

dispatch_return

Handles context switching on function return. When reduction credits have been exhausted at a return instruction, the code jumps here to save the process state and re-enter the scheduler.

i_test_yield_shared

Handles context switching at function entry. Each function prologue decrements the reduction counter and, if it reaches zero, branches to this fragment, which saves the current MFA and arity and yields to the scheduler.

call_nif_shared

Enters a NIF. It switches from the Erlang stack to the C runtime stack, calls the NIF function pointer, and on return checks whether the NIF yielded, raised an exception, or produced a normal result.

call_bif_shared

Calls a BIF through the standard entry path. It saves state, switches stacks, invokes the BIF function, and handles the result, including the case where the BIF traps to Erlang code.

raise_exception / raise_exception_shared

Builds and raises an exception. Module code that detects an error condition jumps here with the error term in a register. The fragment calls the runtime's `handle_error` function, which constructs the stack trace, and then either jumps to a catch handler or re-enters the scheduler if the process should exit.

generic_bp_global / generic_bp_local

Handle breakpoints for tracing and debugging. When a breakpoint is active on a function, the prologue trampoline calls one of these fragments, which inspects the breakpoint flags and dispatches to the trace handler, NIF loader, or both.

i_loop_rec_shared

Implements the inner loop of a `receive` expression. It walks the process message queue, decodes distribution messages on demand, and returns the next candidate message to the generated code for pattern matching.

unloaded_fun

Handles calls to funs whose defining module has been purged. When a fun's dispatch entry points here, the fragment calls `beam_jit_handle_unloaded_fun`. If the module has been loaded again but no longer defines the fun, that raises `badfun`. If the module is not loaded, the call is redirected to `error_handler:undefined_lambda/3`, which tries to load it; only when no error handler exists does the process get an `undef` error.

Example: The `move` Instruction

The `move` instruction copies a value from a source to a destination register. In generic BEAM, it looks like this:

```
{move, {x,0}, {y,1}}
```

The source can also be a literal, an atom, or an integer constant. After the loader applies the transformation rules in `ops.tab`, the generic `move` is mapped to one of several specific forms. The most common is the two-argument form `move Src Dst`, where `Src` is a constant, X register, or Y register, and `Dst` is an X or Y register. The `beam_makeops` script generates a dispatch entry that routes this specific instruction to the emitter function `emit_i_move` in the x86-64 backend.

The emitter function

The emitter for `move` is defined in `erts/emulator/beam/jit/x86/instr_common.cpp`. It is short:

```
void BeamModuleAssembler::emit_i_move(const ArgSource &Src,
                                       const ArgRegister &Dst) {
    x86::Gp spill = alloc_temp_reg();
    mov_arg(Dst, Src, spill);
}
```

The function allocates a temporary CPU register from the register cache and delegates to `mov_arg`, a family of overloaded helper functions that handle the actual code emission. The temporary register is needed because x86-64 cannot move directly from one memory location to another.

How `mov_arg` resolves the move

The `mov_arg` helper in `jit/x86/beam_asm.hpp` dispatches based on the argument types. The two snippets below are shortened OTP 29 extracts, with explanatory comments added for this book. When the destination is a BEAM register and the source is also a BEAM register, the logic proceeds in two steps.

First, it loads the source. The helper checks the register cache to see if the source value already happens to be in a CPU register from a recent operation. If so, it reuses that register and skips the load. Otherwise, it emits a `mov` instruction that loads from the source register's memory location into the spill register:

```
// Load source: mov_arg(x86::Gp to, const ArgVal &from, ...)
auto mem = getArgRef(from);    // Memory address of the source register
load_cached(to, mem);          // Load into CPU register (or reuse cache)
```

Then, it stores to the destination. The helper emits a `mov` instruction from the CPU register to the destination register's memory location:

```
// Store to destination: mov_arg(const ArgRegister &to, x86::Gp from, ...)
auto mem = getArgRef(to);      // Memory address of the destination register
store_cache(from, mem);        // Store and record in register cache
```

The `store_cache` function both emits the store instruction and records that the destination register's value is now cached in the CPU register. If a subsequent instruction reads this destination register, it can skip the memory load entirely.

BEAM register layout in memory

To understand the generated instructions, we need to know where BEAM registers live in memory. The x86-64 backend dedicates certain CPU registers to runtime state that persists across instructions:

CPU register	Name	Purpose
<code>rbx</code>	<code>registers</code>	Base pointer to the scheduler's register array (<code>ErtsSchedulerRegisters</code>)
<code>rsp</code>	<code>E</code>	Erlang stack pointer (top of Y register area)
<code>r13</code>	<code>C_p</code>	Pointer to the current process control block
<code>r15</code>	<code>HTOP</code>	Heap top pointer (next free heap word)

X registers are stored in a contiguous array within the `ErtsSchedulerRegisters` structure, addressed as offsets from `rbx`. The `getXRef` function computes the memory operand through a shared helper (assertions omitted):

```
constexpr x86::Mem getSchedULERRegRef(int offset,
                                       size_t size = sizeof(UWord)) const {
    const int x_reg_offset =
        offsetof(ErtsSchedulerRegisters, x_reg_array.d);
    return x86::Mem(registers, offset - x_reg_offset, size);
}

constexpr x86::Mem getXRef(int index, size_t size = sizeof(UWord)) const {
    int base = offsetof(ErtsSchedulerRegisters, x_reg_array.d);
    int offset = index * sizeof(Eterm);
    return getSchedULERRegRef(base + offset, size);
}
```

The registers base pointer (`rbx`) points at `x_reg_array` itself rather than at the start of the struct, so the two `offsetof` terms cancel and X register 0 is at displacement zero from `rbx`. Successive X registers follow at 8-byte intervals, and the auxiliary fields that precede the array are reached with small negative displacements. Y registers, by contrast, live on the Erlang stack and are addressed as offsets from `E` (`rsp`).

Generated native code

With this layout we can read what the JIT produces for three move variants ([Examining JIT Output with +JDdump](#) shows how to capture the output for a module of your own).

Register to register: `{move, {x,0}, {x,1}}`

This moves X register 0 to X register 1. Both are memory locations relative to `rbx`, so the move goes through a temporary CPU register. On Linux, `alloc_temp_reg` hands out `r10` first (TMP1 in the x86-64 backend). Assuming no cache hits:

```
mov    r10, qword [rbx]      ; load X(0) into temp register
mov    qword [rbx+0x8], r10  ; store to X(1)
```

Two instructions, two memory accesses. If the register cache already holds the value of `X(0)` in a CPU register from a previous instruction, the first `mov` is omitted.

X register to Y register: {move, {x,0}, {y,1}}

Y register 1 lives on the Erlang stack at an offset from `rsp`:

```
mov    r10, qword [rbx]      ; load X(0)
mov    qword [rsp+0x8], r10  ; store to Y(1) on the stack
```

The structure is the same, but the destination uses `rsp` as its base rather than `rbx`.

Immediate to register: {move, {integer,42}, {x,0}}

Small integers are encoded as immediate values in the tagged pointer format. The integer 42 has the tagged representation `0x2AF` (42 shifted left by four bits, with the small integer tag `0xF` in the low nibble). Because this value fits in 32 bits, the JIT can encode it directly in the instruction:

```
mov    qword [rbx], 0x2AF    ; store tagged integer 42 to X(0)
```

A single instruction with no intermediate register. If the immediate value were larger than 32 bits (which can happen with tagged pointers to literals), the JIT would load the immediate into a register first and then store it.

The register cache

The register cache is the JIT's bookkeeping, during code generation, of which BEAM register values are currently held in CPU registers. It is local to a single basic block: at labels and branches the cache is invalidated, because the JIT does not track register state across control flow edges. Within straight-line code, a function that uses `X(0)` in several consecutive instructions loads it from memory once and reuses the CPU register for the rest.

10.7. Type-chunk encoding

10.7.1. What the Type Chunk Contains

The Type chunk is identified by the four-byte tag "Type". At OTP 29 it starts with a version and entry count. Each entry has 16 bits of type flags followed by optional range or bitstring-unit information. Consequently entries have variable lengths. `beam_types:encode_ext/1` and `decode_ext/1` define the encoding in the compiler source.

A typed operand carries a register and a type-table index. Several operands can refer to the same type entry. The table itself does not identify a register or an instruction position, and a type known to the compiler need not survive serialization with all its details.

BeamAsm consumes the constraints while loading the module; the emitted arithmetic does not consult the Type chunk at runtime.

10.8. Record-update loader rules

The loader's transformation rules in `ops.tab` dispatch on the hint. When the hint is `inplace`, the instruction is rewritten to `update_record_in_place`, which has its own emitter:

```
update_record a==am_inplace Size Src=d Dst N Updates=* =>
  update_record_in_place Size Src Dst N Updates
update_record a I s d I *
update_record_in_place I s d I *
```

The `Src=d` qualifier requires the source to be an X or Y register and `Updates=*` marks the variable-length list of index and value pairs. All other hint values (`copy` and `reuse`) fall through to the general `update_record` emitter.

10.9. Copy-path assembly

On x86-64, the emitter (`emit_update_record`) produces code equivalent to the following pseudocode. In these three schematics, `NewValue` is the already-computed balance; the addition is omitted:

```
; update_record copy: size=4, element 3 (balance)
mov RET, Src ; load tagged source tuple pointer
sub RET, 2 ; untag before reading fields
; Copy words 0..2 (header, account, Id) to HTOP
mov ARG1, [RET + 0]
mov [HTOP + 0], ARG1
mov ARG1, [RET + 8]
mov [HTOP + 8], ARG1
mov ARG1, [RET + 16]
mov [HTOP + 16], ARG1
; Write the new balance in word 3
mov [HTOP + 24], NewValue
; Copy word 4 (Currency)
mov ARG1, [RET + 32]
mov [HTOP + 32], ARG1
; Tag the new tuple and advance heap top
lea RET, [HTOP + 2] ; TAG_PRIMARY_BOXED = 2
add HTOP, 40 ; (4 elements + header) * 8 bytes
mov Dst, RET
```

10.10. Reuse-path assembly

When the hint is `reuse` and only a single field is updated, the emitter inserts a comparison before the copy. If the old value matches the new one, execution jumps past the copy and returns the original tuple unchanged:

```

; update_record reuse: size=4, element 3 (balance)
mov  RET, Src
mov  ARG1, RET
; Compare the old balance to the new value
cmp  [ARG1 + 24 - 2], NewValue ; subtract the boxed-pointer tag
je   done                      ; skip copy if unchanged
; ... full copy sequence as above ...
done:
mov  Dst, RET

```

This optimization pays for itself when the same value is written repeatedly, which is common in state machines where a status field is set to active on every iteration.

10.11. In-place safety-check assembly

The check on x86-64 works as follows:

```

; update_record_in_place: size=4, element 3 (balance)
mov  RET, Src                      ; load tagged tuple pointer

; Load the process high water mark
mov  ARG1, [c_p + offsetof(Process, high_water)]

; Is the tuple above HTOP? (i.e., not on the current heap at all)
cmp  RET, HTOP
jae  copy_fallback                ; above heap top: not safe

; Is the tuple at or above high_water? (in the safe region)
cmp  RET, ARG1
jae  do_update                    ; in safe region: modify in place

copy_fallback:
; Copy the entire tuple to fresh heap space
; (same sequence as the copy path)
...copy 5 words from (RET - 2) to HTOP...
lea  RET, [HTOP + 2]
add  HTOP, 40                     ; 4 elements plus header

do_update:
; Write only the changed field

```

```
mov [RET + 24 - 2], NewValue ; balance, correcting the pointer tag
mov Dst, RET
```

10.12. High-water source comment

The garbage collector maintains `high_water` within the current heap when it moves or replaces heap storage. The source explains why:

Keep the high water mark pointing into the current heap to ensure that the test for the safe range in the `update_record_in_place` (JIT) stays honest.

10.13. Destructive-update compiler rewriting

The destructive update pass (`beam_ssa_destructive_update`) then scans for `update_record` instructions where the source variable dies at the update point and the alias analysis has marked it as unique. When both conditions hold, the pass rewrites the hint from `copy` or `reuse` to `inplace`. If the source of an `inplace` update was itself produced by an `update_record` with a `reuse` hint, the pass forces the earlier instruction to use `copy` instead, because a `reuse` hint could return the original tuple without copying, leaving two live references to the same tuple.

The pass also handles literal tuples. A literal cannot be modified in place (it lives in read-only memory), so when the source of a potential in-place update traces back to a literal, the pass inserts a `put_tuple` instruction to create a mutable copy on the heap before the update chain begins.

10.14. Fun layouts and header encoding

In OTP 24 and earlier the `ErlFunThing` struct carried six words of fixed overhead before the environment:

Table 5. Old fun term layout (OTP 24 and earlier, 64-bit)

Word	Field	Contents
0	thing_word	HEADER_FUN, a fixed constant
1	fe	pointer to the ErlFunEntry
2	next	off-heap chain link
3	arity	Uint, a full machine word
4	num_free	Uint, a full machine word
5	creator	pid of the creating process
6	env[0]	first free variable
7	env[1]	second free variable
...		the remaining free variables

Several of these fields existed because of historical design decisions. The `creator` pid recorded the process that created the fun; external funs (fun M:F/A references) were not funs at all in this layout but separate export terms with their own `EXPORT_SUBTAG` header. The `next` pointer threaded the fun into the process's off-heap list so the garbage collector could decrement the reference count on the shared `ErlFunEntry` when the fun was reclaimed. The `arity` and `num_free` fields each consumed a full machine word, even though their values fit in a single byte.

OTP 24 introduced the `make_fun3` instruction, which passes the environment explicitly rather than requiring the compiler to push free variables onto the stack. OTP 25 unified local and external funs into one struct, marking an external fun by storing the atom `external` in the `creator` field. OTP 27 packed the `arity` and the environment size into the header word and dropped the `next`, `creator`, `arity`, and `num_free` fields, but it had no separate local/external flag yet: a fun was external when its environment size was zero, and every local fun carried one extra environment slot pointing at a `FunRef` term that kept the `ErlFunEntry` alive. OTP 28 removed that slot and added the `kind` bit to the header. The result is the compact layout used today:

Table 6. Compact fun term layout (OTP 28+, 64-bit)

Word	Field	Contents
0	thing_word	FUN_SUBTAG plus arity, env_size and the kind bit
1	entry	pointer to the ErlFunEntry or Export
2	env[0]	first free variable
3	env[1]	second free variable
...		the remaining free variables

The header word packs three fields into the bits above the standard tag and subtag:



Figure 11. Fun header word bit layout (64-bit)

The kind bit at position 24 distinguishes external funs (1) from local funs (0). The env_size field holds the number of captured free variables. The arity field holds the apparent arity of the fun. All three are extracted with simple shifts and masks, which the JIT can perform in a single instruction.

10.15. Creating and calling fun objects

10.15.1. The make_fun3 Instruction

The make_fun3 instruction creates a local fun (closure) on the process heap. It replaces the older make_fun2 instruction, which required the compiler to push free variables onto the X register file in a separate step before creating the fun. With make_fun3, the environment values are passed directly as operands to the instruction, giving the JIT more freedom to optimize the allocation sequence.

For make_fun3, BeamAsm writes the header, the entry pointer and the environment straight to the heap top. The MAKE_FUN_HEADER macro combines the arity, environment size, and local/external flag into a single immediate value that one store writes:

```

// Simplified view of what the JIT emits for make_fun3
// (from instr_fun.cpp, x86-64)

// Write the packed header
mov [HTOP + offsetof(ErlFunThing, thing_word)],
    MAKE_FUN_HEADER(Arity, NumFree, 0)

// Write the pointer to the ErlFunEntry
mov [HTOP + offsetof(ErlFunThing, entry.fun)], FunEntryPtr

// Copy each free variable into the environment
mov [HTOP + offsetof(ErlFunThing, env[0])], FreeVar0
mov [HTOP + offsetof(ErlFunThing, env[1])], FreeVar1
...

// Create the boxed pointer and bump the heap top
lea Result, [HTOP + TAG_PRIMARY_BOXED]
add HTOP, (ERL_FUN_SIZE + NumFree) * 8

```

When consecutive environment values happen to reside in adjacent registers, the JIT can copy two values at once using a 128-bit vector move (`vmovups` on CPUs with AVX, the SSE `movups` otherwise).

BeamAsm also generates specialized code for calling funs. When the callee’s arity is statically known, the JIT can skip the dispatch through a generic apply path and compare only the lower 16 bits of the header word against the expected arity and subtag. This turns the arity check into a single compare-and-branch.

10.16. Captured binary-match assembly

BeamAsm compiles the entire `bs_match` instruction into a contiguous block of native code. The emitter function `emit_i_bs_match` in `erts/emulator/beam/jit/x86/instr_bs.cpp` walks the list of match commands and generates native instructions for each one. Two things happen before the commands are emitted. The loader folds the `test_heap` that follows into the match instruction, so the specific instruction is `i_bs_match_test_heap`, and the emitter merges the two fixed-size integer commands into a single read of 40 bits. The listing below is the OTP 29.0 +JDDump output for that instruction in `parse_header/1` (Intel syntax; `rsi` holds the match context, `rdi` the byte address of the data and `rdx` the bits read; the numeric call target and the label numbers depend on the build):

```

# i_bs_match_test_heap_fSIIt
# ensure_at_least 40 8

```

```

# skipped fetching of BEAM register
mov rax, qword ptr [rsi+22]      ; end bit offset
mov rcx, qword ptr [rsi+14]     ; start bit offset
sub rax, rcx                    ; bits remaining
cmp rax, 40                     ; at least the header?
jl label_3                      ; bail out if not
test al, 7                      ; whole bytes left for Rest?
jnz label_3

# test_heap 14
lea rdx, qword ptr [r15+144]
cmp rdx, rsp
short jbe L14
mov ecx, 2

.db 0x90
call 130029666175584           ; garbage_collect fragment
L14:

# read 40
mov rsi, qword ptr [rbx+8]      ; reload the match context
mov rcx, qword ptr [rsi+14]     ; start bit offset
mov rdi, qword ptr [rsi+6]      ; base address with flag bits
and rdi, -4                     ; clear the flag bits
add qword ptr [rsi+14], 40      ; advance the position
mov rax, rcx
shr rax, 3                      ; byte offset
add rdi, rax
and ecx, 7                      ; bit offset within the byte
jnz L16                         ; unaligned: take the slow path
movbe rdx, qword ptr [rdi-3]    ; load 8 bytes, byte-swapped
add ecx, 24
short jmp L18

L16:
mov rdx, qword ptr [rdi-2]
shr rdx, 16

L17:
bswap rdx

L18:
shl rdx, cl                     ; data now left-aligned in rdx

L15:
# extract integer 8
mov rax, rdx
# store extracted integer as a small
shr rax, 52                     ; 64 - 8 - 4
or rax, 15                     ; tag as small integer

L19:
L20:
mov qword ptr [rbx], rax        ; Tag -> x0
shl rdx, 8                     ; drop the consumed bits

# extract integer 32
mov rax, rdx

```

```

# store extracted integer as a small
    shr rax, 28                ; 64 - 32 - 4
    or rax, 15
L21:
L22:
    mov qword ptr [rbx+16], rax    ; Len -> x2
    shl rdx, 32
# get_tail
    mov rdi, rsi
    call L23                      ; stub for bs_get_tail_shared
    mov qword ptr [rbx+8], rax    ; Rest -> x1

```

10.17. Binary extraction shortcuts

10.17.1. Little-Endian Segments

On x86-64, a byte-aligned segment declared with little-endian byte order (<<X:32/little>>) can use a plain mov instruction. No byte swap is needed:

```

;; 32-bit little-endian integer, byte-aligned
mov    eax, dword [bin_base + tmp] ; native load, no swap needed

```

The surrounding size checks and Erlang tagging still matter; the single mov shown here is only the extraction step.

10.17.2. The Fast-Path Shortcut for Small Integers

When an unsigned big-endian segment fits in a small (fewer than 60 bits on a 64-bit system) and the next match command is going to overwrite the register holding the read bits anyway, the JIT applies a small additional optimization. Rather than going through the general emit_extract_integer code path, it shifts and tags in place:

```

;; unsigned, big-endian, fits in small, read bits not needed afterwards
shr    bitdata, (64 - bits - 4)    ; shift value into position
or     bitdata, 0xF                ; tag as small integer
mov    [dst], bitdata              ; store result

```

The general path produces the same shr and or for such a segment, as the parse_header/1 listing shows; what the shortcut saves is the copy into rax and the trailing shl that keeps the remaining bits aligned. Sign extension, the small-versus-bignum test, and a possible bignum allocation only appear for signed segments or for segments of 60 bits or more.

Scheduler Implementation: Counters, Timers and Migration

Optional implementation detail accompanying the developer-facing chapter. The source extracts and version qualifications below describe the second-edition baseline.

11.1. Reduction-counter fields

While a process is running the emulator keeps the number of reductions left to execute in the (register mapped) variable `FCALLS` (see `emu/beam_emu.c`; the JIT keeps the same counter in a machine register, see [The JIT Compiler](#)).

The `reds` field keeps track of the total number of reductions a process has done up until it was last suspended. By monitoring this number you can see which processes do the most work.

You can see the total number of reductions for a process (the `reds` field) by calling `erlang:process_info/2` with the atom `reductions` as the second argument. You can also see this number in the process tab in the observer or with the `i/0` command in the Erlang shell.

As noted earlier, each time a process starts the field `fcalls` is set to the value of `CONTEXT_REDS` and for each function call the process executes `fcalls` is reduced by 1. When the process is suspended the field `reds` is increased by the number of executed reductions. In some C like code something like: `p->reds += (CONTEXT_REDS - p->fcalls)`.

Normally a process would do all its allotted reductions and `fcalls` would be 0 at this point, but if the process suspends in a receive waiting for a message it will have some reductions left.

11.2. Atomic process-state flags

The state of a process lives in the PCB field `state`, a 32-bit word of flags updated with atomic operations. The flags are defined as `ERTS_PSFLG_*` in `erl_process.h`[\[S1\]](#). Each flag records one

fact about the process, and several are usually set at once: `ERTS_PSFLG_ACTIVE` says the process has work to do, `ERTS_PSFLG_IN_RUNQ` that it sits in a run queue, `ERTS_PSFLG_RUNNING` that a scheduler is executing it, and `ERTS_PSFLG_GC`, `ERTS_PSFLG_SUSPENDED`, `ERTS_PSFLG_EXITING` and `ERTS_PSFLG_FREE` mark garbage collection, suspension, exit and the final clean up. Further flags track work on the dirty schedulers (`ERTS_PSFLG_DIRTY_RUNNING`) and pending system tasks. Because the word is changed with compare and swap, a scheduler can move a process between states without taking the process lock, which is what keeps sends and wake ups cheap on a busy system.



Version Info

Until Erlang/OTP R16B the state was a plain status field, and two more fields, `gcstatus` and `rstatus`, saved the previous state across a garbage collection or a suspend. The atomic flag word replaced all three in R16B, as part of the work to reduce lock contention on process state changes.

11.3. Timing wheels and timer entries

Timers are handled in the VM by a timing wheel: an array of time slots which wraps around, where each slot holds the timers due at that point in time. Time is counted in clock ticks of one millisecond of Erlang monotonic time, and a timer is stored with the absolute tick at which it should fire.



Version Info

Up to Erlang/OTP 17 there was one global timing wheel of 65536 slots, and the write lock protecting it could become a point of contention with many timers. Erlang/OTP 18.0 gave each scheduler its own wheel. Erlang/OTP 20.0 reworked the wheels to cut their memory use and the work needed to maintain them; the layout described here is the one at OTP 29.

Each scheduler thread has its own wheel, and the wheel is really two wheels. The soon wheel has 16384 slots (`ERTS_TW_SOON_WHEEL_SIZE`, 2^{14}) of one tick each, so it covers the next sixteen seconds or so. The later wheel also has 16384 slots, but each slot is 8192 ticks wide (about eight seconds), so one revolution of it covers about 37 hours. A timer due within the range of the soon wheel goes straight into the soon slot for its tick. A timer further away goes into the later slot for its time, together with every other timer due in the same eight second window. Timers even further away than one revolution of the later wheel share a later slot by their time modulo the wheel size. Such timers are rare: the high level timer service that implements `receive ... after` and the timer BIFs (`erl_hl_timer.c`) keeps timers further

away than one revolution in a tree of its own, with a single wheel timer set to the nearest of them. A system built for a small memory footprint (`SMALL_MEMORY`) uses 4096 slots per wheel instead.

The current time of a wheel is its `pos` field, an index that only moves forward. A scheduler advances it in `erts_bump_timers`, which triggers every timer in the slots between the old and the new position. A timer set for a time that has already passed skips the wheel and goes on an "at once" list that the next bump empties. When the position comes within one later slot of the later wheel's own position, the scheduler inspects that later slot and moves the timers that are now within reach into the soon wheel. Under normal load this happens eight to sixteen seconds before they are due. Every slot is a circular doubly linked list, so setting and cancelling a timer are both $O(1)$ operations.

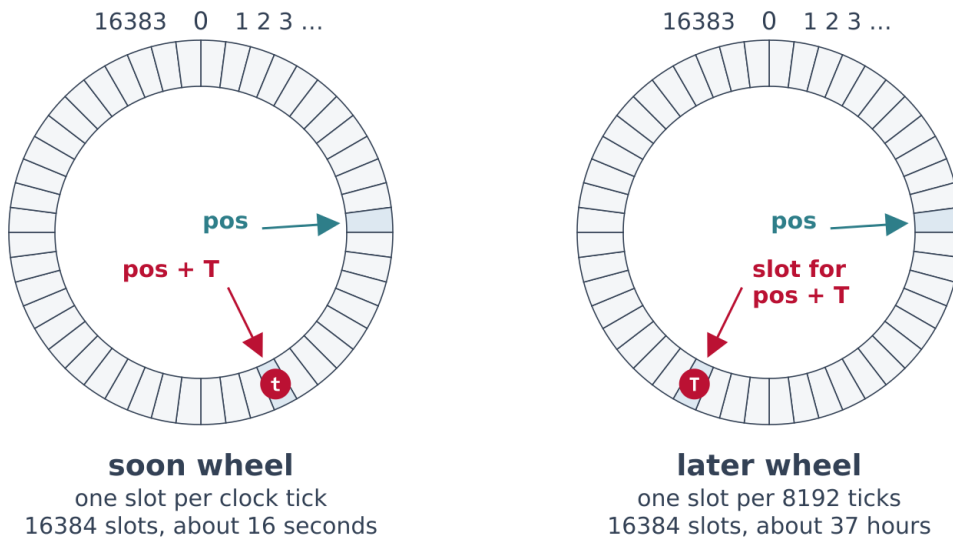


Figure 12. The two timing wheels: a timer due within 16384 ticks sits in the soon wheel; a later one waits in the later wheel until its slot comes within reach

The entry stored in a wheel is an `ErtstWheelTimer`. See `erl_time.h`[\[S2\]](#). Several timers in the same slot are linked together by the `prev` and `next` fields, `timeout_pos` is the absolute tick the timer is due, and `slot` records which slot the timer is in, or `ERTS_TW_SLOT_INACTIVE` when it is not set.

```

/*
** Timer entry:
*/
typedef struct erl_timer {
    ErtsMonotonicTime timeout_pos; /* Timeout in absolute clock ticks */
    struct erl_timer* next;        /* next entry tiw slot or chain */
    struct erl_timer* prev;        /* prev entry tiw slot or chain */
    void (*timeout)(void*);        /* called when timeout */
    void* arg;                     /* argument to timeout/cancel procs */
    int slot;
} ErtsTWheelTimer;

```

11.4. Interpreter and JIT scheduler entry points

Conceptually you can look at the scheduler as the driver of program execution in the Erlang VM. In the interpreter, the way the C code is structured, it is the emulator (process_main in emu/beam_emu.c) that drives the execution and it calls the scheduler as a subroutine to find the next process to execute. When running with the JIT (see [The JIT Compiler](#)), process_main is a global code fragment that serves the same role: it picks the next process, restores its registers, and jumps to the saved instruction pointer in native code.

Still, we will pretend that it is the other way around, since it makes a nice conceptual model for the scheduler loop. That is, we see it as the scheduler picking a process to execute and then handing over the execution to the emulator.

Looking at it that way, the scheduler loop looks like this:

11.5. Task-stealing lock and search mechanics

First the scheduler marks its own run queue as empty, so that other schedulers will not try to steal work from it, and releases its own run queue lock for the duration of the search. Then it checks if there are any inactive schedulers that it can steal a task from. If there are no inactive schedulers with stealable tasks then it will look at active schedulers, starting with schedulers having a higher id than itself, trying to find a stealable task.

The task stealing will look at one scheduler at a time and try to steal the highest priority task of that scheduler. Since this is done per scheduler there might actually be higher priority tasks that are stealable on another scheduler which will not be taken.

The task stealing tries to move tasks towards schedulers with lower numbers by trying to steal from schedulers with higher numbers, but since the stealing also will wrap around and

steal from schedulers with lower numbers the result is that processes are spread out on all active schedulers.

Erlang/OTP 28.0 reworked task stealing to reduce lock contention on systems with many schedulers. A scheduler looking for work first reads each candidate run queue's flag word atomically and skips any queue whose flags say it has nothing runnable, without touching that queue's lock. When a queue does look promising it tries the lock rather than waiting for it; a queue whose lock is busy is set aside and retried only after the others have been tried, and the search stops early once enough run queues are known to be empty. A successful steal also takes several processes at once, up to half of what the victim has. The effect is that idle schedulers no longer bounce lock cache lines between cores while looking for work (`check_possible_steal_victim` and `try_steal_task` in `erl_process.c`[\[S3\]](#)). Erlang/OTP 29.0 followed up by retrying the busy queues in the order they were found and by keeping count of schedulers that are waiting for work.

11.6. Migration planning and queue limits

This is done by the function `check_balance` in `erl_process.c`.

The migration is done by first setting up a migration plan and then letting schedulers execute on that plan until a new plan is set up. Every $2000 \cdot \text{CONTEXT_REDS}$ reductions a scheduler calculates a migration path per priority per scheduler by looking at the workload of all schedulers. The migration path has one of three values:

1. Cleared.
2. Migrate to a scheduler.
3. Immigrate from a scheduler.

When a process becomes ready (for example by receiving a message or triggering a timeout) it will normally be scheduled on the last scheduler it ran on (S1). That is, if the migration path of that scheduler (S1), at that priority, is cleared. If the migration path of the scheduler is set to emigrate (to S2) the process will be handed over to that scheduler if both S1 and S2 have unbalanced run-queues. We will get back to what that means.

When a scheduler (S1) is to pick a new process to execute it checks to see if it has an immigration path from (S2) set. If the two involved schedulers have unbalanced run-queues S1 will steal a process from S2.

The migration path is calculated by comparing the maximum run-queues for each scheduler for a certain priority. Each scheduler will update a counter in each iteration of its scheduler loop keeping track of the maximal queue length. This information is then used to

calculate an average (max) queue length (*AMQL*). Figure 13 shows an example with four equally available schedulers. The left panel has the maximum run queue length of each scheduler and the average, 3. The right panel has the same schedulers sorted on that length.

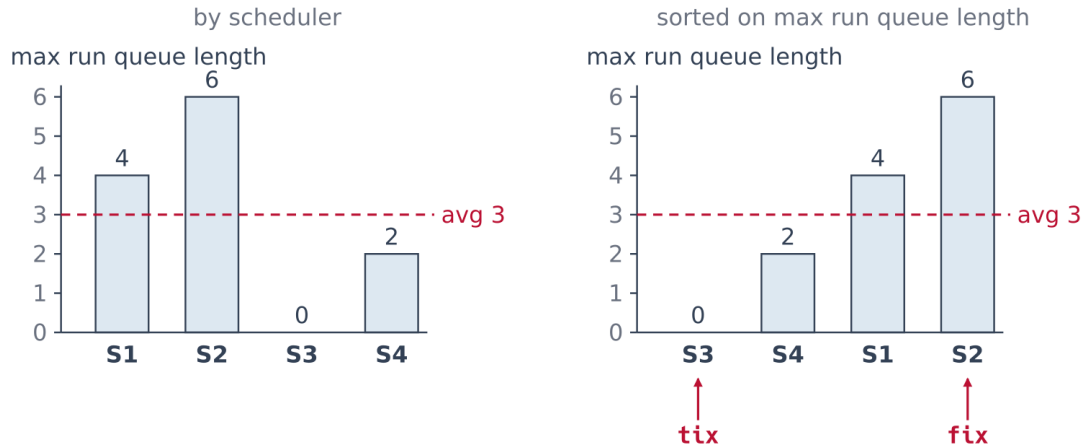


Figure 13. Maximum run queue lengths before and after sorting the schedulers

Any scheduler with a longer max run queue than average (S1, S2) will be marked for emigration and any scheduler with a shorter max run queue than average (S3, S4) will be targeted for immigration.

This is done by looping over the sorted schedulers with two indices, *fix* for the scheduler to immigrate from and *tix* for the scheduler to emigrate to. They start at the two ends of the order, as marked in the right panel of the figure. In each iteration of the loop the immigration path of $S[tix]$ is set to $S[fix]$ and the emigration path of $S[fix]$ is set to $S[tix]$. Then *tix* is increased and *fix* decreased until one reaches the balance point. If schedulers remain on the other side, a second pass reuses partners from the shorter side to set the remaining paths.

In the example:

- Iteration 1: $S2.emigrate_to = S3$ and $S3.immigrate_from = S2$
- Iteration 2: $S1.emigrate_to = S4$ and $S4.immigrate_from = S1$

Then we are done.

In reality things are a bit more complicated since schedulers can be taken offline. The migration planning is only done for online schedulers. Also, as mentioned before, this is done per priority level. OTP 29 weights each migration limit by the scheduler availability for that priority and rounds it up to an integer. It sorts schedulers by the difference between

their maximum queue length and that limit. With equal availability, as in our example, sorting by maximum length gives the same order.

When a process is to be inserted into a ready queue and there is a migration path set from S1 to S2 the scheduler first checks that the run queue of S1 is larger than its migration limit and that the run queue of S2 is smaller than its own limit. This way the migration is only allowed if both queues are still unbalanced.

There are two exceptions though where a migration is forced even when the queues are balanced or even imbalanced in the wrong way. In both these cases a special evacuation flag is set which overrides the balance test.

The evacuation flag is set when a scheduler is taken offline to ensure that no new processes are scheduled on an offline scheduler. The flag is also set when the scheduler detects that no progress is made on some priority. That is, if there for example is a max priority process which always is ready to run so that no normal priority processes ever are scheduled. Then the evacuation flag will be set for the normal priority queue for that scheduler.

Sources and links

[S1] Erlang/OTP Project. Erlang/OTP source code: `erl_process.h`. OTP-29.0; `erts/emulator/beam/erl_process.h`.
<https://github.com/erlang/otp>

[S2] Erlang/OTP Project. Erlang/OTP source code: `erl_time.h`. OTP-29.0; `erts/emulator/beam/erl_time.h`.
<https://github.com/erlang/otp>

[S3] Erlang/OTP Project. Erlang/OTP source code: `erl_process.c`. OTP-29.0; `erts/emulator/beam/erl_process.c`.
<https://github.com/erlang/otp>

Memory Implementation: Allocator

Reference and Term Layouts

Optional implementation detail accompanying the developer-facing chapter. The source extracts and version qualifications below describe the second-edition baseline.

12.1. Allocator catalogue

In the following sections we will go through the different allocators, with a little detour into the general framework for allocators (`alloc_util`).

Each allocator has several names used in the documentation and in the C code. See [Table 7](#) for a short list of all allocators and their names. The C-name is used in the C-code to refer to the allocator. The Type-name is used in `erl_alloc.types` to bind allocation types to an allocator. The Flag is the letter used for setting parameters of that allocator when starting Erlang.

Table 7. List of memory allocators.

Name	Description	C-name	Type-name	Flag
Basic allocator	<code>mmap</code> interface	<code>sys_alloc</code>	SYSTEM	Y
Memory segment allocator	<code>mmap</code> interface	<code>mseg_alloc</code>	-	M
Temporary allocator	Temporary allocations	<code>temp_alloc</code>	TEMPORARY	T
Heap allocator	Erlang heap data	<code>eheap_alloc</code>	EHEAP	H
Binary allocator	Binary data	<code>binary_alloc</code>	BINARY	B
ETS allocator	ETS data	<code>ets_alloc</code>	ETS	E
Driver allocator	Driver data	<code>driver_alloc</code>	DRIVER	R
Short lived allocator	Short lived memory	<code>sl_alloc</code>	SHORT_LIVED	S
Long lived allocator	Long lived memory	<code>ll_alloc</code>	LONG_LIVED	L
Fixed allocator	Fixed size data	<code>fix_alloc</code>	FIXED_SIZE	F

Name	Description	C-name	Type-name	Flag
Standard allocator	For most other data	std_alloc	STANDARD	D
Literal allocator	Module constants	literal_alloc	LITERAL	I (super carrier)

12.2. System allocation and segment-cache parameters

12.2.1. The Basic Allocator: `sys_alloc`

The allocator `sys_alloc` can not be disabled, and is basically a straight mapping to the underlying OS `malloc` implementation in `libc`.

If a specific allocator is disabled then `sys_alloc` is used instead.

All specific allocators use either `sys_alloc` or `mseg_alloc` to allocate memory from the operating system as needed.

When memory is allocated from the OS, `sys_alloc` can add (pad) a fixed number of kilobytes to the requested number. This can reduce the number of system calls by over allocating memory. The default padding is zero.

When memory is freed, `sys_alloc` will keep some free memory allocated in the process. The size of this free memory is called the trim threshold, and the default is 128 kilobytes. This also reduces the number of system calls at the cost of a higher memory footprint. This means that if you are running the system with the default settings you can experience that the Beam process does not give memory back to the OS directly as memory is freed up.

The underlying `malloc` decides where these allocations come from. On Linux it can extend a heap with `brk` or obtain mappings with `mmap`, so `sys_alloc` does not imply one contiguous C-heap region.

12.2.2. The Memory Segment Allocator: `mseg_alloc`

If the underlying operating system supports `mmap` a specific memory allocator can use `mseg_alloc` instead of `sys_alloc` to allocate memory from the operating system.

Memory areas allocated through `mseg_alloc` are called segments. When a segment is freed it is not immediately returned to the OS, instead it is kept in a segment cache.

When a new segment is allocated a cached segment is reused if possible, i.e., if it is the same size or larger than the requested size but not too large. The value of *absolute max cache bad fit* determines the number of kilobytes of extra size which is considered not too large. The default is 4096 kilobytes.

In order not to reuse a 4096 kilobyte segment for really small allocations there is also a *relative max cache bad fit* value which states that a cached segment may not be used if it is more than that many percent larger. The default value is 20 percent. That is a 12 KB segment may be used when asked for a 10 KB segment.

The number of entries in the cache defaults to 10 but can be set to any value from zero to thirty.

12.3. Carrier-size defaults and growth

12.3.1. Carrier Sizes

The main multiblock carrier is never released (`mmbcs`: nominally 128 KB for most allocators, 512 KB for `eheap_alloc` and 2 MB for `ll_alloc` on a 64-bit system). Additional multiblock carriers supplied by `mseg_alloc` grow in size: the first is `smbcs` bytes (nominally 1 MB), each of the next `mbcgs` carriers (ten by default) is a step larger, and from then on every new carrier is `lmbcs` bytes (nominally 10 MB). The manual gives the size of carrier number `nc` as $\text{smbcs} + nc * (\text{lmbcs} - \text{smbcs}) / \text{mbcgs}$.

These nominal values are what a single scheduler system uses. When an allocator runs one instance per scheduler, which every allocator with carrier migration does, `erl_alloc.c` divides `mmbcs` and `smbcs` by the number of schedulers up to four and `lmbcs` by the number of schedulers up to two, so on a machine with four or more schedulers the values in force are 32 KB, 128 KB and 512 KB for `mmbcs`, 256 KB for `smbcs` and 5 MB for `lmbcs`. `temp_alloc` and `literal_alloc` are not divided and keep 1 MB and 10 MB. All of these are per-allocator flags, and `erlang:system_info({allocator, A})` shows the values in force.

12.4. Allocation strategies and the full allocator catalogue

12.4.1. Memory Allocation Strategies

To find a free block of memory in a multi block carrier an allocation strategy is used. Each type of allocator has a default allocation strategy, but you can also set the allocation strategy with the `as` flag.

The Erlang Run-Time System Application Reference Manual (`erts_alloc`[\[S1\]](#)) lists ten strategies. Seven of them pick a block directly:

- Best fit (`bf`): the smallest free block that satisfies the request.

- Address order best fit (aobf): the smallest block, and among equals the one with the lowest address.
- Address order first fit (aoff): the block with the lowest address that satisfies the request.
- Address order first fit carrier best fit (aoffcbf): the carrier with the lowest address that can satisfy the request, then best fit within that carrier.
- Address order first fit carrier address order best fit (aoffcaobf): as above, but address order best fit within the carrier.
- Good fit (gf): a best fit search that gives up after a bounded number of candidates (mbsd, three by default), which makes it constant time.
- A fit (af): inspect one free block and take it if it fits. Only meant for temporary allocations.



Version Info

An Erlang/OTP 20.2 patch release (erts 9.2.1) added three age order strategies, ageffcaoff, ageffcbf and ageffcaobf. They pick the oldest carrier that can satisfy the request and then use address order first fit, best fit or address order best fit within it, so that a long lived allocator can hand its young carriers back.

The address order first fit family (aoff, aoffcbf, aoffcaobf and the age order strategies) are the ones that support carrier migration, where a carrier whose utilization has fallen below the limit (acul) is abandoned by one scheduler's allocator instance and picked up by another. That fact decides the defaults more than anything else. In `erl_alloc.c` the nominal defaults are best fit for `std_alloc`, `ll_alloc`, `binary_alloc`, `ets_alloc`, `driver_alloc` and `fix_alloc`, good fit for `sl_alloc` and `eheap_alloc`, a fit for `temp_alloc` and address order best fit for `literal_alloc`. But carrier migration is enabled by default for every `alloc_util` allocator except `temp_alloc` and `literal_alloc`, at 60% utilization (45% for `eheap_alloc`, 85% for `ll_alloc`), and a strategy that cannot migrate is replaced by `aoffcbf` when migration is on. So on a default Erlang/OTP 29 system eight of the ten allocators actually run address order first fit carrier best fit, which `erlang:system_info({allocator, binary_alloc})` will confirm. Only `temp_alloc` (a fit) and `literal_alloc` (address order best fit) keep their nominal strategy, and even `temp_alloc` runs good fit in the one instance that is shared by threads other than the schedulers. No allocator defaults to plain best fit.

This also means that choosing a strategy is not a free choice. Setting `+MBas aobf` gives `binary_alloc` address order best fit, and it also switches carrier migration off for that

allocator, since the two cannot be combined; the emulator reports `acul` as 0 afterwards. Whether that trade is worth it depends on the workload, and the manual's warning quoted at the start of this chapter applies in full.

12.4.2. The Temporary Allocator: `temp_alloc`

The allocator `temp_alloc` is used for temporary allocations, that is, very short lived allocations. Memory allocated by `temp_alloc` may not be allocated over an Erlang process context switch.

You can use `temp_alloc` as a small scratch or working area while doing some work within a function. Look at it as an extension of the C-stack and free it in the same way. That is, to be on the safe side, free memory allocated by `temp_alloc` before returning from the function that did the allocation. There is a note in `erl_alloc.types` saying that you should free a `temp_alloc` block before the emulator starts executing Erlang code.

Note that no Erlang process running on the same scheduler as the allocator may start executing Erlang code before the block is freed. This means that you can not use a temporary allocation over a BIF or NIF trap (yield).

In a default system there are $N+1$ `temp_alloc` allocators where N is the number of schedulers. The `temp_alloc` uses the "A fit" (af) strategy. Since the allocation pattern of the `temp_alloc` basically is that of a stack (mostly of size 0 or 1), this strategy works fine.

The temporary allocator is, in Erlang/OTP 29, used by the following types of data: `TMP_HEAP`, `ROOTSET`, `LOADER_TMP`, `NC_TMP`, `TMP`, `DCTRL_BUF`, `ESTACK`, `DB_TMP`, `DB_MC_STK`, `DB_MS_CMPL_HEAP`, `LOGGER_DSBUF`, `TMP_DSBUF`, `DDL_TMP_BUF`, `RE_TMP_BUF`, `TEMP_TERM`, `SYS_READ_BUF`, `SYS_WRITE_BUF`.

For an up to date list of allocation types allocated with each allocator, see `erl_alloc.types` (e.g., `grep TEMPORARY erts/emulator/beam/erl_alloc.types`).

I will not go through each of these different types, but in general as you can guess by their names, they are temporary buffers or work stacks.

12.4.3. The Heap Allocator: `eheap_alloc`

The heap allocator manages memory blocks for a process's private data: Erlang terms stored on the process heap (new and old generations), plus related structures like heap fragments and beam registers. Almost every Erlang term a process creates ends up in memory obtained through `eheap_alloc`.

By default, each scheduler has one `eheap_alloc` instance, so memory for processes running on that scheduler remains mostly local, reducing contention. The allocator’s frequent tasks include:

- **Process Heap:** Each Erlang process has its own heap where it stores data such as tuples, lists, maps, integers, and any small binaries (≤ 64 bytes).
- **Heap Fragments:** If a process briefly needs more memory but cannot immediately GC (e.g., constructing a large message), the VM may allocate a “heap fragment” from `eheap_alloc`. On the next garbage collection, these fragments are merged or freed.
- **Register Arrays:** Some runtime-implementation details (e.g., the “`beam_registers`” data structure) also use `eheap_alloc`. As an Erlang developer, you typically optimize around `eheap_alloc` usage by controlling process heap sizes or by understanding when your processes generate large, short-lived data. Good practice includes carefully sizing process heaps if they frequently handle big data, rather than allowing many minor GCs.

12.4.4. The Binary Allocator: `binary_alloc`

The binary allocator handles memory for (yes, you guessed it) binaries. Specifically, it manages the payload of binaries larger than 64 bytes, which live off the process heaps and are reference counted. What a process heap holds is a small handle, a `BinRef` (called a `ProcBin` before OTP 27), and a sub bitstring that refers to it.

These binaries vary widely, from modestly sized binaries (hundreds of bytes) up to massive binaries measured in megabytes (such as entire file contents or external data from networks).

The allocator has a few interesting characteristics:

- **Allocation Strategy:** With default OTP 29 carrier migration, it uses address order first fit carrier best fit (`aoffcbf`), as described above. It first selects a suitable carrier and then a best-fitting block within that carrier.
- **Reference Counting:** Off-heap binaries stick around until the very last process stops referencing them. If just one forgetful process clings to a huge binary, it stays alive, sometimes annoyingly longer than you’d prefer.
- **Sub-Binaries (Slices):** When you match binaries like `<<X:32, Rest/binary>>`, the smaller binary (`Rest`) still references the original larger binary, avoiding copying is great, until you realize you’ve accidentally kept a giant binary alive just to reference a tiny bit. In these cases, calling `binary:copy/1` is your friend.

If memory mysteriously grows, check for large binaries hanging around due to references lingering in processes. Occasionally, you might need to gently remind the garbage collector (via `erlang:garbage_collect/1`) or let processes hibernate to reclaim the memory faster.

12.4.5. The ETS Allocator: `ets_alloc`

The allocator `ets_alloc` manages the table structures and stored objects outside Erlang process heaps. Each table has an owner, initially the creating process. If the owner exits, the table is deleted unless ownership transfers to an heir.

An insertion copies the object into the table's storage. The caller can still hold its original term; inserting it does not move that term off the process heap. Process garbage collection does not reclaim the table's copy. Removing objects releases their storage, although traversal fixations can delay reclamation. Deleting the table, explicitly or when its owner exits without an heir, also removes its fixations. We will follow these lifetimes in [Term Storage: ETS and Persistent Terms](#).

The allocation types include the table header, hash-table objects and tree nodes. Some supporting work uses other allocators: temporary matching data uses `temp_alloc`, for example. A reference-counted binary payload remains in `binary_alloc`, even when an ETS object holds a reference to it.

Use `ets:info(T, memory)` to inspect a table's memory in words, and allocator statistics to inspect the carriers and blocks used by `ets_alloc`. Those measurements describe different layers of the same storage.

12.4.6. The Driver Allocator: `driver_alloc`

The driver allocator handles memory for ports, linked-in drivers and NIF allocations. Follow the allocation through the native interface to find who must release it:

- **Port & Driver Data:** Allocations here include structures for network sockets, open file descriptors, and buffers specific to your linked-in drivers.
- **NIF-Allocated Data:** When a NIF reaches out with `enif_alloc`, the memory ultimately comes from this allocator. The NIF must call `enif_free` on every ownership path. Process exit does not automatically release a raw allocation. A resource destructor can release it only if the NIF explicitly stores that pointer in the resource and frees it from the destructor.

- **Potential For External Leaks:** A driver or NIF that loses an allocated pointer without freeing it retains that memory until the node exits. Compare `driver_alloc` usage before and after the native operation, including its cleanup.

Though developers rarely manipulate `driver_alloc` directly, it's wise to monitor this allocator in production environments, especially if you are using NIFs.

12.4.7. The Short-Lived Allocator: `sl_alloc`

The short-lived allocator (`sl_alloc`) handles blocks expected to be released soon, including state that must survive a scheduling point. By comparison, the guidance for `temp_alloc` is to release its memory before the emulator resumes executing Erlang code.

The OTP 29 allocation types include prepared code waiting to be loaded, timer requests and state saved when a BIF yields. The operation that needs this state releases it when it finishes. These are runtime allocations, separate from the Erlang terms reclaimed by a process's garbage collector.

If `sl_alloc` keeps growing, inspect which operations remain in progress and compare allocated blocks with retained carrier space. A large carrier total alone does not show that the operations have leaked their state.

12.4.8. The Long-Lived Allocator: `ll_alloc`

The long lived allocator handles data intended to stay alive for extended periods, often as long as the Erlang node itself. Typical examples include:

- **Atoms:** Once created, atoms persist indefinitely, making them permanent residents of `ll_alloc`.
- **Loaded Modules and Code:** Compiled modules, exported functions, and metadata related to anonymous functions (`fun`s) are stored here. For instance, when you load a module via `code:load_file(my_module)`, its metadata lands in `ll_alloc`.
- **Scheduler and System Structures:** Internal runtime structures, like scheduler run queues (`run_queue`), pollset information (`pollset`), and the process registry (`proc_tab`), live here because they're fundamental to VM operations.

Because objects in `ll_alloc` tend to stick around, this allocator typically grows slowly but continuously during a node's lifetime. If your system frequently loads and unloads modules, you might see fluctuations. For instance, repeatedly executing hot-code loading without properly unloading old versions can gradually inflate memory usage in `ll_alloc`.

Inspect `erlang:system_info({allocator, ll_alloc})` to see its allocator instances. Compare the current block sizes with the current carrier sizes: blocks describe allocations in use, while carriers also include space available for reuse. Take another sample after loading or purging modules to see which totals changed.

12.4.9. The Fixed-Size Allocator: `fix_alloc`

The fixed allocator, `fix_alloc`, specializes in allocating fixed-size objects, typically small C structs whose size never changes (e.g., message references, driver event data, monitors). Since these objects come in uniform sizes, the allocator can efficiently handle them.

Its nominal default strategy in `erl_alloc.c` is best fit, which carrier migration turns into address order first fit carrier best fit like most of the others. What makes it fast is that freed objects go into per-type free lists of same-sized blocks, so a fixed size allocation rarely searches at all. The next request for the same type can reuse one of those blocks.

Examples include internal VM structures like `ErlMessage`, monitor references, and scheduler bookkeeping data.

A freed object can remain on its type's free list for reuse. Its lifetime has ended even though the allocator still holds the block.

12.4.10. The Standard Allocator: `std_alloc`

`std_alloc` handles allocation types assigned to the standard allocator. For example, OTP 29's `erl_alloc.types` assigns process dictionaries (`PROC_DICT`) and I/O queues (`IOQ`) to it.

Inspect `erlang:system_info({allocator, std_alloc})` and compare current block sizes with current carrier sizes. If carriers grow while blocks remain steady, the allocator holds more space around its live allocations. The figures alone do not identify which allocation type caused the change. Like the other allocators, `std_alloc` has startup controls such as `+MDSbct 1024`, a singleblock threshold in kilobytes. Measure the block and carrier totals before changing it.

12.4.11. The Literal Allocator: `literal_alloc`

The `literal_alloc` stores compile-time constants, often called the *literal pool*, in loaded Erlang modules. Think of it as the VM's "read-only memory shelf," where Erlang safely stores constants like large static binaries, tuples, or lists defined at compile time.

Unlike typical allocators, `literal_alloc` is managed globally rather than per scheduler, as literals aren't frequently modified or reclaimed during runtime. Once loaded, these literals remain until their corresponding module is explicitly purged or reloaded.

Monitoring `literal_alloc` is usually uneventful, but frequent dynamic module updates or loading large literal-heavy modules repeatedly might make it worth a glance.

12.5. Allocator flag reference and original tuning examples

12.5.1. System Flags for Memory

Memory allocator system flags follow this syntax:

```
+M<S><P> <V>
```

- `<S>` is a single uppercase letter identifying the allocator.
- `<P>` specifies the parameter.
- `<V>` specifies the value to use.

Allocator identifiers (`<S>`):

- B: `binary_alloc`
- D: `std_alloc`
- E: `ets_alloc`
- F: `fix_alloc`
- H: `eheap_alloc`
- I: `literal_alloc`
- L: `ll_alloc`
- M: `mseg_alloc`
- R: `driver_alloc`
- S: `sl_alloc`
- T: `temp_alloc`
- Y: `sys_alloc`
- u: `alloc_util` (affects all `alloc_util`-based allocators)

12.5.2. Commonly Used Flags

Allocation Strategy (as):

Determines how memory blocks are selected within carriers.

- bf: Best fit
- aobf: Address order best fit
- aoff: Address order first fit
- aoffcbf: Address order first fit carrier best fit
- ageffcbf: Age order first fit carrier best fit
- gf: Good fit
- af: A fit

Example:

```
+MBas bf
```

(Binary allocator uses best-fit strategy.)

Singleblock Carrier Threshold (sbct): Defines the threshold in KB above which allocations use singleblock carriers.

Example:

```
+MBsbct 1024
```

(Binary allocations above 1024 KB use singleblock carriers.)

Multiblock Carrier Settings:

- **Smallest (smbcs) and Largest (lmbcs) Multiblock Carrier Size:** Control minimum/maximum sizes for multiblock carriers (KB).
- **Carrier Growth Stages (mbcgs):** Defines carrier size growth between minimum and maximum.

Example:

```
+MBsmbcs 512 +MBlmbcs 8192
```

(Binary allocator multiblock carriers range from 512 KB to 8 MB.)

Abandon Carrier Utilization Limit (acul): Percentage threshold below which carriers are abandoned and reused.

Example:

```
+MBacul 50
```

(Binary carriers with utilization below 50% are marked abandoned.)

Abandon Carrier Free Utilization Limit (acful): Below this utilization, the VM informs the OS that unused memory can be reclaimed.

Example:

```
+MDacful 10
```

(std_alloc marks memory as reclaimable by OS if utilization is below 10%.)

Multiple Thread-specific Instances (t): Controls if allocators use multiple instances (one per scheduler).

Example:

```
+Mht true
```

(eheap_alloc uses separate allocator instances per scheduler thread.)

Allocation Tagging (atags): Adds tags to allocations, useful for debugging with instrumentation.

Example:

```
+MRatags true
```

(Enable tagging for driver_alloc.)

12.5.3. Special Flags for mseg_alloc

mseg_alloc (Memory Segment Allocator) has specific settings:

- **Super Carrier Size (scs):**

```
+MMscs 1024
```

Creates a 1GB super carrier.

- **Use Large Pages (lp):**

```
+MMLp on
```

Enable large/huge pages support.

- **Super Carrier Only (sco):**

```
+MMsco true
```

Allocations occur only within the super carrier.

- **Maximum Cached Segments (mcs):**

```
+MMmcs 20
```

Stores up to 20 cached segments.

12.5.4. Special Flags for `sys_alloc`

`sys_alloc` interfaces with the system's `malloc`:

- **Trim Threshold (tt):** Releases memory back to the OS when the free heap exceeds threshold.

```
+MYtt 256
```

Set trim threshold to 256 KB.

- **Top Pad (tp):** Extra memory `malloc` requests from OS to reduce subsequent calls.

```
+MYtp 512
```

`malloc` requests 512 KB padding.

12.5.5. Literal Allocator (`literal_alloc`)

Stores literals (compile-time constants):

- **Literal Super Carrier Size (scs):**

```
+MIscs 2048
```

Set literal allocator super carrier to 2 GB.

12.5.6. Global and Convenience Flags

- **Minimal/Maximal Allocation Setup (ea):**

```
+Mea min|max|config
```

Quick configuration presets for all allocators.

- **Lock Physical Memory (lpm):**

```
+Mlpm all|no
```

Locks VM memory into physical RAM.

- **Dirty Allocator Instances (dai):**

```
+Mdai max|<number>
```

Allocator instances specifically for dirty schedulers.

12.5.7. Practical Examples

Use address order best fit in every allocator, at the cost of carrier migration:

```
+Muas aobf
```

The strategy takes effect, and `acul` becomes 0 for every allocator, because only the carrier order strategies can migrate carriers.

Let the ETS allocator retain carriers until utilization falls below 40%, instead of the default 60% threshold:

```
+MEacul 40
```

Enable allocation tagging for debugging across all allocators:

```
+Muatags true
```

There is no flag that caps the memory of a single allocator. A flag the emulator does not know, such as a made up `+MEamax`, is rejected at start with bad "ME" parameter. To bound ETS memory, bound the tables: `ets:info/2` and `erlang:memory(ets)` show where it goes, and `recon_alloc` shows what the allocator holds.

12.6. Fun-object C layout

Excerpt from OTP 29 `erl_fun.h`; comments omitted or shortened.

```
typedef struct erl_fun_thing {
    Eterm thing_word;
    union {
        const ErtsDispatchable *disp;
        const ErlFunEntry *fun;    /* valid iff this is a local fun */
        const Export *exp;         /* valid iff this is an external fun */
    } entry;
    /* Environment (free variables), may be compound terms. */
    Eterm env[];
} ErlFunThing;
```

12.7. Repeated tuple-update invariants

Follow the tuple through the update: the compiler must establish who can still refer to it, and ERTS must keep the heap safe for collection. The uniqueness decision is made at compile time: the compiler's alias analysis proves that the variable holding the tuple is the only live reference to it and that the tuple dies at the update, and only then does it emit the `update_record` instruction with the `inplace` hint (see [The Compiler](#)). Tuples carry no reference count. The JIT trusts that alias proof, but separately protects the generational GC invariant: if new values might be heap pointers, it checks that the tuple lies between `high_water` and `HTOP`, copying it when necessary. Immediate and literal replacement values need no such copy fallback.

Three invariants make that write safe on the heap. The tuple must be a heap object, so the compiler rewrites updates of literal tuples into a fresh `put_tuple` instead. Its size does not change, so the header word and every neighbour on the heap are left as they were, and the garbage collector sees an ordinary tuple. And nothing else can observe the old contents, because uniqueness was proven before the code ran. When the runtime safety condition also holds, the result is an update with no allocation and no garbage, which is especially helpful for sequential state transitions inside loops, where large record or state tuples are modified repeatedly. The semantics of the language are untouched; a program cannot tell the difference.

12.8. Message-fragment allocation and layout

Let us follow the fragment case. A new heap fragment is allocated (of `sizeof(ErlHeapFragment) - sizeof(Eterm) + 23*sizeof(Eterm)`)¹ which after initialization will look like:

```
erl_heap_fragment:
  ErlHeapFragment* next;      NULL
  ErlOffHeap off_heap:
    erl_off_heap_header* first;  NULL
    Uint64 overhead;             0
  Uint alloc_size;             23
  Uint used_size;              23
  Eterm mem[1];                ?
  ... 22 free words
```

12.9. Message container and signal-queue pointers

In either case a new message container (`ErlMessage`) is allocated, the receiver's signal queue lock (`ERTS_PROC_LOCK_MSGQ`) is taken, and the message on the heap or in the new heap fragment is linked into the container. Sketched, with only the fields that matter here, the container looks like this (the real struct `erl_mesg` in `erl_message.h` has three slots in `m`, for the message, the sequential trace token and the sender, and embeds a heap fragment of its own):

```
erl_mesg {
  struct erl_mesg* next = NULL;
  data: ErlHeapFragment *heap_frag = bp;
  Eterm m[0]           = message;
} ErlMessage;
```

Then the container is linked into the receiver's outer signal queue (`sig_inq` in the PCB) and the lock is released. `sig_inq.last` points to the `next` field of the last entry, so appending is a matter of updating that pointer and the last pointer. Since OTP 21 messages share this queue with every other signal a process can receive, and the receiver takes the signal-queue lock while fetching `sig_inq` into its private queues. It can then process those private queues without holding that lock. The three-queue architecture and the field names are described in [Processes](#).

¹The `-sizeof(Eterm)` comes from `mem` in `ErlHeapFragment` already having the size of one `Eterm`.

12.10. Atom-table allocation classes

Each entry in the atom table is an `Atom` struct (shown in [The Erlang Type System and Tags](#)) holding the index slot that gives the atom its number and links it into the hash chain, the length of the name, the number of characters if the name fits in Latin-1, a precomputed ordinal of the first bytes used when atoms are ordered, and the name itself.

The atom table is accounted for under the `atom_data` allocation class, which has three allocation types, all long lived (see the section on `ll_alloc` above):

- `atom_entry`: The `Atom` structs themselves.
- `atom_tab`: The index table that maps atom numbers to entries and the hash buckets used for lookup.
- `atom_text`: The text area that held the names up to OTP 27. The type is still declared in `erl_alloc.types` at OTP 29, but no code allocates from it any more.

Sources and links

[S1] Erlang/OTP Project. `erts_alloc`. Erlang/OTP 29 documentation.
https://www.erlang.org/docs/29/apps/erts/erts_alloc.html

ETS Implementation: Structures, Locks and Matching

Optional implementation detail accompanying the developer-facing chapter. The source extracts and version qualifications below describe the second-edition baseline.

13.1. Table headers and status flags

The set, protected and size values we just inspected come from the runtime structures for `T`. `ets:new/2` allocates a `DbTable`, a C union defined in `erts/emulator/beam/erl_db.h`. Its alternatives hold a hash table, a tree or a contention adapting tree. Each starts with the same `DbTableCommon` header from `erl_db_util.h`.

For `T`, the header records the shell's pid as owner and the status flags `DB_SET` and `DB_PROTECTED`. Those flags select the hash-table operations and permit other processes to read our objects. The key position says which tuple element the table treats as its key.

The header also records the table name, heir pid and heir data, and whether objects are compressed. Its reference count, fixation count and table-wide read-write lock help ERTS manage access and lifetime. We will reach those mechanisms when another process uses `T`, and when we traverse and delete it.

Table 8. The table's status flags, for reference.

Property	Flags
Representation	<code>DB_SET</code> , <code>DB_BAG</code> , <code>DB_DUPLICATE_BAG</code> , <code>DB_ORDERED_SET</code> , <code>DB_CA_ORDERED_SET</code>
Protection	<code>DB_PRIVATE</code> , <code>DB_PROTECTED</code> , <code>DB_PUBLIC</code>
Deletion in progress	<code>DB_DELETE</code>
Concurrency options	<code>DB_FINE_LOCKED</code> for <code>write_concurrency</code> , <code>DB_FREQ_READ</code> for <code>read_concurrency</code> , <code>DB_FINE_LOCKED_AUTO</code> for <code>{write_concurrency,</code> <code>auto}</code>

The size query returned five because the header counts the objects in `T`. A second counter tracks its memory in bytes. The counters are kept in an `EtsFlxCtr`, which can use a single atomic word per counter or spread the words across cache lines, one per group of schedulers. The `decentralized_counters` option chooses between them; we will compare their update and read costs below.

13.2. Magic references and the meta name table

What `ets:new/2` returns is a reference, and the reference is a *magic reference*: it points to a `Binary` that the runtime allocated for the table, the `btid` field of the header, whose destructor is the function `db_table_tid_destructor`. When a BIF receives a table identifier it checks that the argument is an internal magic reference and that the binary behind it has that destructor, and then it has the `DbTable` pointer in one step, without any global table of tables. A named table is different only in that `ets:new/2` returns the name and the BIFs accept the atom. Atoms are resolved through the *meta name table*, a hash table in `erl_db.c` indexed by the atom's index, with its own array of 256 locks, and a named table is also reachable by reference through `ets:whereis/1`.

13.3. ETS BIF table lookup

The lookup happens in `db_get_table_aux`. After finding the table it takes the lock the operation needs, then checks the protection: a `private` table may only be touched by its owner, a `protected` one may be read by anyone and written only by the owner, and a `public` one is open. The check compares the owner pid in the header with the calling process. This is a small part of the lookup path; its measured cost depends on the operation and workload. The lookup also asserts that it runs on a normal scheduler; the ETS BIFs are never scheduled dirty, because the memory management of the tables depends on the thread progress of the normal schedulers.

13.4. Ownership lists and yielding deletion

Each process keeps a list of the tables it owns, in its process specific data under `ERTS_PSD_ETS_OWNED_TABLES`, and a list of the tables it has fixed. When the process exits, the runtime walks the owned tables in `erts_db_process_exiting`. For each table it first looks at the heir: if one is set and is a different process, `give_away_to_heir` transfers the table and sends the `{'ETS-TRANSFER', Tid, FromPid, HeirData}` message, if heir data was given.

Otherwise the table is deleted: its access bits are cleared, `DB_DELETE` is set, a named table is removed from the meta name table, fixations are released, and the objects are freed. Freeing a large table takes time, so the exit handler yields between tables and inside a table, counting reductions like any other long running operation; a process with a big table does not disappear in one scheduler slice.

`ets:delete/1` does the same work from a running process. It is allowed to be called by a process other than the owner when the table is public, and in that case it first transfers the ownership to the caller so that the deletion completes even if the caller is killed halfway. It then frees the table in steps of `db_free_table_continue`, and when the reductions run out it traps to an internal continuation function with the `DbTable` pointer packaged as a bignum, because a raw pointer on the heap would confuse a traced BIF call. `ets:give_away/3` is the explicit version of the heir mechanism, with the same message.

13.5. DbTerm storage layout

The insertion copy is made by `db_store_term` in `erl_db_util.c` into memory from the ETS allocator, `ets_alloc`, described in [The Memory Subsystem: Stacks, Heaps and Allocators](#), and the copy is a `DbTerm`: a header with a pointer to the off heap list, the size of the term in words, and then the term itself with the top level tuple first. The elements are copied in full, the way a message is copied, with one exception. A reference counted binary, the kind that lives outside process heaps, is not copied; the table gets its own `BinRef` pointing at the same binary and the reference count goes up. The off heap pointer in the `DbTerm` header is what lets the table release those references when the object is deleted.

13.6. Compressed-table encoding

With the `compressed` option the copy is stored differently. The key element and every immediate element are kept as terms, and every other element is encoded in the external term format, the format of `term_to_binary/1`, and stored after the tuple; the tuple slot then holds a byte offset into that area, tagged as a header word so that nothing can mistake it for a term. `copy_to_comp` and `db_copy_from_comp` in `erl_db_util.c` do the conversion in each direction. A lookup decodes the object it returns, and a `match` or `select` over the table decodes every object it inspects, which is why the documentation warns that those operations get much slower.

13.7. Hash-table buckets, indexing and splitting

The three unordered table types share one implementation, `erl_db_hash.c`, a linear hash table. Objects live in buckets as singly linked lists of `HashDbTerm`, each entry holding the next pointer, the stored hash value, a pseudo-deleted bit that we return to under fixation, and the `DbTerm`. A bag keeps all objects with the same key adjacent in the chain, and a `duplicate_bag` additionally allows the same object twice.

The bucket array is not one allocation. It is a table of segments: the first segment has 256 slots and every later one 2048, and the segment table itself grows in steps as segments are added. The hash of a key is computed by `erts_internal_hash`, or taken from the atom table when the key is an atom, and the slot index is the hash masked by the current size mask, with a fold when the index lands above the number of active buckets:

Simplified from OTP 29 `erl_db_hash.c`; the mask expression is elided.

```
static ERTS_INLINE Uint hash_to_ix(DbTableHash* tb, HashValue hval)
{
    Uint mask = ... tb->szm ...;
    Uint ix = hval & mask;
    if (ix >= erts_atomic_read_nob(&tb->nactive)) {
        ix &= mask>>1;
    }
    return ix;
}
```

This is linear hashing. The table grows one bucket at a time: when the number of objects exceeds the number of active buckets, `grow` activates the next slot and splits exactly one existing bucket between the old and the new index, using one more bit of the hash. Nothing else moves. There is no moment where the whole table is rehashed, so an insert pays for at most a handful of bucket splits, five at most and in the common case one, and a split takes the lock of the bucket it splits, not the table lock. Shrinking works the same way in reverse when the table becomes sparse, and neither `grow` nor `shrink` happens while the table is fixed.

13.8. Adaptive-lock resizing thresholds

`{write_concurrency, auto}` starts with the same 64 locks and lets the table choose. Every lock in the array carries a statistic. When a writer's `trylock` fails and it has to wait, the statistic goes up by a contention contribution; when the lock was free it goes down by one. A

statistic above `LCK_AUTO_GROW_LIMIT` (1000) asks the table to double the lock array, up to 8192 locks, provided the table has at least twice as many active buckets as locks; a statistic that sinks far below zero asks it to halve. With `read_concurrency` as well, the frequent-read lock variant is kept only up to 128 locks. The resize is done lazily, the next time a BIF looks up the table and can take the table lock, and never while the table is fixed, since a traversal in progress is indexing buckets by lock. `ets:info(T, write_concurrency)` answers `auto` for such a table, and the documentation recommends `auto` over `true` on OTP 25 and later for most workloads.

13.9. Hash growth counters and scheduler groups

The hash table has a further use for the counters: it needs an estimate of the load to decide when to grow, and with decentralized counting it keeps an item counter in each of the first 64 lock slots, reads the one counter that the key's hash selects and multiplies by 64. The `+dcg` emulator flag limits the number of counter groups, 256 by default; with more schedulers than groups, schedulers share.

13.10. AVL traversal stacks

An `ordered_set` is an AVL tree, in `erl_db_tree.c`, ordered by the term order of the keys. Insertion and deletion are the textbook algorithms with one difference the file comments point out: the implementation is not recursive, it keeps an explicit stack of the path from the root while it descends and rebalances on the way back up. The tree has no buckets to split and no hash to compute, and a lookup is a walk from the root comparing keys with `erts_cmp`, so the cost of an operation grows with the logarithm of the table size and with the cost of comparing keys.

The table keeps one *static stack*, a saved path into the tree, so that a process walking the table with `ets:next/2` does not descend from the root for every step: if the key asked for is the one the stack points at, the next key is a short walk. The stack is claimed with an atomic exchange and released after the call; a second process that arrives while it is busy allocates a temporary stack instead. That is an optimisation for one traverser at a time, and it is the reason the concurrent implementation below does not have it.

13.11. Contention-adapting node statistics and reclamation

There are two kinds of node. A *route node* holds a key and two children, and directs a search left or right the way any binary search tree does; it also has a lock, used only when the tree is restructured. A *base node* holds a read-write lock, a contention statistic, and the root of an ordinary AVL tree with all the keys in its range. An operation walks the route nodes without locking, arrives at a base node, takes its lock in the mode the operation needs, and works on the AVL tree inside. Two writers on different base nodes never meet.

Watch the contention statistic when an operation takes a base node's lock. A failed `trylock` adds 250; acquiring the lock without waiting subtracts one. When a writer releases the lock, `wunlock_adapt_base_node` checks the result.

Above 1000, the base node splits. ERTS cuts its AVL tree around a middle key, puts a route node in the original base node's place and makes each half a base node with its own lock. Writers whose keys now fall in different halves can proceed independently. Splitting stops at a route node depth of 16.

Below minus 1000, or when a base node is empty, ERTS joins it with a neighbour. It removes the route node between them and merges the AVL trees. This reduces the number of nodes a traversal must visit when the table no longer needs as many independently locked ranges.

A new table is one base node. A read operation increments the statistic on a failed `trylock` too, so readers can also cause a split, and readers have a small random chance of triggering a join on release, so that a tree with no writers still contracts. The split and join are done under the locks of the nodes involved and the old nodes are freed through the thread progress mechanism, so that a concurrent reader still walking through a removed route node finishes safely.

13.12. The pattern-matching machine and compiled program

The runtime does not interpret the specification as a term. It compiles it. `db_match_compile` in `erl_db_util.c` turns the triples into a program for a small stack machine the source calls *Pam*, the pattern matching machine, and `db_prog_match` runs that program against an object. The instruction set, the `MatchOps` enumeration, has operations to walk into a tuple, a list or a map (`matchTuple`, `matchPushT`, `matchPushL`, `matchPushM`), to bind and compare variables (`matchBind`, `matchCmp`), to compare against constants of each kind (`matchEq`, `matchEqBin`, `matchEqFloat`, `matchEqBig`, `matchEqRef`), to call a guard BIF with zero to three arguments (`matchCall0` to `matchCall3`), to build the result (`matchMkTuple`, `matchConsA`,

`matchPushV`), and the logic of `andalso`, `orelse` and the guard connectives. The allowed guard operations include arithmetic, comparisons, type tests, and functions such as `length` and `element`. Type tests use their `is_` spellings. These operations cannot have side effects; a guard exception causes the clause to fail for that object.

The compiled program is a `MatchProg` in a magic binary, allocated by `erts_create_magic_binary` with the destructor `erts_db_match_prog_destructor`. `ets:match_spec_compile/1` hands that binary to the caller as a magic reference and `ets:is_compiled_ms/1` checks the destructor:

13.13. Match pseudo processes

A `select` on a table compiles the specification once, at the start of the call, and runs the program against each object it visits. The program needs somewhere to bind its variables and build its result, and it needs a heap of its own, since a match program can run with no calling process at all, as trace match specifications do, and most of what it builds is temporary. Each scheduler therefore owns a *match pseudo process*, an `ErtsMatchPseudoProcess` with a `Process` structure and a default heap of 128 words, that the program runs in; results are copied to the caller's heap afterwards. Guard BIFs called from a match program see that pseudo process as their process.

13.14. Continuation fields

For a hash table the implementation's continuation carries the table identifier, the bucket index where the traversal stopped, the chunk size, a magic reference to the compiled match program, so that it is compiled once, and the matched objects that did not fit in the last chunk, with their count. `ets:select/1` picks the traversal up from the bucket index. The continuation is an ordinary term and can be sent to another process, but the match program reference in it belongs to the node that made it, and a continuation that has been through `term_to_binary/1` needs `ets:repair_continuation/2` before it can be used again.

13.15. Persistent-term key-table representation

A term put with `persistent_term:put/2` is copied once into a literal area of its own, allocated with `ERTS_ALC_T_LITERAL` and tagged as a literal, and the keys are kept in a hash table that starts with eight slots and grows at fifty percent load.

I/O Implementation: Requests and Polling

Optional implementation detail accompanying the developer-facing chapter. The source extracts and version qualifications below describe the second-edition baseline.

14.1. I/O protocol request catalogue

We will discuss group leaders later, but let us look at the actual I/O protocol first. The protocol has the following messages:

14.1.1. Basic Messages

- `{io_request, From, ReplyAs, Request}`:
 - `From`: `pid()` of the client process.
 - `ReplyAs`: Identifier for matching the reply to the request.
 - `Request`: The I/O request.
- `{io_reply, ReplyAs, Reply}`:
 - `ReplyAs`: Identifier matching the original request.
 - `Reply`: The response to the I/O request.

14.1.2. Output Requests

- `{put_chars, Encoding, Characters}`:
 - `Encoding`: `unicode` or `latin1`.
 - `Characters`: Data to be written.

- {put_chars, Encoding, Module, Function, Args}:
 - Module, Function, Args: Function to produce the data.

14.1.3. Input Requests

- {get_until, Encoding, Prompt, Module, Function, ExtraArgs}:
 - Encoding: unicode or latin1.
 - Prompt: Data to be output as a prompt.
 - Module, Function, ExtraArgs: Function to determine when enough data is read.
- {get_chars, Encoding, Prompt, N}:
 - Encoding: unicode or latin1.
 - Prompt: Data to be output as a prompt.
 - N: Number of characters to be read.
- {get_line, Encoding, Prompt}:
 - Encoding: unicode or latin1.
 - Prompt: Data to be output as a prompt.

14.1.4. Server Modes

- {setopts, Opts}:
 - Opts: List of options for the I/O server.
- getopts:
 - Requests the current options from the I/O server.

14.1.5. Multi-Request and Optional Messages

- {requests, Requests}:
 - Requests: List of valid io_request tuples to be executed sequentially.
- {get_geometry, Geometry}:
 - Geometry: Requests the number of rows or columns (optional).

14.1.6. Unimplemented Request Handling

If an I/O server encounters an unrecognized request, it should respond with: `{error, request}`.

14.2. Port-task queue mechanics

Queued tasks enter a public queue protected by a port-specific lock. The executing scheduler moves them into a private structure and works through them without holding that queue lock. Aborted tasks are marked atomically and skipped; thread progress protects their later reclamation.

14.3. Scheduler-pollset migration threshold

The interesting part is what happens when a descriptor is busy. Every read notification costs a trip through a poller thread: it wakes, finds the descriptor ready, and hands the event to a scheduler. For a socket that is read continuously that trip is pure overhead, so the runtime moves such descriptors out of the poller threads' pollset and into a pollset the scheduler threads poll themselves, without blocking, as part of their own loop. The rule in `erl_check_io.c` is simple: when a NIF re-selects a descriptor for reading more than ten times in a row on behalf of the same process, the descriptor migrates to the scheduler pollset, and a select from a different process resets the count, so a listen socket with several processes calling `accept` is left alone. Port drivers get the same treatment after more than ten consecutive ready inputs. The optimization is controlled by `+IOs` (default `true` where `epoll` or `kqueue` is available) and it trades a little scheduling latency on those descriptors for fewer system calls and no poller wake up.

Distribution Implementation: Remote-Spawn Controls

Optional implementation detail accompanying the developer-facing chapter. The source extracts and version qualifications below describe the second-edition baseline.

15.1. Remote-spawn control messages

To create a process on B, A sends a spawn request. Follow its arguments through the request and reply:

1. A process on A calls `spawn/4`. ERTS sends a `SPAWN_REQUEST` control message to B: `{29, ReqId, From, GroupLeader, {Module, Function, Arity}, OptList}` followed by `ArgList` (OTP 23 and later).
2. B handles the request and creates a process that calls `apply(Module, Function, ArgList)`.
3. B sends a `SPAWN_REPLY` control message back: `{31, ReqId, To, Flags, Result}`. `Flags` is an integer bit field. The value 1 indicates that a link was established between `To` and the new process; 2 indicates a monitor from `To` to the new process. The flags report the link and monitor established on the new process's node.
4. A handles the reply and returns the new PID to the caller. On failure, `Result` carries an error reason instead of a PID; the calling API determines how the caller receives that failure.

Native Implementation: Runtime Extensions and Main-Thread Integration

Optional implementation detail accompanying the developer-facing chapter. The source extracts and version qualifications below describe the second-edition baseline.

16.1. Borrowing the macOS main thread

16.1.1. Borrowing the Main Thread: `erl_drv_steal_main_thread`

Some C libraries insist on running on the thread that started the program. The best known case is Cocoa on macOS: `NSApplication` and everything that draws a window must be driven from the main thread of the process, and a thread created with `erl_drv_thread_create` or `enif_thread_create` will not do. The runtime has a way around this, but you will not find it in the `erl_driver` manual.

On startup the main thread of the emulator starts the schedulers and then calls `erts_sys_main_thread()` in `erl_init.c`, a call the source annotates with "May or may not return!". On most Unix systems that function turns the main thread into the signal receiver thread. On macOS it instead creates two pipes and parks the main thread in a `select` loop on the first of them, waiting for work (see `erts/emulator/sys/unix/sys.c`). The work arrives through this pair of functions, declared in `erl_driver.h` only when `DARWIN` is defined:

```
int erl_drv_steal_main_thread(char *name,
                              ErlDrvTid *dtid,
                              void* (*func)(void*),
                              void* arg,
                              ErlDrvThreadOpts *opts);
int erl_drv_stolen_main_thread_join(ErlDrvTid tid, void **respp);
```

The signature mirrors `erl_drv_thread_create`, but the implementation in `erl_drv_thread.c` is a few lines long. It ignores `name` and `opts`, sets `*dtid` to `NULL`, and writes

the function pointer and its argument into the pipe. The parked main thread reads them, calls `func(arg)`, and writes the return value into the second pipe, where `erl_drv_stolen_main_thread_join` picks it up. Your function is now running on the main thread of the process while the runtime carries on undisturbed on the scheduler threads.

This is how `wx` works on macOS. In `lib/wx/c_src/wxe_main.cpp` the library starts its event loop with `erl_drv_steal_main_thread("wxwidgets", &wxe_thread, wxe_main_loop, NULL, NULL)`, where every other platform gets a `enif_thread_create`. The `wx` event loop returns only when `wx` is stopped, so while `wx` runs the main thread is occupied. This is also why only one library per node can use the mechanism: a second caller would sit in the pipe until the first function returned.

Nothing about the mechanism is tied to drivers. `wx` is a NIF library and declares the two prototypes itself, which works because `ErlNifTid` and `ErlDrvTid` are the same pointer type. A NIF that has to run Cocoa code can do the same: steal the main thread once when the library loads, run the native event loop there, and hand it work through whatever queue the library provides.

Before you build on it, know what you are getting. The two functions are not mentioned in the `erl_driver` or `erl_nif` documentation, no test outside `wx` exercises them, and the OTP team is free to change or remove them without a release note. If you need them, the durable fix is a pull request to Erlang/OTP that adds documentation and a test.



Version Info

The two functions were added in Erlang/OTP R14B02 (erts 5.8.3) to make `wx` work with Cocoa on macOS. Up to Erlang/OTP 28 the functions had no public prototype at all. Erlang/OTP 29.0 declares them in `erl_driver.h`, still only for macOS and still undocumented.

16.2. Implementing and building a yielding factorial BIF

Most Erlang users will never need to write a BIF, but it is good to know what they are and how they work. Also if you are writing an EEP (Erlang Enhancement Proposal) you might need to write a BIF to implement the new functionality.

The following sections will go through some of the details of how to implement a BIF.

16.2.1. Implementation Steps

1. Creating a BIF:

- Write the function in C, adhering to the BEAM's internal API for memory and process safety.
- Implement a thread-safe, efficient algorithm for the function's purpose.
- Add the function to the BEAM source code and compile it with the runtime system.
- Calculate the number of reductions the BIF should use in relation to the time complexity of the function.
- If the function can take a long time consider adding the possibility to yield.

2. Performance Considerations:

- Avoid long-running operations within the BIF.
- Ensure memory safety and proper handling of Erlang terms to maintain system stability.

16.2.2. A Checked, Yielding Factorial BIF

The example adds `math:factorial/1` to a private OTP 29 build. This is a book extension, not a function supplied by OTP. Its accepted input range is 0 through 10,000; other values raise `badarg`. The bound limits both temporary storage and the size of any single multiplication.

A C integer accumulator alone is insufficient: $21!$ exceeds a signed 64-bit integer, and a 32-bit integer overflows earlier. The implementation uses checked small-integer multiplication until the result needs a bignum, then calls OTP 29's `small_times` and `big_times` helpers.

```
/* Include from erl_math.c in a private OTP 29 source tree.
 * This example adds math:factorial/1; it is not an OTP API.
 */
#define BOOK_FACTORIAL_MAX 10000
#define BOOK_FACTORIAL_WORD_LIMIT 8192
#define BOOK_FACTORIAL_SLICE_WORK 4096

static Export book_factorial_continue_export;
void erts_init_book_factorial(void);

static BIF_RETTYTYPE book_factorial_continue(BIF_ALIST_3)
{
    Uint n = unsigned_val(BIF_ARG_1);
    Uint i = unsigned_val(BIF_ARG_2);
    Eterm acc = BIF_ARG_3;
    Eterm *src = NULL, *dst = NULL, *tmp, *hp;
```

```

Eterm multiplier[2];
Uint capacity = 8, need, words, work = 0;

/* Only this BIF constructs the continuation arguments. Copy a boxed
 * accumulator before changing buffers; it currently belongs to the
process.
 */
if (is_big(acc)) {
    words = BIG_SIZE(big_val(acc)) + 1;
    while (capacity < words + 1) capacity *= 2;
    ASSERT(capacity <= BOOK_FACTORIAL_WORD_LIMIT);
    src = erts_alloc(ERTS_ALC_T_TMP, capacity * sizeof(Eterm));
    dst = erts_alloc(ERTS_ALC_T_TMP, capacity * sizeof(Eterm));
    sys_memcpy(src, big_val(acc), words * sizeof(Eterm));
    acc = make_big(src);
}

while (i <= n && work < BOOK_FACTORIAL_SLICE_WORK) {
    if (is_small(acc) && unsigned_val(acc) <= (Uint)MAX_SMALL / i) {
        acc = make_small(unsigned_val(acc) * i);
        ++work;
    } else {
        need = is_big(acc) ? BIG_SIZE(big_val(acc)) + 2 : 3;
        if (src == NULL) {
            src = erts_alloc(ERTS_ALC_T_TMP, capacity * sizeof(Eterm));
            dst = erts_alloc(ERTS_ALC_T_TMP, capacity * sizeof(Eterm));
        }
        if (need > capacity) {
            /* The input bound keeps this doubling and byte count
bounded.

            * src and dst always own the current allocations, including
            * after a realloc or a swap. Rebuild the tagged alias too.
            */
            capacity *= 2;
            ASSERT(capacity <= BOOK_FACTORIAL_WORD_LIMIT);
            src = erts_realloc(ERTS_ALC_T_TMP, src,
                             capacity * sizeof(Eterm));
            dst = erts_realloc(ERTS_ALC_T_TMP, dst,
                             capacity * sizeof(Eterm));
            acc = make_big(src);
        }
        work += need;
        if (is_small(acc))
            acc = small_times(signed_val(acc), (Sint)i, dst);
        else
            acc = big_times(acc, small_to_big((Sint)i, multiplier), dst);
        ASSERT(is_big(acc));
        tmp = src; src = dst; dst = tmp;
    }
}

```

```

    ++i;
}

/* No native buffer survives a return or trap. HALloc does not run GC;
 * all term construction finishes before the continuation can be
 * scheduled.
 */
if (is_big(acc)) {
    words = BIG_SIZE(big_val(acc)) + 1;
    hp = HALloc(BIF_P, words);
    sys_memcpy(hp, big_val(acc), words * sizeof(Eterm));
    acc = make_big(hp);
}
if (src != NULL) {
    erts_free(ERTS_ALC_T_TMP, src);
    erts_free(ERTS_ALC_T_TMP, dst);
}
if (i <= n) {
    BUMP_ALL_REDS(BIF_P);
    BIF_TRAP3(&book_factorial_continue_export, BIF_P,
              make_small(n), make_small(i), acc);
}
BIF_RET2(acc, 1 + work / 32);
}

BIF_RETTYPER math_factorial_1(BIF_ALIST_1)
{
    Eterm args[3];
    if (!is_small(BIF_ARG_1) || signed_val(BIF_ARG_1) < 0 ||
        signed_val(BIF_ARG_1) > BOOK_FACTORIAL_MAX)
        BIF_ERROR(BIF_P, BADARG);
    args[0] = BIF_ARG_1;
    args[1] = make_small(2);
    args[2] = make_small(1);
    return book_factorial_continue(BIF_P, args, A_I);
}

void erts_init_book_factorial(void)
{
    erts_init_trap_export(&book_factorial_continue_export,
                        am_math, am_book_factorial_continue, 3,
                        &book_factorial_continue);
}

```

Follow `acc` through one multiplication. While the result fits a small integer, it stays in the term itself and we need no temporary buffers. Once it becomes a bignum, `src` owns the

digits we read and `dst` owns the space where we write the product. Swapping those pointers makes the product the input for the next multiplication.

A growing product eventually needs more room. Reallocation can move `src`, so we rebuild the boxed term from its new address before using it again. The two owning pointers always refer to the current allocations, including after a swap. If the internal allocator runs out of native memory, its allocation-failure path terminates the runtime; control does not return to this BIF.

Now follow the path that runs out of work for this slice. Before trapping, we copy the boxed accumulator into process memory and free both temporary buffers, just as we do before returning a final result. The continuation carries the original bound, the next multiplier and the accumulated factorial as three ordinary Erlang terms. Those terms are all we need to resume; no native buffer remains allocated across the trap. Only the entry BIF accepts user input, and its range check establishes the small-integer arguments that the continuation decodes.

Notice that `work` increases with the digit count. As the factorial grows, fewer multiplications fit in a slice: the loop stops once the work reaches 4,096. Its last multiplication can cross that budget, with the input cap limiting the size of that operation. `BUMP_ALL_REDS` then exhausts the process's reduction budget before the trap, giving other processes a turn. We have bounded the algorithmic work between traps; the elapsed time still depends on the machine and its operating-system scheduling.

Building the Extension

Use a disposable OTP 29.0 source tree. The installation script copies the included implementation into `erts/emulator/beam/book_factorial.c`, includes it from `erl_math.c`, registers the BIF in `bif.tab`, adds its continuation atom, initializes the trap export after BIF initialization, and exports a stub from `math.erl`. The stub and range are local additions, without an upstream release annotation.

```
python3 code/c_chapter/test/install_factorial.py \
/path/to/private/otp_src_29.0
cd /path/to/private/otp_src_29.0
./configure --without-javac --without-wx --without-odbc
make -j4
bin/erl
```

The build must use the modified emulator and matching `math` module. Loading the stub into an unmodified runtime does not install a BIF.

```

1> math:factorial(0).
1
2> math:factorial(7).
5040
3> length(integer_to_list(math:factorial(1000))).
2568
4> math:factorial(10001).
** exception error: bad argument

```

The accompanying checks compare every result from 0 through 100 with an Erlang implementation, then exercise larger bignums, repeated calls, invalid inputs, buffer growth, and process scheduling during traps. Run the following command from the book checkout, with the private runtime; the ordinary book example checker cannot validate a modified emulator.

```

ESCRIPTEMULATOR=/path/to/private/otp_src_29.0/bin/erl \
ERL_FLAGS="+S 1" escript code/c_chapter/test/factorial_checks.escript

```

Shell Setup and Connection Exercises

Optional companion material retained from the second-edition source. Return to the corresponding chapter for the diagnostic workflow.

17.1. Connecting three nodes

In the distributed case this is somewhat more complicated since we need to make sure that all nodes know or share the cookie. This can be done in three ways. You can set the cookie used when talking to a specific node, you can set the same cookie for all systems at start up with the `-setcookie` parameter, or you can copy the file `.erlang.cookie` to the home directory of the user running the system on each machine.

The last alternative, to have the same cookie in the cookie file of each machine in the system is usually the best option since it makes it easy to connect to the nodes from a local OS shell. Just set up some secure way of logging in to the machine either through VPN or ssh.

Let's create a node on a separate machine and connect it to the nodes we have running. We start a third terminal window, check the contents of our local cookie file, and ssh to the other machine:

```
happi@GDC08:~$ cat ~/.erlang.cookie
pepparkaka
happi@GDC08:~$ ssh gds01
happi@gds01:~$
```

We launch a node on the remote machine, telling it to use the same cookie passphrase, and then we are able to connect to our existing nodes. (Note that we have to specify `node1@GDC08` so Erlang knows where to find these nodes.)

```

happi@gds01:~$ erl -sname node3 -setcookie pepparkaka

(node3@gds01)1> net_kernel:connect_node('node1@GDC08').
true
(node3@gds01)2> nodes().
[node1@GDC08,node2@GDC08]
(node3@gds01)3>

```

Even though we did not explicitly talk to node 2, it has automatically been informed that node 3 has joined, as we can see:

```

iex(node2@GDC08)3> Node.list
[:node1@GDC08,:node3@gds01]
iex(node2@GDC08)4>

```

In the same way, if we terminate one of the nodes, the remaining nodes will remove that node from their lists automatically. If the node gets restarted, it can simply rejoin the network again.

17.1.1. Configuring Your Shell

Both the Elixir shell and the Erlang shell can be configured to provide you with shortcuts for functions that you often use.

The Elixir shell will look for the file `.iex.exs` first in the local directory and then in the user's home directory. The code in this file is executed in the shell process and all variable bindings will be available in the shell.

In this file you can configure aspects such as the syntax coloring and the size of the history. See the IEx configuration documentation[\[S1\]](#).

You can also execute arbitrary code in the shell context.

When the shell starts, it first evaluates the code in the Erlang configuration file, as if it were typed into the shell. The default location of this file is the user's home directory, `~/.erlang`, with `.erlang` under `filename:basedir(user_config, "erlang")` as the second place searched. It can contain any Erlang expressions, each terminated by a dot and a newline.

This file is typically used to add directories to the code path for loading Erlang modules:

```

code:add_path("/home/happi/hacks/ebin").

```

as well as to load the custom `user_default` module (a `.beam` file) which you can use to extend the Erlang shell with user-defined functions:

```
code:load_abs("/home/happi/.config/erlang/user_default").
```

Just replace the paths above to match your own system. Do not include the `.beam` extension in the `load_abs` command.

If you call a function from the shell without specifying a module name, for example `foo(bar)`, it will try to find the function first in the module `user_default` (if it exists) and then in the module `shell_default` (which is part of `stdlib`). This is how shell commands such as `ls()` and `help()` are implemented, and you are free to add your own or override the existing ones.

17.2. Shell configuration exercises

Let us first add one command to the Erlang shell. From the repository root, compile the supplied module into the directory our startup file will load:

```
mkdir -p "$HOME/.config/erlang"
erlc -o "$HOME/.config/erlang" code/ops_chapter/src/user_default.erl
```

Add the following expressions to `~/.erlang`, keeping any settings you already use:

```
%% Loaded when the shell starts. Load a user_default module from a
%% per-user directory and say where the node was started.
code:load_abs(filename:join([os:getenv("HOME"), ".config", "erlang",
"user_default"])).

io:format("ERTS is starting in ~s~n", [os:cmd("pwd")]).
```

And the `user_default.erl` file looks like this:

```
-module(user_default).

-export([tt/0]).

tt() ->
    test.
```

Start a fresh shell with `erl -sname shell_demo`. After the startup message with your working directory, try the new command:

```
(shell_demo@host)1> tt().
test
```

This result was checked on OTP 29; host stands for your host name. Leave this shell with `q()`. before the next exercise.

The following IEx example is adapted from the first-edition transcript. We use fresh node names `cfg1` through `cfg4` so earlier nodes can stay running; banners and redundant startup lines have been omitted. Create two directories `~/example/dir1` and `~/example/dir2` and put the following `.iex.exs` files in them. They set the prompt to show `<d1>` and `<d2>`, respectively.

```
# File 1
IO.puts "iEx starting in "
pwd()
IO.puts "iEx starting on "
IO.puts Node.self

IEx.configure(
  colors: [enabled: true],
  alive_prompt: [
    "\e[G",
    "(%node)",
    "%prefix",
    "<d1>",
  ] |> IO.ANSI.format |> IO.chardata_to_string
)
```

```
# File 2
IO.puts "iEx starting in "
pwd()
IO.puts "iEx starting on "
IO.puts Node.self

IEx.configure(
  colors: [enabled: true],
  alive_prompt: [
    "\e[G",
    "(%node)",
    "%prefix",
    "<d2>",
  ] |> IO.ANSI.format |> IO.chardata_to_string
)
```

Now if we start four different nodes from these directories we will see how the shell configurations are loaded. First `cfg1` in `dir1`:

```
GDC08:~/example/dir1$ iex --sname cfg1

ERTS is starting in /home/happi/example/dir1
iEx starting in
/home/happi/example/dir1
iEx starting on
cfg1@GDC08
(cfg1@GDC08)iex<d1>
```

Then `cfg2` in `dir2`:

```
GDC08:~/example/dir2$ iex --sname cfg2

ERTS is starting in /home/happi/example/dir2
iEx starting in
/home/happi/example/dir2
iEx starting on
cfg2@GDC08
(cfg2@GDC08)iex<d2>
```

Then `cfg3` in `dir1`, but launching a remote shell on `cfg2`:

```
GDC08:~/example/dir1$ iex --sname cfg3 --remsh cfg2@GDC08

ERTS is starting in /home/happi/example/dir1
iEx starting in
/home/happi/example/dir2
iEx starting on
cfg2@GDC08
(cfg2@GDC08)iex<d2>
```

As we see, the remote shell started up in `dir2`, since that is the directory of `cfg2`. Finally, we start an Erlang node and check that the function we defined in our `user_default` module can be called from the shell:

```
GDC08:~/example/dir2$ erl -sname cfg4

ERTS is starting in /home/happi/example/dir2
(cfg4@GDC08)1> tt().
test
(cfg4@GDC08)2>
```

These shell configurations are loaded from the node running the shell, as you can see from the above examples. If we were to connect to a node on a different machine, these configurations would not be present.

Passing the `-remsh` flag at startup is not the only way to launch a remote shell. You can actually change which node and shell you are connected to on the fly by going into job control mode.

17.2.1. Job Control Mode

By pressing `ctrl+g` you enter the job control mode (JCL). You are then greeted by another prompt:

```
User switch command (enter 'h' for help)
-->
```

By typing `h` (followed by `enter`) you get a help text with the available commands in JCL:

```
c [nn]           - connect to job
i [nn]           - interrupt job
k [nn]           - kill job
j               - list all jobs
s [shell]        - start local shell
r [node [shell]] - start remote shell
q               - quit erlang
? | h           - this message
```

The interesting command here is the `r` command which starts a remote shell. You can give it the name of the shell you want to run, which is needed if you want to start an Elixir shell, since the default is the standard Erlang shell. Once you have started a new job (i.e., a new shell) you need to connect to that job with the `c` command. You can also list all jobs with `j`.

```
(cfg2@GDC08)ix<d2>
User switch command (enter 'h' for help)
--> r cfg1@GDC08 'Elixir.IEx'
--> c
iEx starting in
/home/happi/example/dir1
iEx starting on
cfg1@GDC08
```

Starting a new local shell with `s` and connecting to that instead is useful if you have started a very long-running command and want to do something else in the meantime. If your

command seems to be stuck, you can interrupt it with `i`, or you can kill that shell with `k` and start a fresh one.

See the Erlang Shell manual[S2] for a full description of JCL mode.

You can quit your session by typing `ctrl+g q [enter]`. This shuts down the local node.



Warning

You do **not** want to quit with any of `q()`, `halt()`, `init:stop()`, or `System.halt`. All of these will bring down the remote node which seldom is what you want when you have connected to a live server. Instead use `ctrl+\, ctrl+c ctrl+c, ctrl+g q [enter]` or `ctrl+c a [enter]`.

If you do not want to use a remote shell, which requires you to have two instances of ERTS running, there are actually two other ways to connect to a node. You can also connect either through a Unix pipe or directly through `ssh`, but both of these methods require that you have prepared the node you want to connect to by starting it in a special way or by starting an `ssh` server.

17.3. BREAK build and signal options

If you have built your runtime with `OPPROF` or `DEBUG` you will be able to get even more information. We will look at how to do this in [Building ERTS](#). The code that implements the break mode can be found in `erts/emulator/beam/break.c`[S3].



Tip

If you prefer that the node shuts down immediately on `ctrl+c` instead of bringing up the BREAK menu, you can pass the option `+Bd` to `erl`. This is typically what you want for running Erlang in Docker or similar. There is also the variant `+Bc` which makes `ctrl-c` just interrupt the current shell command, this can be nice for interactive work.

Sources and links

[S1] Elixir Project. IEx. The `.iex.exs` file. <https://hexdocs.pm/iex/IEx.html#module-the-iex-exs-file>

[S2] Erlang/OTP Project. shell. Erlang/OTP 29 documentation.
<https://www.erlang.org/docs/29/apps/stdlib/shell.html>

[S3] Erlang/OTP Project. Erlang/OTP source code: `break.c`. OTP-29.0; `erts/emulator/beam/break.c`.
<https://github.com/erlang/otp>

Application Debugging Background

Optional companion material retained from the second-edition source. Return to the corresponding chapter for the diagnostic workflow.

The callback and logging fragments in this section are schematic; they require the surrounding application module and its state.

18.1. Application debugging background

Start with the symptom you can observe: a growing queue, a failed call, or a node that stopped. Keep the input and the runtime version with your notes, so you can compare the same observation after a change.

18.1.1. Systematic Approaches to Debugging

Choose the next inspection from what is still unknown. If the process is alive, inspect it; if the node has stopped, begin with the reports it left. The five steps below run from a symptom you can repeat to a fix you can trust.

Reproduce the Problem

A bug you cannot reproduce is a bug you cannot fix or confirm fixed. Turn up the logging (logger), narrow a trace to the suspect calls with `dbg` or `recon`, and drive the failing path from a controlled node. For a trace scoped to one process and one argument, with cleanup, follow [Function Tracing with dbg](#). A fault that appears only under load often hides a race; reach for it under `PropEr` or `Common Test`.

Isolate the Faulty Component

Narrow the failure to one process and one function before changing anything. Sample the queue length, then sample it again to see whether it grows:

```
process_info(Pid, message_queue_len).
```

The walkthrough in [Start with the symptom](#) reads queue length against status and reductions to tell a waiting process from a slow one. Table size and node memory place the cost:

```
ets:info(my_table, size).
erlang:memory().
```

Those figures choose the process and function to trace in [Function Tracing with dbg](#).

Analyze Logs and Crash Dumps

A process crash report and an ERTS text crash dump are different things. A process can fail while the node runs on, leaving a report but no dump. ERTS writes a text dump only on certain fatal terminations, and only when its settings and the failure leave it able to. Follow [Producing a Text Crash Dump](#) to produce a deliberate test dump and [Reading the Dump](#) to read it, including what to check when no dump appears.

Use the Right Tool

Match the tool to the question. Observer (`observer:start()`) watches processes as they run, `dbg` and `recon` trace and inspect at a lower level, and `SystemTap` or `DTrace` reach into the kernel when the answer is below the runtime. [Debugging Tools and Techniques](#) works through the first two on a live node.

Verify the Fix

A fix is not done until a test would catch its return. Capture the failing input in a Common Test or EUnit case, cover the edges with a property test under `PropEr` or `QuickCheck`, and run it against a staging node before it reaches production.

18.1.2. Learning from Mistakes and Improving Code Quality

This is an optional application-development branch. To continue inspecting the runtime, go to [The Usual Suspects: Common Sources of Bugs](#); return here when you want to turn the diagnosis into changes to the application.

A fixed bug is worth more than the time it took if it changes the code around it. The habits below come out of that.

Conducting Post-Mortems

After a serious bug, write down its root cause, its effect on the system, and the change that stops the class of bug, not just this instance. When a process died on a message it did not expect, the fix is often a catch-all clause that logs and continues rather than crashing:

```
handle_info(Unexpected, State) ->
    logger:warning("unexpected message: ~p", [Unexpected]),
    {noreply, State}.
```

Improving Logging and Observability

You can only debug what you can see. Structured logging through `logger`, with levels, gives you that record:

```
logger:log(info, "user logged in: ~p", [UserId]).
```

and a periodic check such as `recon:bin_leak(10)` surfaces a slow leak before it becomes an outage rather than after.

Enhancing Code Readability and Maintainability

Readable code is debuggable code. Name a function for what it does, `handle_request/1` rather than `do_it/1`, keep to the OTP behaviours, and let pattern matching split the cases a case would otherwise nest:

```
process_request({ok, Data}) -> process(Data);
process_request({error, _}) -> handle_error().
```

Each clause is now a place a trace or a breakpoint can land.

Implementing Fail-Fast Mechanisms

A process that crashes on bad input tells you where the fault is; one that carries invalid state hides it. Guard the entry and let the rest of the system restart the process:

```
handle_request({ok, Data}) when is_list(Data) ->
    process(Data);
handle_request(_) ->
    exit(bad_request).
```

Learning from Open Source Erlang Systems

Production Erlang is largely open source, and its diagnostic code repays reading. RabbitMQ leans on structured logging and monitoring, MongooseIM on tracing, and Riak on distributed fault recovery; each shows the technique at a scale a small example cannot.

The spawn examples below are schematic; `my_function/0` and `handle_large_binaries/0` stand for application functions.

18.2. Heap-setting background

A few system flags control how a process allocates memory. `min_heap_size` sets the heap a process starts with. The default is 233 words; raising it to 256 or 512 for a process you know will grow saves the first few expansions at the cost of memory in every process it applies to. Set it per process:

```
spawn_opt(fun() -> my_function() end, [{min_heap_size, 512}]).
```

or for every new process at startup:

```
erl +hms 512
```

Each process then starts with at least 512 words, which trades a little memory per process for fewer early heap expansions.

`min_bin_vheap_size` (Minimum Binary Virtual Heap Size)

This setting controls the minimum virtual-heap threshold used by garbage collection to account for reference-counted binaries held by a process. The virtual heap is an accounting mechanism; it does not reserve binary payload storage or preallocate an off-heap arena. The default is 46,422 words.

For example, the following requests a larger minimum threshold:

```
spawn_opt(fun() -> handle_large_binaries() end, [{min_bin_vheap_size, 100000}]).
```

A larger threshold can reduce collection frequency while allowing unused binary references to retain payload memory longer. A smaller threshold can trigger collections sooner; 512 words would be substantially below the default. Compare collection counts, retained binary memory, and latency under representative load before changing this setting.

The `fullsweep_after` flag controls how many generational collections pass before a full sweep reclaims old-generation garbage; a process that holds binaries benefits from a lower value. Keep ETS tables trimmed of entries you no longer read, and reach for `set` before `bag` or `ordered_set` unless the semantics require them. And where a large term is long-lived and shared, send a reference to it, in ETS or a database, rather than copying the term into each recipient's heap.

18.3. Exceptions and supervision background

The following background on exceptions and supervision is optional if you already use OTP applications. Continue at [Debugging Tools and Techniques](#) for the tracing and breakpoint exercises.

Erlang's *Let It Crash* approach turns error handling on its head. Rather than guard every call against every fault, a process does the straightforward thing and dies when its assumptions break, and a supervisor restarts it from a known state. The code that remains is the code that does the work.

18.3.1. Overview and Rationale

Defensive code anticipates failures with `try-catch` around every risky call, and grows harder to read with each case it adds. Erlang accepts that failures happen and puts the recovery somewhere else, in the supervision tree, so the worker keeps only its business logic. Four properties make that trade pay off:

- Isolation: separate heaps keep one process's memory to itself, though an untrapped exit still reaches linked processes, and a supervisor may restart siblings by its strategy.
- Recovery: a supervisor monitors its children and restarts a failed one.
- Simpler code: the worker holds the logic, not the error handling.
- Containment: a crash and restart stop a fault where it happened instead of spreading it.

18.3.2. Exceptions in Erlang

Erlang can handle an exception where it happens, but it steers you toward letting the process end and restart under supervision instead.

Types of Exceptions

Erlang raises three kinds of exception:

- Errors (`error:Reason`), from a serious fault such as a division by zero or a call to an undefined function.
- Throws (`throw:Reason`), for a non-local return the program chose.
- Exits (`exit:Reason`), when a process ends, by fault or on purpose.

Example of Exception Handling

You can catch one where local handling is genuinely required:

```
try 1 / 0 of
  Result -> io:format("Result: ~p~n", [Result])
catch
  error:badarith -> io:format("cannot divide by zero~n")
end.
```

Most faults, though, are left to crash and reach a supervisor.

Process Exits and Monitoring

A crash sends an exit signal to every linked process. Monitor one to be told of its exit without dying with it:

```
spawn_monitor(fun() -> exit(died) end).
```

The monitoring process receives a 'DOWN' message and decides what to do.

18.3.3. Designing Systems with Supervisor Trees

A supervision tree puts the recovery in one place: supervisors watch workers and other supervisors, and restart a child that fails. Armstrong sets out the split between the two in his thesis, sections 4.3.1 and 4.3.2 [\[Armstrong2003\]](#). A supervisor manages the workers and any

supervisors beneath it; a worker does the computation and, when it crashes, leaves the restart decision to its supervisor.

```
-module(my_supervisor).
-behaviour(supervisor).

-export([start_link/0, init/1]).

start_link() ->
    supervisor:start_link(?MODULE, []).

init([]) ->
    {ok, [{strategy => one_for_one, intensity => 3, period => 10},
        [{id => worker1, start => {my_worker, start_link, []},
          restart => permanent, shutdown => 5000, type => worker,
          modules => [my_worker]}]}]}.
```

The `one_for_one` strategy above restarts `my_worker` on its own when it crashes, up to three times in ten seconds before the supervisor itself gives up.

Supervision Strategies

The strategy decides who else restarts when one child fails. `one_for_one` restarts only the child that crashed, and is the common case. `one_for_all` restarts every child, for a group that shares state. `rest_for_one` restarts the failed child and those started after it, when later children depend on earlier ones. `simple_one_for_one` manages a dynamic set of identical workers.

The supervisor module compiles on its own, but running it requires an application module `my_worker` exporting `start_link/0`.

BeamAsm Profiling Metadata Options

The `+JPperf` flag accepts finer-grained values when you do not need all of that at once:

Table 9. `+JPperf` sub-options

Value	Effect
<code>true</code>	Enables map, dump, and frame pointers (recommended default)
<code>map</code>	Perf map file and frame pointers; no jitdump
<code>dump</code>	Jitdump file for annotation; no perf map
<code>fp</code>	Frame pointers only, for accurate stack traces in other tools; no perf files
<code>no_fp</code>	Disables the frame pointers that <code>map</code> adds, saving their overhead at the cost of call graphs
<code>false</code>	Disables all perf support (the default)

Additional Runtime Settings

This optional companion preserves the fuller settings discussion from the second-edition source. It is a reference for investigating a measured runtime issue. The main tuning chapter supplies the symptom, measurement and trade-off needed before adopting a setting.

20.1. Memory tuning introduction

There are several ways to tweak the memory subsystem depending on what your goals are. If you have limited hardware or want to run many instances on the same hardware you might want to reduce the memory footprint. If you have a high speed transaction system you might want to tune the memory usage for speed. If your application has a very large memory footprint you might want to optimize for that.

I will show you some basic strategies and settings for each of these scenarios but only you know the needs of your application. The best strategy is to read up on how the VM handles memory (see [The Memory Subsystem: Stacks, Heaps and Allocators](#)) and on the flags you can set, and then experiment to find the settings that suit you.

20.2. Small-footprint settings and representation details

There is usually a space versus time trade off. Most algorithms can either use more space and less time or less space and more time, and optimizing for a small footprint will often cost performance. This is not always true. A node that uses less memory can start faster, swap less, and fit more of its working set in the caches, so a smaller footprint sometimes pays for itself.

A just started node spends most of its memory on things you can shrink from the command line. Every scheduler thread, dirty scheduler and poller thread has a stack and its own instances of the allocators, so on a small machine start with `+S 1, +SDcpu 1:1` and `+SDio 1`.

The process table is allocated in full at start: `+P` sets the maximum number of processes, 1,048,576 by default, and the table costs two words per slot, the entry and its free-list entry, so the default table is 16 MiB on a 64-bit system before a single process exists. `+P 32768` is plenty for many small nodes.

The port table limit (`+Q`) also affects startup allocation. The atom table has a different layout: `+t` sizes its segment-pointer structure, while atom index segments grow on demand. Lowering the atom limit does not recover a fully populated table of atom entries.

The allocators themselves reserve memory in carriers before it is needed. Each allocator can be disabled with `+M<S>e false`, in which case its requests go straight to `sys_alloc` and thus to `malloc`, and `+Mea min` disables every allocator that can be disabled at once. This saves the pre-allocated carriers and the per scheduler instances at the cost of the locality and the reduced lock contention that the allocators exist to provide, which is a good trade on a node with one scheduler and a bad one on a node with sixty four.

The literal allocator reserves 1 GB of virtual address space by default (`+MIscs`, in MB); that is address space rather than memory and shows up in the virtual size only, so leave it unless a 32-bit build or a tight virtual memory limit forces the issue.

On the Erlang side, the default initial heap is 233 words (`+hms`) and the nominal binary virtual-heap threshold is 32,768 words (`+hmbs`). OTP 29 rounds that request to the supported heap size of 46,422 words, which `process_info(Pid, garbage_collection)` reports. This threshold controls binary-memory accounting; it does not reserve a binary heap. Lowering either setting can increase collections in young processes.

The larger lever is to hibernate processes that spend most of their time waiting, described under large heaps below, since a hibernated process keeps only its live data.

A 32-bit emulator uses four-byte words for pointers and word-sized fields. Fixed-size payloads and representation changes mean that whole terms need not halve in size. Its address space is at most 4 GiB, with less available to the application under typical OS layouts. A toolchain with the required 32-bit libraries can use `--enable-m32-build` (see [Building ERTS](#)).



Version Info

Building without SMP support or without HiPE is no longer an option: the non-SMP emulator was removed in Erlang/OTP 21.0 and HiPE in Erlang/OTP 24.0.

20.3. Super carriers and locked mappings

`+MMscs Size` (in MB) reserves a super carrier, a contiguous address region used by `mseg_alloc`. With `+MMsco true`, its default, that allocator cannot obtain carriers outside the region. Adding `+Musac false` also disables `sys_alloc` carriers in the allocation framework.

These restrictions apply to carriers, not the node's RSS: other native allocations and mappings can remain outside the region. Measure the whole process and keep headroom under a container memory limit. Exhaustion can fail an allocation; these flags do not guarantee recovery or prevent an OS OOM kill. `+Mlpm all` additionally requests that the affected mappings be locked into physical memory.

20.4. Scheduler wakeup threshold

`+swt` sets the wakeup threshold, the point at which a sleeping scheduler is woken because the awake ones have more work than they can handle, from `very_low` to `very_high` with `medium` as default; a low threshold spreads work faster and bounces it between schedulers more.

20.5. Scheduler binding type catalogue

The types spread the schedulers in different ways: `ns` keeps schedulers with close identifiers close, `ts` and `ps` spread them over hardware threads and processors, `s` spreads as much as possible, the `nn` variants keep NUMA nodes together, and `db` is the default binding, which is `tnnps`.

20.6. Native-code queue-scan workaround

`+sfwi Interval` makes the runtime scan all run queues every `Interval` milliseconds and wake a scheduler for each non-empty one. The documentation calls it a temporary workaround for native code that does not yield. Use `{long_schedule, Ms}` in the system monitor to identify the long-running process or port, then inspect its native work. A long NIF may need dirty CPU or dirty IO scheduling, according to the work it does. A linked-in driver can hand work to `driver_async` or its own threads. [Interfacing Other Languages and Extending BEAM and ERTS](#) explains these separate interfaces.

20.7. Time correction compatibility settings

Time is the last group. `+c false` disables time correction and `+C` selects the time warp mode; `multi_time_warp` has been the default since Erlang/OTP 26.0 and is the right mode for any code that uses `erlang:monotonic_time/0` for timing, which is all of OTP. The flag mostly matters when an old application relies on `erlang:now/0` semantics, and the cure there is the application.

Appendix A: Third-party Notices

A.1. Erlang/OTP Excerpts

The Erlang/OTP source code and documentation excerpts in this book are © Ericsson AB and contributors and are used under the Apache License, Version 2.0.

The Apache License 2.0 applies only to the identified Erlang/OTP excerpts, including adaptations of those excerpts. It does not apply to the book as a whole or change the licensing of other material.

The source file and release are identified beside each excerpt or in its source note. The instruction descriptions in [BEAM Instructions](#) are extracted from OTP 29's `lib/compiler/src/genop.tab`.

The book edits these extracts for teaching: surrounding code and comments are omitted, formatting is changed, and some examples add annotations or show a simplified sequence. Local explanations identify simplifications; the linked source contains the complete implementation. These editorial changes are by Erik Stenman. The excerpts retain their original licence.

A.2. Source Copyright Notices

The source files below carry the notice “Copyright Ericsson AB [years]. All Rights Reserved.” The table preserves each notice’s year range. All identify their licence as Apache-2.0. Paths are relative to `erts/emulator/beam/` unless they start with `erts/`, `lib/` or `system/`. They refer to the OTP 29.0 source tree.[\[S1\]](#)

Years	Source files
1996-2025	<code>erl_process.h</code> , <code>atom.h</code>
1996-2026	<code>emu/beam_emu.c</code>
1997-2025	<code>erl_message.h</code>
1997-2026	<code>emu/ops.tab</code> , <code>jit/x86/ops.tab</code> , <code>jit/arm/ops.tab</code>
1998-2025	<code>erl_db_hash.c</code>
1998-2026	<code>erts/emulator/utils/beam_makeops</code> , <code>lib/compiler/src/genop.tab</code>
1999-2025	<code>erl_driver.h</code>

Years	Source files
2000-2026	erl_fun.h
2006-2025	erl_time.h
2017-2025	emu/arith_instrs.tab,emu/macros.tab
2017-2026	emu/instrs.tab
2020-2026	predicates.tab,generators.tab,jit/asm_load.c,jit/beam_jit_main.cpp, jit/x86/instr_common.cpp,jit/x86/beam_asm.hpp
2021-2026	jit/x86/instr_fun.cpp
2023-2025	system/doc/tutorial/c_portdriver.md,system/doc/tutorial/example.md
2024-2026	erl_record.h

A.3. Apache License, Version 2.0

The complete Apache License 2.0 follows. Its wording is unchanged; line wrapping and paragraph layout have been adjusted for the book. A plain-text copy is also supplied as LICENSE-APACHE-2.0.txt in the book's repository. The licence is published by the Apache Software Foundation.[S2]

Apache License Version 2.0, January 2004 <http://www.apache.org/licenses/>

TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

"License" shall mean the terms and conditions for use, reproduction, and distribution as defined by Sections 1 through 9 of this document.

"Licensor" shall mean the copyright owner or entity authorized by the copyright owner that is granting the License.

"Legal Entity" shall mean the union of the acting entity and all other entities that control, are controlled by, or are under common control with that entity. For the purposes of this definition, "control" means (i) the power, direct or indirect, to cause the direction or management of such entity, whether by contract or otherwise, or (ii) ownership of fifty percent (50%) or more of the outstanding shares, or (iii) beneficial ownership of such entity.

"You" (or "Your") shall mean an individual or Legal Entity exercising permissions granted by this License.

"Source" form shall mean the preferred form for making modifications, including but not limited to software source code, documentation source, and configuration files.

"Object" form shall mean any form resulting from mechanical transformation or translation of a Source form, including but not limited to compiled object code, generated documentation, and conversions to other media types.

"Work" shall mean the work of authorship, whether in Source or Object form, made available under the License, as indicated by a copyright notice that is included in or attached to the work (an example is provided in the Appendix below).

"Derivative Works" shall mean any work, whether in Source or Object form, that is based on (or derived from) the Work and for which the editorial revisions, annotations, elaborations, or other modifications represent, as a whole, an original work of authorship. For the purposes of this License, Derivative Works shall not include works that remain separable from, or merely link (or bind by name) to the interfaces of, the Work and Derivative Works thereof.

"Contribution" shall mean any work of authorship, including the original version of the Work and any modifications or additions to that Work or Derivative Works thereof, that is intentionally submitted to Licensor for inclusion in the Work by the copyright owner or by an individual or Legal Entity authorized to submit on behalf of the copyright owner. For the purposes of this definition, "submitted" means any form of electronic, verbal, or written communication sent to the Licensor or its representatives, including but not limited to communication on electronic mailing lists, source code control systems, and issue tracking systems that are managed by, or on behalf of, the Licensor for the purpose of discussing and improving the Work, but excluding communication that is conspicuously marked or otherwise designated in writing by the copyright owner as "Not a Contribution."

"Contributor" shall mean Licensor and any individual or Legal Entity on behalf of whom a Contribution has been received by Licensor and subsequently incorporated within the Work.

2. Grant of Copyright License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable copyright license to reproduce, prepare Derivative Works of, publicly display, publicly perform, sublicense, and distribute the Work and such Derivative Works in Source or Object form.

3. Grant of Patent License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except as stated in this section) patent license to make, have made, use, offer to sell, sell, import, and otherwise transfer the Work, where such license applies only to those patent claims licensable by such Contributor that are necessarily infringed by their Contribution(s) alone or by combination of their Contribution(s) with the Work to which

such Contribution(s) was submitted. If You institute patent litigation against any entity (including a cross-claim or counterclaim in a lawsuit) alleging that the Work or a Contribution incorporated within the Work constitutes direct or contributory patent infringement, then any patent licenses granted to You under this License for that Work shall terminate as of the date such litigation is filed.

4. Redistribution. You may reproduce and distribute copies of the Work or Derivative Works thereof in any medium, with or without modifications, and in Source or Object form, provided that You meet the following conditions:

(a) You must give any other recipients of the Work or Derivative Works a copy of this License; and

(b) You must cause any modified files to carry prominent notices stating that You changed the files; and

(c) You must retain, in the Source form of any Derivative Works that You distribute, all copyright, patent, trademark, and attribution notices from the Source form of the Work, excluding those notices that do not pertain to any part of the Derivative Works; and

(d) If the Work includes a "NOTICE" text file as part of its distribution, then any Derivative Works that You distribute must include a readable copy of the attribution notices contained within such NOTICE file, excluding those notices that do not pertain to any part of the Derivative Works, in at least one of the following places: within a NOTICE text file distributed as part of the Derivative Works; within the Source form or documentation, if provided along with the Derivative Works; or, within a display generated by the Derivative Works, if and wherever such third-party notices normally appear. The contents of the NOTICE file are for informational purposes only and do not modify the License. You may add Your own attribution notices within Derivative Works that You distribute, alongside or as an addendum to the NOTICE text from the Work, provided that such additional attribution notices cannot be construed as modifying the License.

You may add Your own copyright statement to Your modifications and may provide additional or different license terms and conditions for use, reproduction, or distribution of Your modifications, or for any such Derivative Works as a whole, provided Your use, reproduction, and distribution of the Work otherwise complies with the conditions stated in this License.

5. Submission of Contributions. Unless You explicitly state otherwise, any Contribution intentionally submitted for inclusion in the Work by You to the Licensor shall be under the terms and conditions of this License, without any additional terms or conditions. Notwithstanding the above, nothing herein shall supersede or modify the terms of any

separate license agreement you may have executed with Licensor regarding such Contributions.

6. Trademarks. This License does not grant permission to use the trade names, trademarks, service marks, or product names of the Licensor, except as required for reasonable and customary use in describing the origin of the Work and reproducing the content of the NOTICE file.

7. Disclaimer of Warranty. Unless required by applicable law or agreed to in writing, Licensor provides the Work (and each Contributor provides its Contributions) on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied, including, without limitation, any warranties or conditions of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A PARTICULAR PURPOSE. You are solely responsible for determining the appropriateness of using or redistributing the Work and assume any risks associated with Your exercise of permissions under this License.

8. Limitation of Liability. In no event and under no legal theory, whether in tort (including negligence), contract, or otherwise, unless required by applicable law (such as deliberate and grossly negligent acts) or agreed to in writing, shall any Contributor be liable to You for damages, including any direct, indirect, special, incidental, or consequential damages of any character arising as a result of this License or out of the use or inability to use the Work (including but not limited to damages for loss of goodwill, work stoppage, computer failure or malfunction, or any and all other commercial damages or losses), even if such Contributor has been advised of the possibility of such damages.

9. Accepting Warranty or Additional Liability. While redistributing the Work or Derivative Works thereof, You may choose to offer, and charge a fee for, acceptance of support, warranty, indemnity, or other liability obligations and/or rights consistent with this License. However, in accepting such obligations, You may act only on Your own behalf and on Your sole responsibility, not on behalf of any other Contributor, and only if You agree to indemnify, defend, and hold each Contributor harmless for any liability incurred by, or claims asserted against, such Contributor by reason of your accepting any such warranty or additional liability.

END OF TERMS AND CONDITIONS

APPENDIX: How to apply the Apache License to your work.

To apply the Apache License to your work, attach the following boilerplate notice, with the fields enclosed by brackets "[]" replaced with your own identifying information. (Don't include the brackets!) The text should be enclosed in the appropriate comment syntax for the file format. We also recommend that a file or class name and description of purpose be

included on the same "printed page" as the copyright notice for easier identification within third-party archives.

Copyright [yyyy] [name of copyright owner]

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at

<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

Sources and links

[S1] Erlang/OTP Project. Erlang/OTP. OTP-29.0. <https://github.com/erlang/otp>

[S2] Apache Software Foundation. Apache License, Version 2.0. January 2004.
<https://www.apache.org/licenses/LICENSE-2.0>

References

Entries are ordered by the first author's surname or the organisation's name, then by year.

- [\[DragonBook\]](#) Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Second edition, Addison-Wesley, 2006. ISBN 978-0-321-48681-3. Book website[\[S1\]](#).
- [\[Armstrong2003\]](#) Joe Armstrong. *Making Reliable Distributed Systems in the Presence of Software Errors*. Doctoral dissertation, Royal Institute of Technology (KTH), 2003. Dissertation[\[S2\]](#).
- [\[Armstrong2007\]](#) Joe Armstrong. “A History of Erlang.” *Proceedings of the Third ACM SIGPLAN Conference on History of Programming Languages (HOPL III)*, 2007, pp. 6-1–6-26. DOI[\[S3\]](#).
- [\[CoreErlang2000\]](#) Richard Carlsson, Björn Gustavsson, Erik Johansson, Thomas Lindgren, Sven-Olof Nyström, Mikael Pettersson, and Robert Virding. *Core Erlang 1.0 Language Specification*. Technical Report 2000-030, Department of Information Technology, Uppsala University, 2000. Institutional record[\[S4\]](#).
- [\[Cheney1970\]](#) C. J. Cheney. “A Nonrecursive List Compacting Algorithm.” *Communications of the ACM* 13(11), 1970, pp. 677–678. DOI[\[S5\]](#).
- [\[CytronEtAl1991\]](#) Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. “Efficiently Computing Static Single Assignment Form and the Control Dependence Graph.” *ACM Transactions on Programming Languages and Systems* 13(4), 1991, pp. 451–490. DOI[\[S6\]](#).
- [\[BeamAsmOTP29\]](#) Ericsson AB. *BeamAsm, the Erlang JIT*. Erlang/OTP 29 ERTS documentation. Documentation[\[S7\]](#). The versioned documentation follows the OTP 29 patch series.
- [\[OTP29Source\]](#) Erlang/OTP Project. *Erlang/OTP 29.0 Source Code*. Release tag OTP-29.0, 2026. Source tree[\[S8\]](#).
- [\[Gustavsson2018\]](#) Björn Gustavsson. “A Brief History of the BEAM Compiler.” Erlang/OTP blog, June 18, 2018. Article[\[S9\]](#).
- [\[Hogberg2020\]](#) John Högborg. “A First Look at the JIT.” Erlang/OTP blog, November 3, 2020. Article[\[S10\]](#).

- [\[JohanssonJonsson1996\]](#) Erik Johansson and Christer Jonsson. “Native Code Compilation for Erlang.” Uppsala Master’s Theses in Computing Science 100, Uppsala University, October 1996. ISSN 1100-1836.
- [\[Johansson1999\]](#) Erik Johansson. “Performance Measurements and Process Optimization for Erlang.” Licentiate thesis, Uppsala Theses in Computing Science 32, Uppsala University, 1999. ISSN 0283-359X.
- [\[JohanssonEtAl2002\]](#) Erik Johansson, Konstantinos Sagonas, and Jesper Wilhelmsson. “Heap Architectures for Concurrent Languages Using Message Passing.” *Proceedings of the Third International Symposium on Memory Management (ISMM)*, 2002, pp. 195–206. DOI[\[S11\]](#).
- [\[KlafteneggerEtAl2013\]](#) David Klaftenegger, Konstantinos Sagonas, and Kjell Winblad. “On the Scalability of the Erlang Term Storage.” *Proceedings of the Twelfth ACM SIGPLAN Workshop on Erlang*, 2013, pp. 15–26. DOI[\[S12\]](#).
- [\[LindahlSagonas2006\]](#) Tobias Lindahl and Konstantinos Sagonas. “Practical Type Inference Based on Success Typings.” *Proceedings of the Eighth ACM SIGPLAN International Conference on Principles and Practice of Declarative Programming (PPDP)*, 2006, pp. 167–178. DOI[\[S13\]](#).
- [\[Pettersson2000\]](#) Mikael Pettersson. *A Staged Tag Scheme for Erlang*. Technical Report 2000-029, Department of Information Technology, Uppsala University, 2000. Institutional record[\[S14\]](#).
- [\[PolettoSarkar1999\]](#) Massimiliano Poletto and Vivek Sarkar. “Linear Scan Register Allocation.” *ACM Transactions on Programming Languages and Systems* 21(5), 1999, pp. 895–913. DOI[\[S15\]](#).
- [\[HiPE2003\]](#) Konstantinos Sagonas, Mikael Pettersson, Richard Carlsson, Per Gustafsson, and Tobias Lindahl. “All You Wanted to Know about the HiPE Compiler (but Might Have Been Afraid to Ask).” *Proceedings of the 2003 ACM SIGPLAN Workshop on Erlang*, pp. 36–42. DOI[\[S16\]](#).
- [\[SagonasWilhelmsson2006\]](#) Konstantinos Sagonas and Jesper Wilhelmsson. “Efficient Memory Management for Concurrent Programs That Use Message Passing.” *Science of Computer Programming* 62(2), 2006, pp. 98–121. DOI[\[S17\]](#).
- [\[SagonasWinblad2014\]](#) Konstantinos Sagonas and Kjell Winblad. “More Scalable Ordered Set for ETS Using Adaptation.” *Proceedings of the Thirteenth ACM SIGPLAN Workshop on Erlang*, 2014, pp. 3–11. DOI[\[S18\]](#).

- [Stenman2002] Erik Stenman. *Efficient Implementation of Concurrent Programming Languages*. Doctoral dissertation, Uppsala University, 2002. Acta Universitatis Upsaliensis, Uppsala Dissertations from the Faculty of Science and Technology 43. ISBN 91-554-5422-4. Dissertation[S19].
- [TrinderEtAl2017] Phil Trinder et al. “Scaling Reliably: Improving the Scalability of the Erlang Distributed Actor Platform.” *ACM Transactions on Programming Languages and Systems* 39(4), Article 17, 2017, pp. 1–46. DOI[S20]. Institutional record and accepted manuscript[S21].
- [Virding1995] Robert Virding. “A Garbage Collector for the Concurrent Real-Time Language Erlang.” *Memory Management*, IWMM 1995, Lecture Notes in Computer Science 986, Springer, pp. 343–354. DOI[S22].
- [Warren] David H. D. Warren. *An Abstract Prolog Instruction Set*. Technical Note 309, SRI International, October 1983. Report[S23].
- [Wilson1992] Paul R. Wilson. “Uniprocessor Garbage Collection Techniques.” *Memory Management*, IWMM 1992, Lecture Notes in Computer Science 637, Springer, pp. 1–42. DOI[S24].

Sources and links

[S1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Second edition, Addison-Wesley, 2006; ISBN 978-0-321-48681-3. <https://suif.stanford.edu/dragonbook/>

[S2] Joe Armstrong. *Making Reliable Distributed Systems in the Presence of Software Errors*. Doctoral dissertation, KTH, 2003. <https://kth.diva-portal.org/smash/get/diva2:9492/FULLTEXT01.pdf>

[S3] Joe Armstrong. A History of Erlang. HOPL III, 2007, pp. 6-1–6-26. <https://doi.org/10.1145/1238844.1238850>

[S4] Richard Carlsson, Björn Gustavsson, Erik Johansson, Thomas Lindgren, Sven-Olof Nyström, Mikael Pettersson, and Robert Virding. *Core Erlang 1.0 Language Specification*. Uppsala University, Technical Report 2000-030, 2000. <https://urn.kb.se/resolve?urn=urn:nbn:se:uu:diva-484586>

[S5] C. J. Cheney. A Nonrecursive List Compacting Algorithm. *Communications of the ACM* 13(11), 1970, pp. 677–678. <https://doi.org/10.1145/362790.362798>

[S6] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. Efficiently Computing Static Single Assignment Form and the Control Dependence Graph. *ACM TOPLAS* 13(4), 1991, pp. 451–490. <https://doi.org/10.1145/115372.115320>

[S7] Erlang/OTP Project. BeamAsm, the Erlang JIT. Erlang/OTP 29 documentation. <https://www.erlang.org/docs/29/apps/erts/beamasm.html>

[S8] Erlang/OTP Project. Erlang/OTP. OTP-29.0. <https://github.com/erlang/otp>

[S9] Björn Gustavsson. A Brief History of the BEAM Compiler. Erlang/OTP blog, June 18, 2018. <https://www.erlang.org/blog/beam-compiler-history/>

[S10] John Högberg. A First Look at the JIT. Erlang/OTP blog, November 3, 2020. <https://www.erlang.org/blog/a-first-look-at-the-jit/>

- [S11] Erik Johansson, Konstantinos Sagonas, and Jesper Wilhelmsson. Heap Architectures for Concurrent Languages Using Message Passing. ISMM 2002, pp. 195–206. <https://doi.org/10.1145/512438.512440>
- [S12] David Klaftenegger, Konstantinos Sagonas, and Kjell Winblad. On the Scalability of the Erlang Term Storage. Erlang Workshop 2013, pp. 15–26. <https://doi.org/10.1145/2505305.2505308>
- [S13] Tobias Lindahl and Konstantinos Sagonas. Practical Type Inference Based on Success Typings. PPDP 2006, pp. 167–178. <https://doi.org/10.1145/1140335.1140356>
- [S14] Mikael Pettersson. A Staged Tag Scheme for Erlang. Uppsala University, Technical Report 2000-029, 2000. <https://urn.kb.se/resolve?urn=urn:nbn:se:uu:diva-484585>
- [S15] Massimiliano Poletto and Vivek Sarkar. Linear Scan Register Allocation. ACM TOPLAS 21(5), 1999, pp. 895–913. <https://doi.org/10.1145/330249.330250>
- [S16] Konstantinos Sagonas, Mikael Pettersson, Richard Carlsson, Per Gustafsson, and Tobias Lindahl. All You Wanted to Know about the HiPE Compiler (but Might Have Been Afraid to Ask). Erlang Workshop 2003, pp. 36–42. <https://doi.org/10.1145/940880.940886>
- [S17] Konstantinos Sagonas and Jesper Wilhelmsson. Efficient Memory Management for Concurrent Programs That Use Message Passing. Science of Computer Programming 62(2), 2006, pp. 98–121. <https://doi.org/10.1016/j.scico.2006.02.006>
- [S18] Konstantinos Sagonas and Kjell Winblad. More Scalable Ordered Set for ETS Using Adaptation. Erlang Workshop 2014, pp. 3–11. <https://doi.org/10.1145/2633448.2633455>
- [S19] Erik Stenman. Efficient Implementation of Concurrent Programming Languages. Doctoral dissertation, Uppsala University, 2002; ISBN 91-554-5422-4. <https://www.diva-portal.org/smash/get/diva2:162047/FULLTEXT01.pdf>
- [S20] Phil Trinder et al.. Scaling Reliably: Improving the Scalability of the Erlang Distributed Actor Platform. ACM TOPLAS 39(4), Article 17, 2017, pp. 1–46. <https://doi.org/10.1145/3107937>
- [S21] Phil Trinder et al.. Scaling Reliably: Improving the Scalability of the Erlang Distributed Actor Platform. ACM TOPLAS 39(4), Article 17, 2017, pp. 1–46. <https://eprints.gla.ac.uk/143232/>
- [S22] Robert Virding. A Garbage Collector for the Concurrent Real-Time Language Erlang. IWMM 1995, LNCS 986, pp. 343–354. https://doi.org/10.1007/3-540-60368-9_33
- [S23] David H. D. Warren. An Abstract Prolog Instruction Set. SRI International, Technical Note 309, October 1983. <https://www.sri.com/wp-content/uploads/2021/12/641.pdf>
- [S24] Paul R. Wilson. Uniprocessor Garbage Collection Techniques. IWMM 1992, LNCS 637, pp. 1–42. <https://doi.org/10.1007/BFb0017182>